

BASI DI DATI

Kristian Xhani

Ottobre 2025

Indice

1	Introduzione	4
1.1	Problemi e soluzione	4
1.2	DBMS	4
1.3	Architettura di un DBMS	5
1.4	Indipendenza dei dati	5
2	Progettazione di una base di dati	6
2.1	Cos'è una metodologia di progettazione	7
2.2	Caratteristiche di una buona metodologia	7
3	Progettazione Concettuale : il Modello ER	9
3.1	Cos'è il modello ER	9
3.2	Costrutti del modello ER	9
3.3	Entità	9
3.4	Relazione	9
3.4.1	Relazione Ricorsiva	11
3.4.2	Relazione Ternaria	11
3.4.3	Indizi di modellazione errata (uso improprio della relazione ternaria) . . .	12
3.4.4	Esempio corretto di relazione ternaria	12
3.4.5	Come capire se un concetto ha vita propria	13
3.5	Attributo	13
3.5.1	Attributo opzionale e multivalore	14
3.5.2	Attributo composto	15
3.6	Identificatore di Entità	16
3.7	Generalizzazione	17
3.7.1	Classificazione delle Generalizzazioni	18
4	Strategie per la progettazione concettuale	20
4.1	Strategia Top-Down	20
4.2	Strategia Bottom-Up	20
4.3	Strategia Inside-Out	21
4.4	Analisi di qualità dello schema	21
4.5	Ridondanze	22
4.6	Semplificazioni dello schema	23
5	Modello Relazionale	25
5.1	Costrutti Principali del Modello Relazionale	25
5.2	Domini di Base	25
5.3	Costrutto Relazionale	26

5.3.1	Definizione: Relazione come insieme di ennuple (LISTS)	26
5.3.2	Definizione RELAZIONE come insieme di tuple (MAPPINGS)	27
5.4	Come progettare una basi di dati relazionale	27
5.4.1	Terminologia di schema di una relazione	30
5.4.2	Terminologia Schema di una base di dati	30
5.4.3	Terminologia istanza di una relazione di schema $R(X)$	30
5.4.4	Terminologia istanza di una base di dati di schema $S = \{R_1(X_1), \dots, R_n(X_n)\}$	30
5.5	Valori nulli	30
5.6	Vincoli di integrità	31
5.7	Esempi di vincoli	32
5.8	Classificazione vincoli di integrità	32
5.9	Notazione per indicare i vincoli strutturali	34
5.9.1	Chiave primaria	34
5.9.2	Vincolo di integrità referenziale	34
5.10	Esercizio	35
5.11	Progettazione Logica	36
5.11.1	Analisi della derivabilità	36
5.11.2	Analisi delle ridondanze	37
5.11.3	Eliminazione delle Generalizzazioni	37
5.11.4	Accorpamento delle entità figlie nel padre	37
5.11.5	Accorpamento del padre nelle entità figlie	38
5.11.6	Sostituzione con relazioni	39
5.12	Traduzione verso il modello relazionale	40
5.13	Classificazione delle relazioni binarie	41
5.14	Regole di traduzione	41
6	Tecnologie NoSQL: Sistemi Document-Based	48
6.1	Approcci NoSQL	49
6.2	Sistemi Document-Store	49
6.3	Regole di traduzione di uno schema ER in collezioni di documenti	50
6.3.1	Prima Fase (entità principali)	50
6.3.2	Seconda Fase (incapsulamento entità non principali)	51
6.3.3	Terza Fase (relazioni non strutturali)	53
6.4	Quarta fase	55
7	Algebra relazionale	61
7.1	Classificazione degli operatori	61
7.2	Operatori insiemistici	62
7.2.1	Cardinalità degli operatori insiemistici	62
7.3	Operatori specifici	63
7.3.1	Operatore di rinominaizione	63
7.3.2	Operatore di Selezione	64
7.3.3	Proiezione	65
7.3.4	Cardinalità degli operatori specifici	65
7.3.5	Operatori di giunzione (JOIN)	66
7.3.6	JOIN naturale	66
7.3.7	Cardinalità del JOIN naturale	68
7.3.8	THETA JOIN	69
7.4	Algebra con valori nulli	70
7.4.1	Selezione	71
7.4.2	Join naturale	71
7.5	Ottimizzazione di espressioni DML	71

7.6	Equivalenza tra espressioni algebriche	72
7.6.1	Equivalenza dipendente dallo schema	72
7.6.2	Equivalenza assoluta	72
7.7	Trasformazioni di equivalenza	72
7.7.1	Idempotenza delle proiezioni	72
7.8	Trasformazioni di equivalenza con il join	72
7.9	Combinazione di anticipazione e idempotenza	73
8	Calcolo Relazionale	77
8.1	Sintassi	78
8.2	Semantica	79
8.3	Calcolo relazionale sulle tuple	80
9	Linguaggi di interrogazione nei sistemi NoSql	85
10	Tecnologie - Transazioni	88
10.1	Tecnologia dei DBMS	88
10.2	Architettura su 3 livelli dei DBMS	88
10.3	Transazione	89
10.3.1	Proprietà delle transazioni	90
10.4	Architettura di un DBMS	91
11	Linguaggio di Interrogazione SQL	96
11.1	Clausola SELECT	97
11.2	Clausola FROM	99
11.3	Clausola WHERE	100
11.3.1	Operatore LIKE	102
11.3.2	Operatore SIMILAR TO	103
11.3.3	Operatore BETWEEN	103
11.3.4	Operatore IN	104
11.3.5	Operatore IS NULL	105
11.4	SQL e Algebra Relazionale	106
11.5	SELECT	107
11.5.1	SELECT : alias di tabella o di attributo	107
11.5.2	SELECT: senza proiezione	107
11.5.3	Proiezione: gestione dei duplicati in SQL	107
11.5.4	SELECT: Selezione, Proiezione, Join	109
11.6	Interrogazioni con JOIN	109

1 Introduzione

- **Sistema informativo:** è l'insieme delle attività umane e dei dispositivi di memorizzazione ed elaborazione che organizzano e gestiscono le informazioni di interesse per un'organizzazione, di qualunque dimensione. Un sistema informativo **non contiene necessariamente tecnologia informatica**.
- **Dato:** è un elemento grezzo, un fatto isolato o un valore che, preso da solo, non ha significato.
- **Informazione:** nasce quando un dato viene interpretato e collocato in un contesto che lo rende comprensibile e utile.

Ad esempio: il numero 37 da solo è un semplice dato. Se però diciamo che “37 °C è la temperatura corporea normale di un essere umano”, allora stiamo fornendo un'informazione.

Per capire meglio

Un dato è come un mattone: preso da solo non serve a molto. Quando i mattoni vengono messi insieme per costruire una casa, essi diventano informazione.

Per capire meglio come funziona un sistema informativo (cioè come circolano i dati e chi li utilizza) si utilizzano **diagrammi di flusso**, analizzando quattro aspetti principali:

1. Definizione di archivi e sorgenti di dati
 2. Definizione degli utenti
 3. Definizione di procedure e processi
 4. Definizione dei flussi di dati
- **Base di dati:** è una collezione di dati utilizzata per rappresentare, con l'ausilio della tecnologia informatica, le informazioni di interesse di un sistema informativo.

1.1 Problemi e soluzione

Negli anni '70 ogni programma aveva i propri file di dati separati, gestiti direttamente dal **File System**. Questa soluzione presentava alcuni problemi:

- Scarsa efficienza nell'accesso ai dati su file
- Ridondanza dei dati (duplicazione)
- Inconsistenza dei dati (informazioni non aggiornate ovunque)
- Progettazione dei dati replicata per ogni programma

Per risolvere questi limiti vennero introdotti i **DBMS (Database Management System)**: software che fanno da intermediari tra programmi e dati.

Vantaggi dei DBMS:

- Maggiore astrazione e potenza espressiva per descrivere i dati
- Operazioni complesse di accesso e interrogazione tramite linguaggi specifici

1.2 DBMS

- **Linguaggi di interazione:**
 - **DDL (Data Definition Language):** definisce la struttura del database.

- **DML (Data Manipulation Language)**: consente di interrogare e modificare i dati (es. SQL).

Differenza tra Database e DBMS

Database: è l'insieme organizzato dei dati. *Esempio:* un archivio con le informazioni sugli studenti di una scuola.

DBMS (Database Management System): è il software che permette di creare, gestire e consultare un database. *Esempi:* MySQL, PostgreSQL, Oracle, SQLite.

1.3 Architettura di un DBMS

- **Schema logico:** rappresentazione delle strutture e delle proprietà della base di dati, definita tramite i costrutti del modello del DBMS.
- **Schema interno:** rappresentazione della base di dati attraverso le strutture fisiche di memorizzazione.
- **Schema esterno:** descrive una porzione dello schema logico di interesse per uno specifico utente o applicazione.

1.4 Indipendenza dei dati

- **Indipendenza fisica:** lo schema logico della base di dati è indipendente dallo schema interno.
- **Indipendenza logica:** gli schemi esterni della base di dati sono indipendenti dallo schema logico.

2 Progettazione di una base di dati

Il ciclo di vita del processo di automazione di un sistema informativo comprende diverse fasi fondamentali, che vanno dall'analisi iniziale fino alla progettazione vera e propria.

1. Studio di fattibilità

In questa fase si valuta se il progetto è **realizzabile e conveniente**. Si analizzano i **costi**, i **tempi** e le **alternative possibili**. L'obiettivo è decidere se vale la pena procedere con lo sviluppo del sistema informativo. *È come chiedersi: "Ha senso costruire questo sistema?"*

2. Raccolta e analisi dei requisiti

In questa fase si cercano di capire i **bisogni reali degli utenti** e cosa dovrà fare il sistema. Si individuano le **proprietà e le funzionalità** principali (dati e applicazioni), producendo una descrizione **completa ma informale**. *È come chiedersi: "Cosa deve fare il sistema per essere utile?"*

3. Progettazione

Una volta compresi i requisiti, si passa alla fase di **progettazione del database e delle applicazioni**. Si decide come saranno organizzati i **dati** (modello ER, tabelle, relazioni) e come le applicazioni interagiranno con essi. *È la fase in cui si costruisce il progetto su carta, prima di realizzarlo.*

4. Progettazione del sistema

Una volta definiti i requisiti, si passa alla **progettazione del sistema**, che si divide in due parti principali:

- **Progettazione dei dati:** descrive in modo formale come i dati saranno organizzati, ovvero la struttura logica del database. Il risultato è lo **schema** del database (modello concettuale e logico).
- **Progettazione delle applicazioni:** definisce come gli utenti potranno interagire con i dati. Il risultato è la **specifica delle applicazioni**, cioè le operazioni e le funzionalità del sistema.

In sintesi: si progetta sia la parte "tecnica dei dati", sia la parte "funzionale" che li utilizza.

5. Implementazione e collaudo

Dopo la progettazione, si passa alla fase di **realizzazione concreta**:

- **Implementazione su un DBMS:** lo schema del database viene tradotto e caricato in un sistema di gestione (DBMS). Si inseriscono i dati e si realizzano le applicazioni.
- **Validazione e collaudo:** si verifica che il sistema funzioni correttamente e rispetti i requisiti definiti nelle fasi precedenti. Eventuali errori o mancanze vengono corretti prima dell'utilizzo effettivo.

In questa fase il progetto diventa finalmente operativo e utilizzabile.

6. Progettazione dei dati

In questa fase si realizza la **struttura logica del database**. Si definiscono:

- le **entità**, gli **attributi** e le **relazioni** tra di essi;
- le **chiavi primarie** e le **chiavi esterne**;
- i vincoli di integrità che garantiscono la coerenza dei dati.

Il risultato è uno **schema completo del database**, pronto per essere implementato.

7. Implementazione su un DBMS

Qui il progetto diventa concreto: lo schema logico del database viene **tradotto in un linguaggio SQL** e installato in un **DBMS** (Database Management System).

- Si creano le tabelle e le relazioni.
- Si inseriscono i dati iniziali.
- Si configurano le applicazioni che interagiranno con il database.

È la fase di costruzione reale del sistema.

8. Validazione e collaudo

Ultima fase del ciclo: si verifica che il sistema funzioni in modo corretto.

- Si confronta il risultato con i requisiti iniziali.
- Si testano le funzionalità e le prestazioni.
- Si correggono eventuali errori o incongruenze.

Solo dopo la validazione, il sistema è **pronto per l'uso reale** da parte degli utenti.

2.1 Cos'è una metodologia di progettazione

Una **metodologia di progettazione** definisce il modo in cui si affronta e si organizza il progetto di un database. Serve a garantire che il lavoro sia ordinato, coerente e ripetibile, evitando errori e migliorando la qualità del risultato finale.

Una metodologia di progettazione è costituita da:

- una **decomposizione** in passi dell'attività di progetto, che suddivide il lavoro in fasi chiare e gestibili;
- un insieme di **strategie** da seguire e di **criteri di scelta**, che guidano le decisioni del progettista;
- un insieme di **modelli di riferimento**, che forniscono strutture e standard per rappresentare correttamente i dati.

2.2 Caratteristiche di una buona metodologia

Una buona metodologia deve essere:

- **Generale**, ovvero applicabile a diversi tipi di progetti e contesti;
- **Facile da usare**, in modo che i progettisti possano adottarla senza difficoltà;
- **Efficace**, cioè in grado di produrre un risultato di qualità, con un progetto **completo, coerente e corretto**.

In sintesi, una metodologia ben definita aiuta a costruire database **ben strutturati, comprensibili e facilmente mantenibili nel tempo**.

Per essere una buona metodologia deve essere:

- Generale
- Facile da Usare
- In grado di produrre un risultato di qualità
- **Progettazione concettuale:** L'obiettivo della progettazione concettuale è quello di rappresentare in modo chiaro, formale e completo le informazioni che dovranno essere memorizzate nel database, senza ancora preoccuparsi di come verranno salvate in un sistema reale. In questa fase:
 - si analizza cosa deve contenere la base di dati (le entità, i loro attributi e le relazioni tra di esse);
 - si realizza un modello concettuale, di solito attraverso il diagramma Entità-Relazione (E-R);
 - si mantiene tutto indipendente dalla tecnologia, cioè senza pensare al tipo di DBMS o al linguaggio che verrà usato in seguito.
- **Progettazione logica :** il suo obiettivo è quello di tradurre lo schema concettuale nello schema logico aderente al modello dei dati del DBMS scelto per l'implementazione.
- **Progettazione fisica :** Completare lo schema logico con i parametri relativi alla memorizzazione fisica dei dati e con gli opportuni metodi d'accesso (INDICI) per garantire un accesso efficiente ai dati.
- **Schema Concettuale:** È la rappresentazione astratta dei dati e delle loro relazioni, realizzata durante la progettazione concettuale. Usa strumenti come il diagramma E-R (Entità-Relazione) e mostra cosa deve contenere la base di dati, senza dipendere dal tipo di DBMS.
- **Schema logico:** È la traduzione dello schema concettuale in una forma più tecnica, adatta al modello di database scelto (ad esempio, relazionale). Si decide come rappresentare le entità e le relazioni (tabelle, chiavi primarie, chiavi esterne).
- **Schema fisico:** È la realizzazione concreta del database all'interno di un DBMS specifico. Si definiscono i nomi delle tabelle, i tipi di dato (VARCHAR, INT, DATE...), gli indici, i vincoli e l'organizzazione fisica dei file.

Differenza tra Progettazione e Schema

Progettazione → è il **processo**, cioè l'attività di analisi e definizione che porta alla costruzione del modello dei dati. In questa fase si studiano le informazioni, si identificano le entità e le relazioni e si costruisce il modello concettuale.

Schema → è il **risultato finale** della progettazione. Rappresenta in modo formale e sintetico la struttura dei dati ottenuta al termine del processo.

In sintesi: la **progettazione** è l'attività di creazione, mentre lo **schema** è il prodotto che ne deriva.

3 Progettazione Concettuale : il Modello ER

3.1 Cos'è il modello ER

Il **modello Entità-Relazione (ER)** è un linguaggio visivo usato per rappresentare i dati e comprendere come essi siano collegati tra loro. Viene utilizzato soprattutto nella fase iniziale di progettazione di un database, per avere un'idea chiara di quali informazioni siano necessarie e di come esse siano organizzate. Esso è un linguaggio formale e non ambiguo.

3.2 Costrutti del modello ER

Gli strumenti fondamentali del modello ER vengono definiti come **costrutti**. Ogni costrutto viene descritto specificando:

- il **significato**;
- la **sintassi grafica**;
- la **rappresentazione delle sue istanze**.

Cos'è un'istanza

Un'**istanza** è un singolo elemento concreto che appartiene a un'entità.

- L'**entità** descrive il concetto generale (es. *Studente*);
- L'**istanza** è un esempio reale di quell'entità (es. *Mario Rossi*, matricola S123).

Esempio:

- Entità: *Studente* $\rightarrow$ attributi: matricola, nome, cognome;
- Istanza: {Matricola: S123, Nome: Luca, Cognome: Rossi}.

In sintesi: l'entità è il modello generale, mentre l'istanza è un caso reale e specifico di quell'entità.

3.3 Entità

- **Significato:** un'entità E rappresenta una classe di oggetti con le seguenti caratteristiche:
 - hanno proprietà comuni;
 - hanno esistenza autonoma;
 - hanno identificazione univoca (esiste una chiara corrispondenza tra gli oggetti istanze di E e i concetti istanziati nel sistema informativo).
- **Sintassi grafica:** un'entità E si rappresenta nello schema con un rettangolo che contiene il nome dell'entità.
- **Istanza:** un'istanza dell'entità E è un oggetto appartenente alla classe rappresentata da E . Si indica con $I(E)$ l'insieme delle istanze di E che esistono nella base di dati in un certo istante. (come definito nel riquadro precedente).
- **Esempio:** rappresento con il costrutto entità il concetto di **PERSONA**, ipotizzando che nei requisiti sia indicata la necessità di gestire nella base di dati le informazioni che descrivono un gruppo di persone e che tali informazioni possiedano nel sistema informativo le caratteristiche indicate nella semantica del costrutto entità.

3.4 Relazione

- **Significato:** una relazione R rappresenta un **legame logico** tra due o più entità. Possono esistere relazioni:

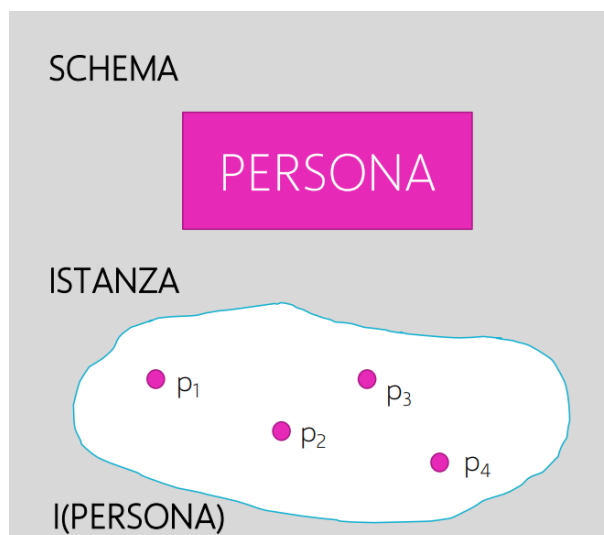


Figura 1: Esempio di Entita

- **binarie**, quando coinvolgono due entità diverse;
- **ternarie**, quando coinvolgono tre entità;
- **ennarie**, quando coinvolgono n entità.

Caso particolare: una relazione binaria può essere definita anche sulla stessa entità (detta **relazione ricorsiva**), ad esempio la relazione “*impiegato-supervisore*” all’interno dell’entità *Dipendente*.

- **Sintassi grafica:** una relazione R si rappresenta nello schema con un **rombo** collegato, tramite linee spezzate, alle entità coinvolte nella relazione. Il nome della relazione viene scritto all’interno o accanto al rombo.
- **Istanza:** data una relazione R tra n entità $E_1, E_2, \dots, E_n$, un’istanza della relazione è una **ennupla** di istanze delle entità coinvolte. L’insieme di tutte le ennuple costituisce la **popolazione della relazione**.

In altre parole, le ennuple rappresentano i **collegamenti concreti** tra le istanze delle entità. Se immaginiamo la relazione come una tabella, ciascuna riga (ennupla) contiene i riferimenti alle istanze delle due entità collegate.

Esempio: Sia $E = \text{Studente}$ e $F = \text{Corso}$, e sia $R = \text{Iscrizione}$. Un’istanza della relazione può essere l’ennupla (Mario Rossi, Analisi 1), che rappresenta il fatto che lo studente “Mario Rossi” è iscritto al corso “Analisi 1”. L’insieme di tutte le ennuple del tipo (Studente, Corso) forma la popolazione della relazione *Iscrizione*.

- **Esempio :** Supponiamo di aver inserito nello schema le entità *PERSONA* e *COMUNE*, e ipotizziamo che nei requisiti sia indicata la necessità di gestire la *RESIDENZA* delle persone nei comuni italiani. Rappresento con il costrutto relazione il concetto di *RESIDENZA*

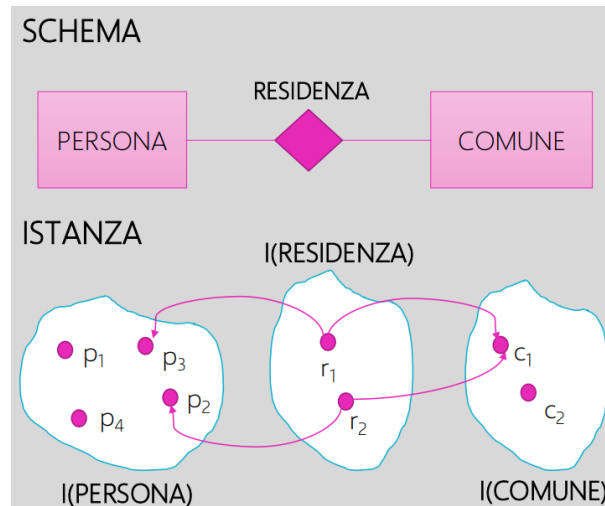


Figura 2: Esempio di Relazione

3.4.1 Relazione Ricorsiva

E' una relazione binaria sulla stessa entità.

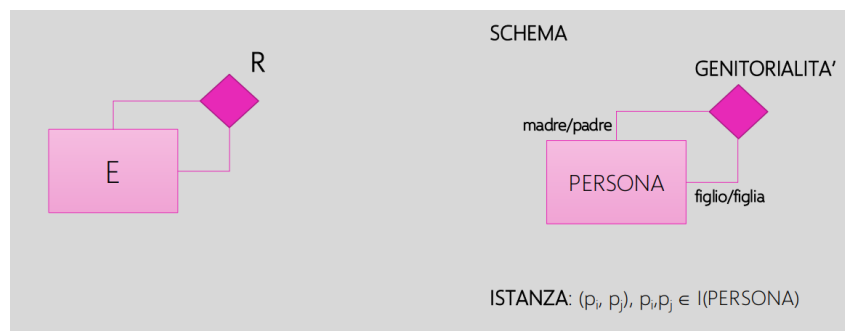
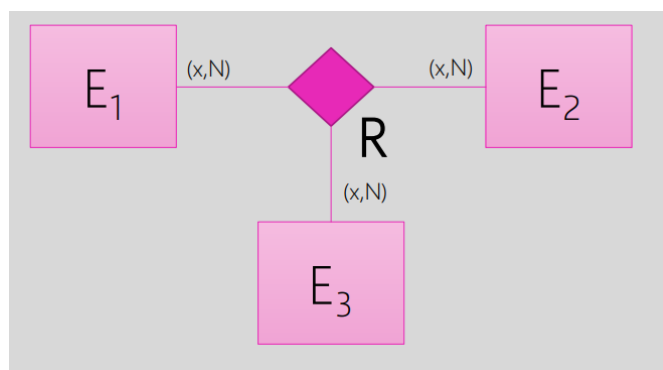


Figura 3: Relazione Ricorsiva

3.4.2 Relazione Ternaria

Una **relazione ternaria** coinvolge tre entità e viene utilizzata quando un concetto del sistema informativo può essere rappresentato solo se sono presenti contemporaneamente tre istanze, una per ciascuna entità coinvolta.

- Ogni istanza della relazione associa **una terna** di entità (E_1, E_2, E_3) .
- L'esistenza della relazione **richiede sempre** la presenza di tutte e tre le entità: non è possibile che ne manchi una.
- Non esistono casi in cui la stessa terna debba essere rappresentata più volte.



3.4.3 Indizi di modellazione errata (uso improprio della relazione ternaria)

In alcuni casi, l'utilizzo di una **relazione ternaria** può indicare una modellazione errata. Questo avviene quando una relazione tra tre entità non richiede realmente la presenza simultanea di tutte e tre le istanze.

- Se per rappresentare lo stato del sistema informativo sarebbe sufficiente collegare **solo due entità per volta**, la relazione ternaria è superflua.
- In tali casi, la relazione ternaria può essere sostituita da **tre relazioni binarie indipendenti**.

Esempio: La relazione ternaria *Partecipa(Medico, Paziente, Ospedale)* è errata se nel sistema esistono già le relazioni binarie: *Lavora_in(Medico, Ospedale)*, *Visita(Medico, Paziente)* e *Ricoverato_in(Paziente, Ospedale)*. In questo caso, la relazione ternaria non aggiunge alcuna informazione nuova ed è quindi ridondante. Un ulteriore indizio di modellazione errata si verifica quando, in una relazione ternaria $R(A, B, C)$, si osserva il seguente comportamento:

- Se una coppia (a, b) compare in una terna (a, b, c) istanza della relazione,
- e una coppia (b, c') compare in una terna (a', b, c') istanza della relazione,

allora risultano automaticamente presenti anche le terne (a, b, c') e (a', b, c) .

Questo significa che le combinazioni tra le entità non dipendono realmente tutte e tre tra loro: la relazione ternaria può quindi essere scomposta in più relazioni binarie indipendenti senza perdita di informazione. **In sintesi:** se da due terne si possono dedurre automaticamente altre combinazioni tra le stesse entità, la relazione ternaria è ridondante e andrebbe sostituita da più relazioni binarie.

Kristian semplifica in questo modo

Le **relazioni ternarie** vanno usate solo quando il significato della relazione dipende **strettamente da tutte e tre le entità coinvolte**.

- Se anche una sola entità può essere esclusa o rappresentata separatamente, allora la relazione non è veramente ternaria.
- Nella maggior parte dei casi, è meglio sostituire una relazione ternaria con **più relazioni binarie indipendenti**.

In sintesi: se puoi evitarla, **evitala**. Le relazioni ternarie rendono più difficile la progettazione e la traduzione nel modello relazionale.

3.4.4 Esempio corretto di relazione ternaria

Vogliamo rappresentare il fatto che un prodotto venga fornito ad un dipartimento da un fornitore. Ogni dipartimento può scegliere quali prodotti ordinare dai fornitori selezionando un sottoinsieme

dei prodotti forniti dal fornitore. Ogni fornitore vende a più dipartimenti prodotti diversi. Ogni prodotto viene venduto da più fornitori.

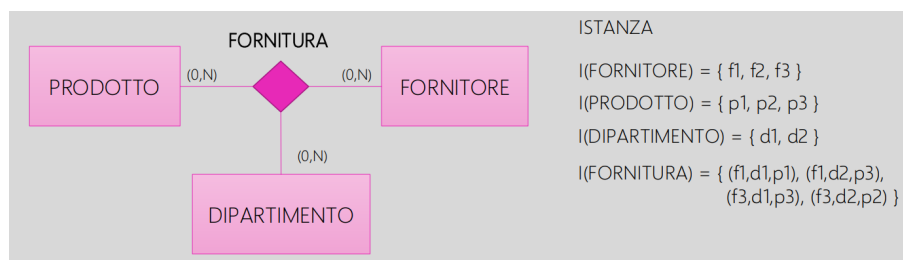


Figura 4: Esempio Relazione Ternaria

3.4.5 Come capire se un concetto ha vita propria

Nel modello Entità-Relazione, per stabilire se un concetto debba essere rappresentato come **entità** o come **relazione**, è fondamentale chiedersi se tale concetto *ha una vita propria* all'interno del sistema informativo.

Definizione

Un concetto ha **vita propria** se può esistere in modo indipendente dalle altre entità, cioè se ha senso considerarlo come un oggetto autonomo, anche quando non è collegato ad altri.

Per verificarlo, è possibile porsi alcune domande:

- Può esistere anche senza essere collegato ad altre entità?
- Ha senso registrarlo nel sistema anche se non è associato ad altri elementi?
- Possiede attributi che descrivono sé stesso e non il legame con altri?

Se la risposta è **sì** a queste domande, il concetto possiede vita propria e deve essere rappresentato come **entità**. In caso contrario, se esiste solo come collegamento tra altre entità, è più corretto rappresentarlo come **relazione**.

Caratteristica	Entità (ha vita propria)	Relazione (non ha vita propria)
Esiste indipendentemente da altre	Può esistere da sola nel sistema	Esiste solo se collega altre entità
Ha senso da sola	Ha significato anche isolata	Ha senso solo nel contesto di un legame
Attributi	Descrivono l'oggetto in sé	Descrivono il legame tra entità
Esempio	Studente, Insegnamento, Docenti	Sostiene, Frequenta, Acquista

Esempio applicativo: nel caso della relazione *ESAME*, se essa rappresenta l'atto con cui uno studente sostiene un insegnamento (registrando solo *data* e *voto*), non ha vita propria e va modellata come **relazione**. Se invece si vogliono gestire appelli d'esame con *aula*, *docente* e *orario*, esso assume significato autonomo e può essere rappresentato come **entità**.

3.5 Attributo

- **Significato** :Rappresenta una proprietà elementare di un'entità (o relazione). Ogni attributo di entità (o relazione) associa ad ogni istanza di entità (o relazione) UNO e UN

SOL valore appartenente ad un dominio (insieme di VALORI AMMISSIBILI).

- **Sintassi grafica** : collegato a una entità e a forma di pallino
- **Esempio** : Rappresento con il costrutto entità il concetto di PERSONA, ipotizzando che nei requisiti sia indicata la necessità di gestire nella base di dati il nome e il cognome di un gruppo di persone. Aggiungo quindi il nome e il cognome come attributi dell'entità PERSONA.

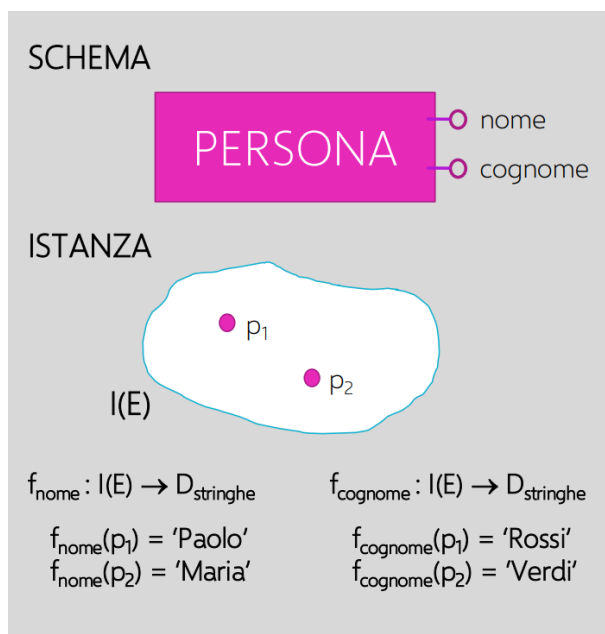


Figura 5: Esempio Attributo

3.5.1 Attributo opzionale e multivalore

Un **attributo opzionale** o **multivalore** deriva da un attributo normale, specificando un **vincolo di cardinalità** sui valori che esso può assumere. Il vincolo di cardinalità indica quante volte un attributo può comparire in corrispondenza di una stessa entità. Il valore predefinito è **(1,1)**, che significa “obbligatorio e singolo valore”.

I principali casi sono:

- **(0,1)** : attributo opzionale → può essere assente o presente una sola volta;
- **(1,N)** : attributo multivalore → deve essere presente almeno una volta e può comparire più volte;
- **(0,N)** : attributo opzionale e multivalore → può essere assente o, se presente, avere più valori.

Esempio pratico

Si consideri l'entità *PERSONA*, per la quale si vogliono memorizzare:

- **Codice fiscale, nome, cognome e data di nascita** — attributi obbligatori e a singolo valore;
- **Patente di guida** — attributo **opzionale** (0, 1), poiché una persona può possederla o meno;

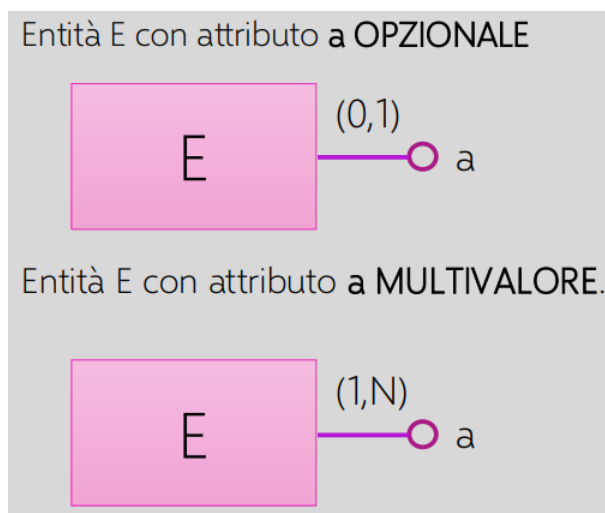


Figura 6: Attributo opzionale e/o multivalore

- **Recapiti telefonici** — attributo **multivalore** $(1, N)$, poiché ogni persona può avere uno o più numeri di telefono (ad esempio cellulare, casa, lavoro).

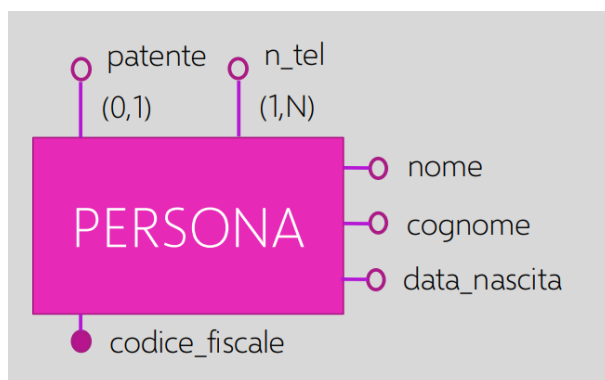


Figura 7: Esempio di attributo opzionale e multivalore per l'entità PERSONA

3.5.2 Attributo composto

Un **attributo composto** consente di raggruppare più attributi di un'entità o di una relazione che presentano una stretta **affinità di significato**. In questo modo si possono rappresentare in modo più ordinato e logico informazioni che, pur essendo distinte, appartengono a un unico concetto complessivo.

Ad esempio, per rappresentare l'entità *PERSONA*, si possono gestire nella base di dati i seguenti attributi: **codice fiscale**, **nome**, **cognome**, **data di nascita** e **indirizzo**.

L'attributo **indirizzo** può essere considerato un attributo composto, poiché a sua volta può essere suddiviso nei sottoattributi:

- **Città**
- **Via**
- **Numero civico**

In un diagramma ER, l'attributo composto viene rappresentato con un'ellisse principale collegata a più ellissi minori, ognuna delle quali rappresenta un sottoattributo.

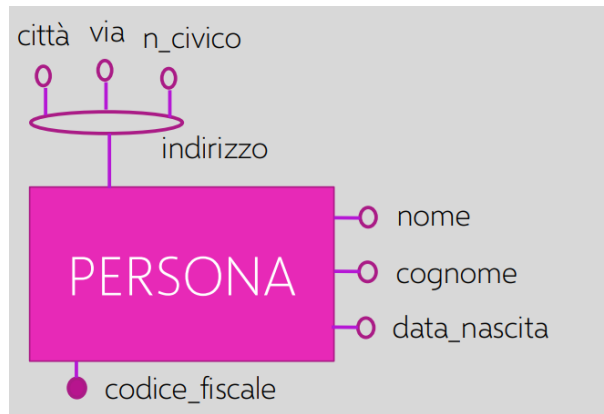


Figura 8: Esempio di attributo composto per l'entità PERSONA

3.6 Identificatore di Entità

- **Significato:** Data un'entità E, è un insieme di proprietà (attributi e/o relazioni) che identificano univocamente le istanze dell'entità. Un insieme di proprietà **IDENTIFICA UNIVOCAMENTE** le istanze di E se non esistono due istanze di E che presentano gli stessi valori/istanze nelle proprietà dell'insieme.
- **Sintassi grafica :** collegato a una entità e a forma di un pallino pieno, può anche essere una linea che intercorre in due attributi sempre a fine con un pallino pieno

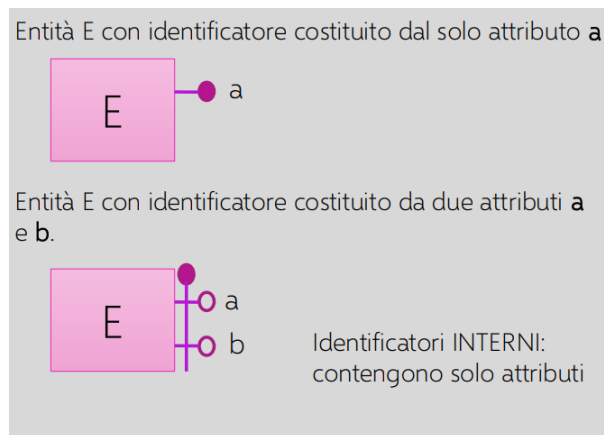


Figura 9: sintassi grafica: identificatori interni

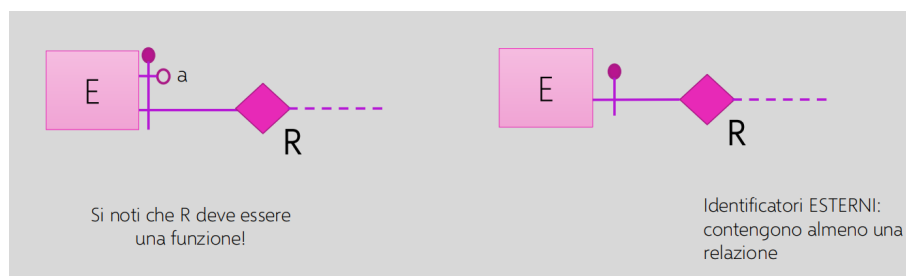
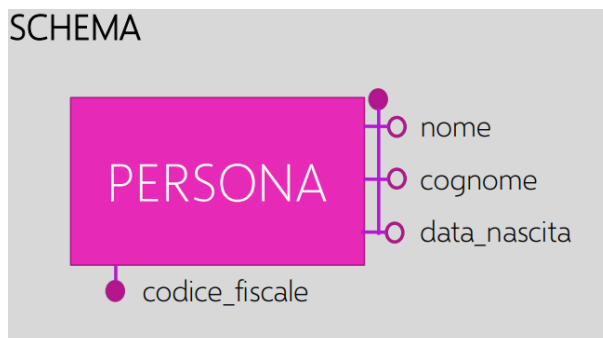


Figura 10: sintassi grafica: identificatori esterni

- **Esempio:** Rappresento con il costrutto entità il concetto di *PERSONA*, ipotizzando che nei requisiti sia indicata la necessità di gestire nella base di dati, il codice fiscale, il nome, il cognome e la data di nascita di un gruppo di persone.



La scelta dell'identificatore va sempre fatta considerando proprietà significative per il sistema informativo. A livello concettuale l'introduzione di nuovi attributi identificatori (ad esempio l'ID) è da **evitare**.

3.7 Generalizzazione

La **generalizzazione** rappresenta un legame logico, analogo al concetto di **ereditarietà** nella programmazione a oggetti, tra un'entità padre e una o più entità figlie. Serve per modellare situazioni in cui più entità condividono alcune caratteristiche comuni, ma possiedono anche attributi o relazioni specifiche.

- **Significato:** È una relazione logica tra un'entità padre E e n entità figlie $(E_1, E_2, \dots, E_n)$, dove E rappresenta una classe di oggetti più generale rispetto a quelle rappresentate dalle entità figlie. Le entità figlie ereditano gli attributi e le relazioni dell'entità padre, aggiungendo eventualmente elementi propri.
- **Sintassi grafica:** Le entità figlie coinvolte nella generalizzazione vengono collegate all'entità padre mediante una linea che converge in un triangolo (o freccia) rivolto verso l'entità padre.

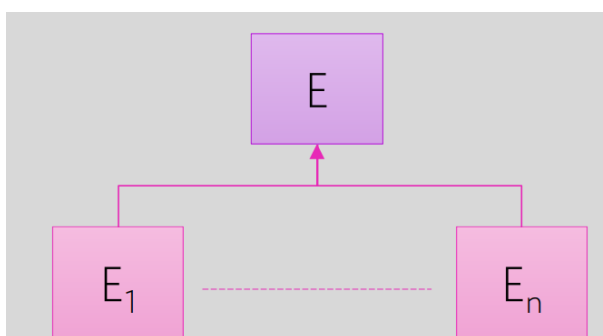


Figura 11: Sintassi grafica della generalizzazione

- **Esempio:** Si consideri l'entità *PERSONA*, per la quale nei requisiti è indicata la necessità di distinguere tra *STUDENTE* e *DOCENTE*. In questo caso si può utilizzare una generalizzazione in cui:
 - **PERSONA** è l'entità padre, che contiene gli attributi comuni (ad esempio: nome, cognome, data di nascita);

- **STUDENTE** e **DOCENTE** sono le entità figlie, che ereditano gli attributi di **PERSONA** e possono avere attributi propri (es. matricola per **STUDENTE**, dipartimento per **DOCENTE**).

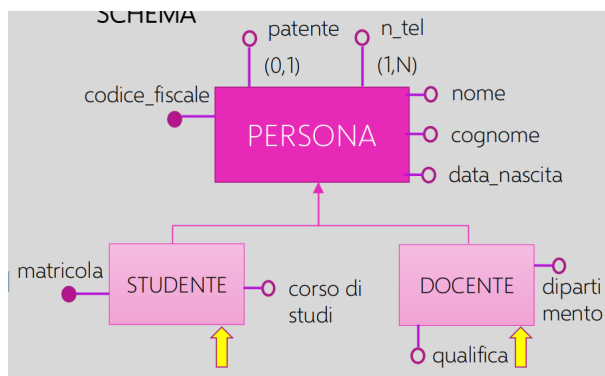


Figura 12: Esempio di generalizzazione tra **PERSONA**, **STUDENTE** e **DOCENTE**

Proprietà

Le **proprietà delle istanze di entità** che partecipano a una **generalizzazione** sono le seguenti:

- Ogni istanza di un'entità figlia E_i è anche un'istanza dell'entità padre E .
- Ogni proprietà (attributi, identificatori e relazioni) dell'entità padre E è condivisa da tutte le entità figlie $E_1, E_2, \dots, E_n$.

In altre parole, le entità figlie **ereditano** tutte le caratteristiche dell'entità padre, e possono aggiungerne di proprie.

3.7.1 Classificazione delle Generalizzazioni

Nel modello **Entità-Relazione**, una generalizzazione può assumere diverse forme, a seconda del tipo di legame tra l'entità padre (superclasse) e le entità figlie (sottoclassi). Le combinazioni derivano dall'incrocio di due caratteristiche fondamentali:

- il grado di **copertura** (Totale o Parziale);
- il grado di **esclusività** (Esclusiva o Sovrapposta).

Ecco i principali tipi di gerarchia nel modello ER:

1. **Totale ed Esclusiva (t,e)**: ogni istanza della superclasse appartiene **obbligatoriamente** a una e a una sola sottoclasse. *Esempio*: l'entità **VEICOLO** è generalizzata in **MOTO** e **AUTO**. Ogni veicolo deve essere o moto o auto, ma non può essere entrambi.
2. **Totale e Sovrapposta (t,s)**: ogni istanza della superclasse deve appartenere ad almeno una sottoclasse, ma può appartenere a **più sottoclassi** contemporaneamente. *Esempio*: l'entità **PERSONA** è generalizzata in **STUDENTE** e **LAVORATORE**. Una persona può essere sia studente sia lavoratore.
3. **Parziale ed Esclusiva (p,e)**: non tutte le istanze della superclasse devono appartenere a una sottoclasse; se appartengono, ne hanno solo una. *Esempio*: l'entità **ANIMALE** è generalizzata in **DOMESTICO** e **SELVATICO**. Un animale può essere domestico, selvatico o nessuno dei due, ma non entrambi.
4. **Parziale e Sovrapposta (p,s)**: non tutte le istanze devono appartenere a una sottoclasse; chi appartiene può essere in più sottoclassi. *Esempio*: l'entità **PERSONA** è generalizzata

in *SPORTIVO* e *ARTISTA*. Una persona può essere sportiva, artista, entrambe o nessuna delle due.

Kristian semplifica in questo modo

Le **gerarchie di generalizzazione** si classificano in base a due proprietà fondamentali:

- **Totale** → ogni istanza della superclasse deve appartenere ad almeno una sottoclasse.
- **Parziale** → alcune istanze della superclasse possono non appartenere a nessuna sottoclasse.
- **Esclusiva (o Disgiunta)** → un'istanza può appartenere solo a una sottoclasse.
- **Sovrapposta** → un'istanza può appartenere a più sottoclassi contemporaneamente.

In sintesi: **Totale/Parziale** indica *se tutte le istanze partecipano*, mentre **Esclusiva/Sovrapposta** indica *come partecipano*.

4 Strategie per la progettazione concettuale

Lo sviluppo di uno **schema concettuale** può essere considerato a tutti gli effetti un **processo di ingegnerizzazione**, in quanto richiede un insieme di fasi metodiche e ragionate per trasformare le specifiche di un problema reale in una rappresentazione astratta ma coerente e precisa.

In altre parole, progettare uno schema concettuale non significa solo disegnare entità e relazioni, ma **analizzare, organizzare e strutturare le informazioni** in modo sistematico, seguendo principi simili a quelli utilizzati nell'ingegneria del software.

Nota Bene

Per **ingegnerizzazione** si intende l'*applicazione di un metodo rigoroso e pianificato* alla progettazione, allo sviluppo e alla verifica di un sistema complesso. Nel contesto della modellazione dei dati, ciò significa:

- partire da requisiti chiari e ben definiti;
- progettare in modo graduale e controllato (Top-Down, Bottom-Up o Inside-Out);
- assicurarsi che lo schema finale sia corretto, completo e privo di ambiguità.

Ricorda che uno schema concettuale è una rappresentazione astratta e indipendente dal linguaggio di implementazione, che descrive le informazioni e le relazioni fondamentali di un sistema informativo.

4.1 Strategia Top-Down

La strategia **Top-Down** consiste nel partire da una **visione globale del sistema** e procedere verso una **maggiore specificazione dei dettagli**. È un approccio deduttivo: si comincia con concetti generali e astratti per poi raffinarli gradualmente fino a ottenere uno schema concettuale completo e preciso.

- **Fase 1:** considerare le specifiche globalmente e produrre uno schema iniziale **completo ma con pochi concetti astratti**.
- **Fase 2:** **raffinare** progressivamente i concetti astratti fino ad arrivare a uno schema concettuale **dettagliato in ogni parte**.

Nota Bene

La strategia Top-Down è particolarmente utile quando si ha una visione chiara e globale del dominio applicativo. Permette di mantenere coerenza tra le parti del progetto, ma richiede una buona capacità di astrazione e pianificazione.

Esempio: prima si individuano le entità principali (*Studente, Corso, Docente*), poi si aggiungono relazioni e attributi specifici come *data di nascita, esame sostenuto, voto*, ecc.

4.2 Strategia Bottom-Up

La strategia **Bottom-Up** adotta un approccio opposto rispetto al Top-Down. In questo caso si parte dai **dettagli elementari** per poi integrarli progressivamente fino a ottenere la visione complessiva del sistema. È un metodo **induttivo**, ideale quando si dispone di molte informazioni di base ma non ancora di una struttura generale.

- **Fase 1:** **decomporre** le specifiche iniziali in **parti elementari**, ovvero piccole frasi o porzioni di testo che descrivono uno stesso concetto o una singola informazione.
- **Fase 2:** **generare gli schemi** per ciascuna parte elementare individuata.

- **Fase 3: fondere gli schemi** parziali introducendo, se necessario, altri costrutti del modello E-R, in modo da **integrare** tutti gli elementi e ottenere lo **schema finale**.

Nota Bene

La strategia Bottom-Up è utile quando le specifiche non sono complete o sono distribuite in frammenti separati. Permette di costruire lo schema passo dopo passo, ma richiede attenzione nella fase di integrazione per evitare ridondanze o incoerenze tra i diversi schemi parziali.

Esempio: si analizzano prima frasi come “uno studente sostiene un esame” o “un docente insegna un corso”, si crea uno schema per ciascuna, e infine si uniscono tutti in un unico schema concettuale completo.

4.3 Strategia Inside-Out

La strategia **Inside-Out** rappresenta un approccio intermedio rispetto a Top-Down e Bottom-Up. Si parte da alcuni **concetti centrali o fondamentali** del dominio, detti **concetti guida**, e si costruisce lo schema espandendolo progressivamente verso l'esterno.

- **Fase 1: individuare** nelle specifiche alcuni concetti importanti, detti **concetti guida**.
- **Fase 2: generare gli schemi** relativi ai concetti guida scelti.
- **Fase 3: fondere gli schemi** precedenti e aggiungere gradualmente gli altri concetti fino a ottenere lo **schema concettuale completo**.

Nota Bene

La strategia Inside-Out è utile quando si riescono a identificare con chiarezza alcuni elementi chiave del sistema. Permette di concentrarsi inizialmente sulle parti più rilevanti e poi di estendere lo schema, garantendo coerenza e controllo durante l'espansione.

Esempio: si parte dai concetti guida come *Studente* e *Corso*, si definiscono le loro relazioni principali, e poi si estende lo schema aggiungendo altri concetti come *Esame*, *Docente* o *Dipartimento*.

4.4 Analisi di qualità dello schema

L'**analisi di qualità** di uno schema concettuale serve a valutare se il modello prodotto rappresenta correttamente e in modo efficace il dominio di interesse. Essa comprende quattro aspetti fondamentali:

- Verifica di correttezza
- Verifica di completezza
- Verifica di minimalità
- Verifica di leggibilità

Verifica di correttezza Uno schema concettuale è **corretto** se utilizza propriamente i costrutti del modello concettuale adottato (nel nostro caso, il modello E-R). La correttezza si valuta controllando che lo schema rispetti le regole sintattiche e semantiche del modello.

- **Errori sintattici:** utilizzo scorretto dei costrutti del modello E-R, senza rispettarne le regole formali. *Esempi:*
 - introduzione di una generalizzazione di relazioni;

- relazioni non collegate ad alcuna entità;
- relazioni che possiedono un identificatore.
- **Errori semantici (errori d'uso):** uso improprio dei costrutti del modello, senza rispettarne il significato. *Esempi:*
 - uso di una relazione per rappresentare un concetto che dovrebbe essere un'entità autonoma;
 - uso di un attributo per rappresentare un concetto complesso o collegato ad altri concetti in più punti dello schema.

Verifica di completezza e minimalità Uno schema concettuale è **completo** se rappresenta tutte le informazioni di interesse e consente di eseguire tutte le operazioni previste dalle specifiche. È invece **minimale** quando tutti i concetti presenti nei requisiti sono rappresentati una sola volta, evitando ridondanze o duplicazioni.

Verifica di leggibilità La leggibilità riguarda la **chiarezza e l'organizzazione visiva** dello schema. Uno schema ben leggibile deve essere ordinato, coerente e facilmente interpretabile anche da chi non lo ha progettato.

Nota Bene

Un buon schema concettuale deve essere:

- **Corretto** → rispetta le regole sintattiche e semantiche del modello E-R;
- **Completo** → include tutte le informazioni richieste;
- **Minimale** → evita ridondanze e duplicazioni;
- **Leggibile** → chiaro, ordinato e facilmente comprensibile.

Solo quando tutte queste condizioni sono soddisfatte, lo schema può essere considerato di **alta qualità**.

4.5 Ridondanze

Uno **schema concettuale non minimale** può contenere delle **ridondanze**, ossia rappresentazioni multiple dello stesso concetto logico. I casi più comuni di ridondanza sono:

- **Ereditarietà delle relazioni**
- **Cicli di relazioni**

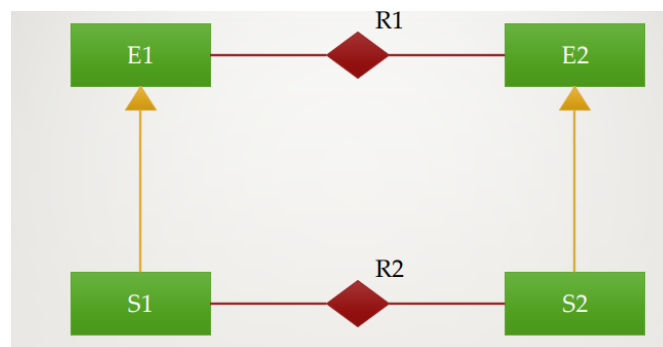


Figura 13: Ereditarietà delle relazioni

È **probabile, ma non certo**, che la relazione *R2* sia ridondante. Va quindi verificato se *R2* rappresenti lo stesso legame logico già espresso da *R1*; se così fosse, *R2* deve essere eliminata in quanto effettivamente ridondante.

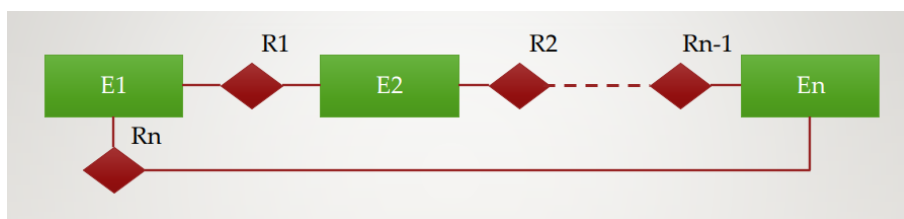


Figura 14: Ciclo di relazioni

È **probabile, ma non certo**, che una delle relazioni R_i sia ridondante. Va quindi verificato se la relazione R_i possa essere ottenuta **dalla composizione delle altre relazioni**. Se così fosse, R_i deve essere eliminata in quanto effettivamente ridondante.

4.6 Semplificazioni dello schema

Non sempre le **relazioni ternarie** sono facili da leggere e interpretare; per questo motivo, la loro eliminazione può talvolta migliorare la **leggibilità dello schema concettuale**.

Le seguenti operazioni possono essere applicate per semplificare o trasformare una relazione ternaria:

- **Trasformazione di una relazione ternaria in un'entità** Questa operazione è **sempre applicabile** e consiste nel sostituire la relazione ternaria con una nuova entità, collegata tramite relazioni binarie alle entità originarie.
- **Riduzione di una relazione ternaria a due relazioni binarie** Questa operazione è possibile solo se una delle entità coinvolte partecipa alla relazione ternaria con **cardinalità (1,1)**. In tal caso, la relazione ternaria può essere sostituita da due relazioni binarie equivalenti, mantenendo il significato logico dello schema.

Nota Bene

La trasformazione o riduzione delle relazioni ternarie deve essere eseguita con attenzione: l'obiettivo è migliorare la leggibilità dello schema senza alterarne il significato semantico.

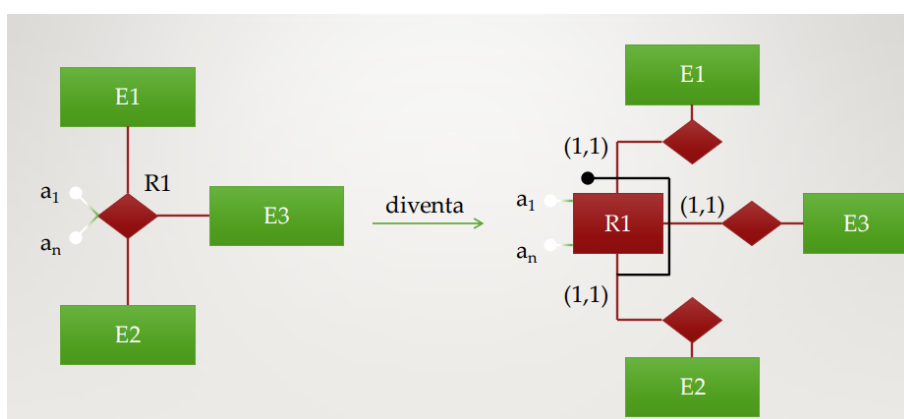


Figura 15: Trasformazione di una relazione ternaria in un'entità



Figura 16: Riduzione di una relazione ternaria a due relazioni binarie

Un ulteriore miglioramento nella leggibilità si ottiene esplicitando, nelle **generalizzazioni sovrapposte**, l'entità che rappresenta la sovrapposizione. In questo modo, la generalizzazione sovrapposta viene trasformata in una **generalizzazione esclusiva**, rendendo lo schema più chiaro e facilmente interpretabile.

5 Modello Relazionale

Nel corso del tempo si sono sviluppati diversi modelli per la rappresentazione e l'organizzazione dei dati all'interno dei sistemi informativi. Tra i principali ricordiamo:

- **Modello gerarchico/reticolare** (anni 1960–1970): tra i primi modelli di gestione dei dati, organizzati in forma gerarchica o a rete.
- **Modello relazionale** (anni 1980–oggi): introdotto da Edgar F. Codd, rappresenta i dati mediante tabelle (relazioni) e costituisce la base dei moderni DBMS relazionali.
- **Modello ad oggetti** (anni 1990–2000): integra i concetti di programmazione orientata agli oggetti con la gestione dei dati.
- **Modello a oggetti/relazionale (object–relational)** (anni 2000–oggi): un'evoluzione del modello relazionale che consente l'integrazione di tipi complessi, eredità e relazioni tra oggetti. Esempio: SQL3.
- **Modello a grafo – RDF (Resource Description Framework)** (anni 2005–oggi): rappresenta i dati sotto forma di grafo orientato, utile nel contesto del web semantico. Esempio: Neo4j.
- **Modelli noSQL** (document-based, column-based, key–value, ecc.) (anni 2010–oggi): nati per la gestione di grandi volumi di dati non strutturati e per garantire maggiore scalabilità. Esempio: MongoDB.

5.1 Costrutti Principali del Modello Relazionale

Il modello relazionale costituisce la base dei moderni database e si fonda su alcuni **costrutti principali**:

- **Domini di base**: rappresentano l'insieme dei valori ammissibili per ciascun attributo.
- **Relazione (o tabella)**:
 - Definizione come insieme di **n–uple** (righe).
 - Ogni n–upla rappresenta un record all'interno della relazione.
- **Superchiavi, Chiavi e Chiavi primarie**: identificano univocamente le tuple all'interno di una relazione.
- **Vincoli di integrità referenziale**: garantiscono la coerenza dei riferimenti tra tabelle.
- **Vincoli di integrità generici**: assicurano la validità logica dei dati rispetto a condizioni predefinite.

5.2 Domini di Base

Sono i domini da cui si scelgono i valori delle proprietà delle istanze di informazione da rappresentare. I domini tipici sono:

- Caratteri
- Stringhe
- Numeri
- Numeri decimali a virgola fissa
- Numeri decimali a virgola mobile
- Dominio del Tempo

5.3 Costrutto Relazionale

Una relazione può essere vista come una tabella ad esempio:

Città	CAP	Abitanti
Milano	20100	1 300 000
Verona	37100	350 000
Brescia	25100	250 000

Tabella 1: Esempio di relazione **Città** con attributi (*Nome*, *CAP*, *Abitanti*)

Una tabella è un contenitore di dati la cui struttura è caratterizzata da una lista di colonne:

- I dati sono scritti nelle righe dove ogni riga descrive le caratteristiche di un'istanza dell'informazione da rappresentare
- I valori contenuti nelle colonne descrivono sempre la stessa proprietà delle istanze di informazione da rappresentare.

5.3.1 Definizione: Relazione come insieme di ennuple (LISTS)

Dati n insiemi di valori (detti **domini**) $D_1, \dots, D_n$ con $n > 0$, si indica con $D_1 \times \dots \times D_n$ il loro **prodotto cartesiano**:

$$D_1 \times \dots \times D_n = \{(v_1, \dots, v_n) \mid v_1 \in D_1, \dots, v_n \in D_n\}$$

Una **relazione** ρ di grado n è un qualsiasi **sottoinsieme** di $D_1 \times \dots \times D_n$:

$$\rho \subseteq D_1 \times \dots \times D_n$$

dove $(v_1, \dots, v_n)$ è una **ennupla** della relazione e $|\rho|$ rappresenta la **cardinalità** della relazione (ovvero il numero di ennuple presenti). Si noti che i domini $D_1, \dots, D_n$ possono essere a **cardinalità infinita**, mentre le relazioni sono SEMPRE a **cardinalità finita**. Possiamo dire che

- Non è definito alcun ordinamento sulle ennuple di una relazione
- Non sono ammessi DUPLICATI di una ennupla
- Nella definizione di relazione come insieme di ennuple, i valori nelle ennuple sono ordinati

Relazione delle città

$$\rho = \{ (\text{MILANO}, 20100, 1.300.000), (\text{VERONA}, 37100, 350.000), (\text{BRESCIA}, 25100, 250.000) \}$$

$$\rho \sqsubseteq D_1 \sqsubseteq D_2 \sqsubseteq D_3$$

D_1 = Stringhe di caratteri

D_2 = Numeri interi

D_3 = Numeri interi

Figura 17: Esempio di relazione con insieme di ennuple

Accesso ai valori di una ennupla: Se t è una ennupla $(V_1, \dots, V_n)$ il valore posto in i -esima posizione si indica con la notazione $t[i]$. Questa modalità di accesso ai valori non è efficace per l'uso pratico delle relazioni. Si preferisce quindi assegnare un nome alle colonne; ciò conduce all'introduzione della definizione come insieme di **TUPLE**.

5.3.2 Definizione RELAZIONE come insieme di tuple (MAPPINGS)

Sia X un insieme di nomi e si Δ l'insieme di tutti i domini di base ammessi dal modello. Si definisce la funzione:

$$DOM : X \rightarrow \Delta$$

Tale funzione associa ad ogni nome A di X un dominio $DOM(A)$ di Δ . I nomi di X si dicono **attributi**.

Una tupla t su X è una funzione: $t : X \rightarrow \bigcup_{A \in X} DOM(A)$
dove: $t[A] = v \in DOM(A)$

Una relazione su X è un insieme di tuple su X , dove X è l'insieme di attributi della relazione.

Esempio di relazione come insieme di tuple

Relazione delle città (vista in precedenza):

$X = \{\text{Nome, CAP, Abitanti}\}$

$DOM(\text{Nome}) = \text{Stringhe di caratteri}$

$DOM(\text{CAP}) = \text{Numeri interi}$

$DOM(\text{Abitanti}) = \text{Numeri interi}$

$\rho_X = \{t_1, t_2, t_3\}$

$t_1[\text{Nome}] = \text{MILANO}, t_2[\text{Nome}] = \text{VERONA}, t_3[\text{Nome}] = \text{BRESCIA}$

$t_1[\text{CAP}] = 20100, t_2[\text{CAP}] = 37100, t_3[\text{CAP}] = 25100$

$t_1[\text{Abitanti}] = 1.300.000, t_2[\text{Abitanti}] = 350.000, t_3[\text{Abitanti}] = 250.000$

Figura 18: Esempio

- Una relazione è un insieme di tuple e quindi non può contenere tuple duplicate
- I domini per gli attributi possono essere solo domini di base, non sono ammessi altri domini, né il prodotto cartesiano di domini
- In generale una base di dati relazionale è costituita da più relazioni.

5.4 Come progettare una base di dati relazionale

Nel modello relazionale lo schema è costituito da un insieme di relazioni o tabelle. Lo strumento formale che serve per definire dipendenze tra attributi nel modello relazionale è costituito dalle **dipendenze funzionali**.

Per Esempio: Vogliamo rappresentare in una base di dati relazionale le informazioni sulla proprietà degli appartamenti siti nel comune di Verona.

- Per ogni appartamento vogliamo memorizzare : il codice ecografico (univoco), la categoria catastale e l'indirizzo composto da : via, numero civico, subalterno
- Per ogni proprietario si registra: il codice fiscale, il nome, il cognome e la data di nascita.
- Un appartamento può essere in comproprietà e un proprietario può possedere più appartamenti.

PROPRIETÀ								
Categoria	Via	Civico	Sub	Codice ECO	Codice Fiscale	Nome	Cognome	Data Nas
A1	Roma	23	A	RMAXXX	RSSMRR75D	Mario	Rossi	1/1/1975
A1	Roma	23	A	RMAXXX	BNCMRA80F	Maria	Bianchi	10/9/1980
A3	Milano	9		MILYYY	BNCMRA80F	Maria	Bianchi	10/9/1980
A3	Milano	9		MILYYY	BNCPLO77A	Paolo	Bianchi	3/1/1977
A1	Torino	2	C	TORZZZ	BNCPLO77A	Paolo	Bianchi	3/1/1977
A2	Garibaldi	33		GARWWW	RSSMRR75D	Mario	Rossi	1/1/1975

Figura 19: Schema relazionale con solo una relazione

Nella relazione unica ogni riga rappresenta un contratto di proprietà e richiede di precisare tutti i dati di un appartamento e tutti i dati di un proprietario. Ciò implica che:

- Non è possibile rappresentare un appartamento che non abbia un proprietario, né rappresentare un proprietario senza appartamento,
- I dati di un appartamento vengono replicati per ogni proprietario
- I dati di un proprietario vengono replicati per ogni appartamento di proprietà.

Anomalie: La presenza di ridondanza inutile nella base di dati provoca le seguenti anomalie:

- **ANOMALIE DI AGGIORNAMENTO** : per aggiornare il valore di un attributo si è obbligati a modificare tale valore su più tuple.
- **ANOMALIE DI INSERIMENTO** : per inserire una nuova tupla è necessario inserire valori al momento sconosciuti (sostituibili da valori nulli) per gli attributi non disponibili.
- **ANOMALIE DI CANCELLAZIONE** : per cancellare una tupla è necessario cancellare valori ancora validi oppure inserire valori nulli per gli attributi da cancellare.

È possibile ridurre la ridondanza nei dati decomponendo la relazione. Sono possibili diverse decomposizioni.

APPARTAMENTO					PROPRIETARIO			
Categoria	Via	Civico	Sub	Codice ECO	Codice Fiscale	Nome	Cognome	Data Nas
A1	Roma	23	A	RMAXXX	RSSMRR75D	Mario	Rossi	1/1/1975
A3	Milano	9		MILYYY	BNCMRA80F	Maria	Bianchi	10/9/1980
A1	Torino	2	C	TORZZZ	BNCPLO77A	Paolo	Bianchi	3/1/1977
A2	Garibaldi	33		GARWWW				

Figura 20: Decomposizione 1

APPARTAMENTO					
Categoria	Via	Civico	Sub	Codice ECO	Codice Fiscale
A1	Roma	23	A	RMAXXX	RSSMRR75D
A1	Roma	23	A	RMAXXX	BNCMRA80F
A3	Milano	9		MILYYY	BNCMRA80F
A3	Milano	9		MILYYY	BNCPLO77A
A1	Torino	2	C	TORZZZ	BNCPLO77A
A2	Garibaldi	33		GARWWW	RSSMRR75D

PROPRIETARIO			
Codice Fiscale	Nome	Cognome	Data Nas
RSSMRR75D	Mario	Rossi	1/1/1975
BNCMRA80F	Maria	Bianchi	10/9/1980
BNCPLO77A	Paolo	Bianchi	3/1/1977

Figura 21: Decomposizione 2

PROPRIETARIO				
Codice ECO	Codice Fiscale	Nome	Cognome	Data Nas
RMAXXX	RSSMRR75D	Mario	Rossi	1/1/1975
RMAXXX	BNCMRA80F	Maria	Bianchi	10/9/1980
MILYYY	BNCMRA80F	Maria	Bianchi	10/9/1980
MILYYY	BNCPLO77A	Paolo	Bianchi	3/1/1977
TORZZZ	BNCPLO77A	Paolo	Bianchi	3/1/1977
GARWWW	RSSMRR75D	Mario	Rossi	1/1/1975

APPARTAMENTO				
Categoria	Via	Civico	Sub	Codice ECO
A1	Roma	23	A	RMAXXX
A3	Milano	9		MILYYY
A1	Torino	2	C	TORZZZ
A2	Garibaldi	33		GARWWW

Figura 22: Decomposizione 3

APPARTAMENTO					PROPRIETARIO			
Categoria	Via	Civico	Sub	Codice ECO	Codice Fiscale	Nome	Cognome	Data Nas
A1	Roma	23	A	RMAXXX	RSSMRR75D	Mario	Rossi	1/1/1975
A3	Milano	9		MILYYY	BNCMRA80F	Maria	Bianchi	10/9/1980
A1	Torino	2	C	TORZZZ	BNCPLO77A	Paolo	Bianchi	3/1/1977
A2	Garibaldi	33		GARWWW				

Codice ECO	Codice Fiscale	PROPRIETÀ
RMAXXX	RSSMRR75D	
RMAXXX	BNCMRA80F	
MILYYY	BNCMRA80F	
MILYYY	BNCPLO77A	
TORZZZ	BNCPLO77A	
GARWWW	RSSMRR75D	

Figura 23: Decomposizione 4

Si noti che i legami tra relazioni si realizzano attraverso la replicazione di un insieme di attributi, dove il legame tra due tuple si intende stabilito quando esse presentano gli stessi valori negli attributi replicati. Il modello relazionale è **value-based** (basato su valori). Il modello non prevede l'uso di puntatori per rappresentare legami tra i dati. Essere value-based implica:

- Completa indipendenza dall'implementazione e dallo schema fisico
- Facilità di trasferimento dei dati tra sistemi
- Viene rappresentato solo ciò che è rilevante per il sistema informativo

5.4.1 Terminologia di schema di una relazione

E costituito dal nome della relazione e da un insieme di nomi per i suoi attributi:

$R(X)$ oppure $R(A_1, \dots, A_n)$ oppure $R(A_1 : D_1, \dots, A_n : D_n)$

5.4.2 Terminologia Schema di una base di dati

E costituito da un insieme di schemi di relazioni:

$S = \{R_1(X_1), \dots, R_n(X_n)\}$ con $R_1 \neq R_2 \neq \dots R_n$

5.4.3 Terminologia istanza di una relazione di schema $R(X)$

E un insieme r di tuple su X dove $r = \{t_1, \dots, t_k\}$

5.4.4 Terminologia istanza di una base di dati di schema $S = \{R_1(X_1), \dots, R_n(X_n)\}$

E costituito da un insieme di istanze delle relazioni $R_1(X_1), \dots, R_n(X_n) : db = \{r_1, \dots, r_n\}$

5.5 Valori nulli

Secondo la definizione formale di *relazione*, ogni tupla dovrebbe contenere per ciascun attributo un valore appartenente al relativo dominio. Nelle basi di dati reali, però, questa condizione non è sempre rispettabile: può accadere che un attributo non abbia un valore disponibile.

I motivi principali per cui un valore può mancare sono i seguenti:

- **Valore inesistente** L'attributo A è concettualmente opzionale: per quella specifica istanza non esiste alcun valore significativo da inserire. *Esempio:* il campo “secondo numero di telefono” per un cliente che possiede un solo numero.
- **Valore sconosciuto** Il valore dell'attributo A esiste, ma non è attualmente noto alla base di dati. *Esempio:* l'indirizzo e-mail di un utente appena registrato che non lo ha ancora comunicato.
- **Valore sconosciuto o inesistente** Non è possibile stabilire se l'informazione manchi perché non esiste o perché non è ancora conosciuta, oppure si preferisce rappresentare entrambe le situazioni allo stesso modo. *Esempio:* il campo “data di scadenza del contratto”, che potrebbe non essere ancora stata definita oppure semplicemente non comunicata.

Per gestire queste situazioni si introduce il **valore nullo**. In questo modo il valore di un attributo in una tupla può essere:

- un valore appartenente al dominio dell'attributo;
- il NULL, utilizzato per rappresentare un'informazione assente (inesistente o sconosciuta).

Esempio: nel campo “numero di telefono fisso” di una tabella dei clienti, un cliente che non possiede un telefono fisso avrà come valore NULL.

Una tupla su X è una funzione:

$$t : X \rightarrow \{NULL\} \cup \left(\bigcup_{A \in X} DOM(A) \right)$$

dove:

$$t[A] = v \in DOM(A) \vee t[A] = NULL$$

Figura 24: Tupla su X con valori nulli

Si noti che:

- **La presenza di valori nulli è ammessa solo in alcuni attributi.** Non è infatti possibile avere tuple composte esclusivamente da valori nulli, poiché non rappresenterebbero alcuna informazione utile.
- **I valori nulli possono creare problemi negli attributi usati per rappresentare legami tra tuple.** In particolare, negli attributi che fungono da chiave esterna, un valore NULL potrebbe rendere impossibile stabilire la relazione con altre tuple.

Esempio: Si consideri una tabella **Prenotazioni** con l'attributo **idCliente** come chiave esterna verso la tabella **Clienti**. Se **idCliente** fosse NULL, non sarebbe possibile sapere quale cliente ha effettuato la prenotazione, rendendo l'informazione inutilizzabile.

5.6 Vincoli di integrità

Spesso è necessario introdurre dei **vincoli** (restrizioni) sull'istanza di una base di dati, poiché non tutte le configurazioni possibili sono compatibili con il sistema informativo che si vuole rappresentare.

Per specificare quali istanze sono considerate corrette si introduce il concetto di **vincolo di integrità**.

Vincolo di Integrità

Un **vincolo di integrità** è una condizione (espressa come un predicato) che deve essere sempre verificata da ogni istanza della base di dati. I vincoli di integrità assicurano che i dati memorizzati siano consistenti, coerenti e validi rispetto al modello del sistema informativo.

5.7 Esempi di vincoli

Siano date le seguenti relazioni:

- $TRENO(Numero, OraPart, MinutoPart, Categoria, Destinazione, OraArr, MinutoArr)$
- $FERMATA(NumTreno, Stazione, OraFer, MinutoFer)$

I seguenti predicati esprimono possibili **vincoli di integrità** sulla relazione $TRENO$:

$$\forall t \in TRENO : t[OraPart] \in \{0, 1, \dots, 23\}$$

$$\forall t \in TRENO : t[MinutoPart] \in \{0, 1, \dots, 59\}$$

$$\forall t \in TRENO : t[Numero] > 0$$

$$\forall t \in TRENO : t[Numero] > 5000 \Rightarrow t[Categoria] = \text{'regionale'}$$

$$\forall t, t' \in TRENO : t \neq t' \Rightarrow t[Numero] \neq t'[Numero]$$

(*Vincolo di unicità sul Numero del treno*)

$$\forall f \in FERMATA : \exists t \in TRENO : t[Numero] = f[NumTreno]$$

(*Vincolo di chiave esterna: ogni fermata deve riferirsi a un treno esistente*)

$$\forall t \in TRENO : t[Categoria] = \text{'regionale'} \Rightarrow \exists f \in FERMATA : f[NumTreno] = t[Numero]$$

(*Se un treno è regionale allora deve avere almeno una fermata*)

5.8 Classificazione vincoli di integrità

Si distinguono inoltre alcuni vincoli particolari che rivestono un ruolo fondamentale, poiché assicurano proprietà generali valide per tutti gli schemi relazionali. Tali vincoli sono detti **vincoli strutturali**.

Essi comprendono:

- **Vincoli di dominio** Specificano quali valori possono essere assegnati a ciascun attributo (esempio: l'attributo `OraPart` deve essere compreso tra 0 e 23).
- **Vincoli di tupla** Impongono condizioni che devono essere verificate all'interno di una singola tupla (esempio: `Numero > 0`).

- **Vincoli intrarelazionali** I vincoli intrarelazionali impongono condizioni sul contenuto di una relazione e richiedono che ogni tupla soddisfi una certa proprietà rispetto alle altre tuple della stessa relazione.

Una categoria particolarmente rilevante di tali vincoli è costituita dai **vincoli di chiave**, che comprendono:

- **Superchiave**: Data una relazione di schema $R(X)$, un insieme di attributi $K \subseteq X$ è una **superchiave** per $R(X)$ se, per ogni istanza r di $R(X)$, vale la seguente condizione:

$$\forall t, t' \in r : t \neq t' \Rightarrow t[K] \neq t'[K]$$

dove:

$$t[K] \neq t'[K] \equiv \exists A_i \in K : t[A_i] \neq t'[A_i]$$

- **Chiave candidata** : Data una relazione di schema $R(X)$, un insieme di attributi K , sottoinsieme di X , è **chiave candidata** (o **chiave**) per $R(X)$ se K è superchiave per $R(X)$ e vale la seguente condizione:

$$\neg \exists K' \subset K : K' \text{ è superchiave per } R(X)$$

- **Chiave primaria** : Data una relazione di schema $R(X)$, la sua **chiave primaria** è la chiave candidata scelta per *identificare le tuple della relazione*.

Una chiave primaria K ha le seguenti caratteristiche:

- * Non contiene mai valori nulli.
- * Su K il sistema genera una struttura d'accesso ai dati (o indice) per supportare le interrogazioni.

Kristian semplifica così – Le tre chiavi spiegate intuitivamente

Superchiave Una superchiave è **qualsiasi insieme di attributi che distingue sempre le tuple**. Se due tuple sono diverse, almeno un attributo della superchiave è diverso.

Esempio: Nella tabella STUDENTI(Matricola, Nome, Cognome, Email) - Insieme come {Matricola}, {Matricola, Nome}, {Email, Cognome} sono tutte superchiavi. Perché? Perché non esistono due studenti con la stessa combinazione di quei valori.

Chiave candidata Una chiave candidata è una **superchiave minimale**: non puoi togliere nessun attributo senza perdere l'unicità.

Esempio: - {Matricola} è chiave candidata perché basta da sola. - {Email} è un'altra chiave candidata. - Ma {Matricola, Nome} **non** è candidata: contiene attributi inutili, è troppo "larga".

Chiave primaria È la **chiave candidata scelta** per identificare ufficialmente le tuple. Non può contenere NULL e il DB crea un indice su di essa.

Esempio: Se in STUDENTI scegliamo la chiave candidata {Matricola}, quella diventa la chiave primaria. Serve per identificare uno studente, fare ricerche, creare collegamenti con altre tabelle.

- **Vincoli interrelazionali** I vincoli interrelazionali impongono una restrizione al contenuto di una relazione e specificano una condizione che ogni tupla della relazione deve soddisfare rispetto alle tuple di altre relazioni della base di dati. Una sottocategoria importante di tali vincoli è costituita dai:
 - **Vincoli di integrità referenziale** impone che ogni valore di una *chiave esterna* (foreign key) presente in una relazione debba necessariamente comparire come valore di *chiave primaria* (primary key) nella relazione a cui fa riferimento. In questo modo si garantisce la coerenza dei dati tra più tabelle e si impedisce l’inserimento di tuple “orfane” non collegate a dati esistenti.

Kristian semplifica in questo modo

Il vincolo di integrità referenziale è una regola che assicura che due tabelle rimangano sempre coerenti tra loro.

Significa: se in una tabella inserisco una *chiave esterna*, quel valore deve esistere come *chiave primaria* nella tabella collegata.

Esempio intuitivo:

* Tabella **Studenti**(id, nome)

* Tabella **Esami**(idStudente, voto)

Nella tabella **Esami**, il campo **idStudente** è una **foreign key**: può contenere solo valori che esistono come **id** nella tabella **Studenti**.

Se provo a inserire un esame per lo studente **id = 50**, ma nella tabella **Studenti** non esiste lo studente 50, il database blocca l’inserimento perché violerei il vincolo di integrità referenziale.

5.9 Notazione per indicare i vincoli strutturali

5.9.1 Chiave primaria

La chiave primaria di una relazione $R(X)$ viene indicata **sottolineando gli attributi** che la compongono.

Supponiamo che, per la tabella **TRENO**, venga scelto come chiave primaria l’attributo **Numero**. La notazione per indicare tale chiave è:

TRENO(Numero, OraPart, MinutoPart, Categoria, Destinazione, OraArr, MinutoArr)

Per la tabella **FERMATA**, la chiave primaria è composta dagli attributi **NumTreno**, **Stazione**, **OraFer** e **MinutoFer**. Si indica così:

FERMATA(NumTreno, Stazione, OraFer, MinutoFer)

5.9.2 Vincolo di integrità referenziale

Un **vincolo di integrità referenziale** viene indicato **riquadrando gli attributi soggetti al vincolo** (nella tabella figlia) e collegandoli con una freccia al riquadro degli attributi della tabella da cui la chiave primaria proviene (tabella padre).

Supponiamo che nella tabella **FERMATA** l’attributo **NumTreno** sia una chiave esterna che fa riferimento alla chiave primaria **Numero** della tabella **TRENO**.

La notazione diventa:

Numero ← TRENO(Numero, OraPart, MinutoPart, Categoria, Destinazione, OraArr, MinutoArr)

NumTreno FERMATA(NumTreno, Stazione, OraFer, MinutoFer)

Kristian semplifica in questo modo

Chiave primaria: sottolineo gli attributi che identificano univocamente le tuple della tabella.

Chiave esterna (integrità referenziale): metto un riquadro attorno all'attributo nella tabella "figlia" e lo collego con una freccia alla chiave primaria della tabella "padre".

In questo modo si vede subito da dove arriva il valore e quale tabella controlla la sua validità.

5.10 Esercizio

2.7 Individuare le chiavi e i vincoli di integrità referenziale che sussistono nella base di dati di Figura 2.23 e che è ragionevole assumere siano soddisfatti da tutte le basi di dati sullo stesso schema. Individuare anche gli attributi sui quali possa essere sensato ammettere valori nulli.

Esercizio 2.7
del
libro di testo.

PAZIENTI

Cod.	Cognome	Nome
A102	Necchi	Luca
B372	Rossini	Piero
B543	Missoni	Nadia
B444	Missoni	Luigi
S555	Rossetti	Gino

RICOVERI

Paz.	Inizio	Fine	Rep.
A102	2/05/11	9/05/11	A
A102	2/12/11	2/01/12	A
S555	25/04/11	3/05/11	B
B444	1/12/11	2/01/12	B
S555	5/10/11	1/11/11	A

MEDICI

Matr.	Cogn.	Nome	Rep.
203	Neri	Piero	A
574	Bisi	Mario	B
461	Bargio	Sergio	B
530	Belli	Nicola	C
405	Mizzi	Nicola	A
501	Monti	Mario	A

REPARTI

Cod.	Nome	Primario
A	Chirurgia	203
B	Pediatria	574
C	Medicina	530

Figura 2.23 Una base di dati per l'Esercizio 2.6 e 2.7.

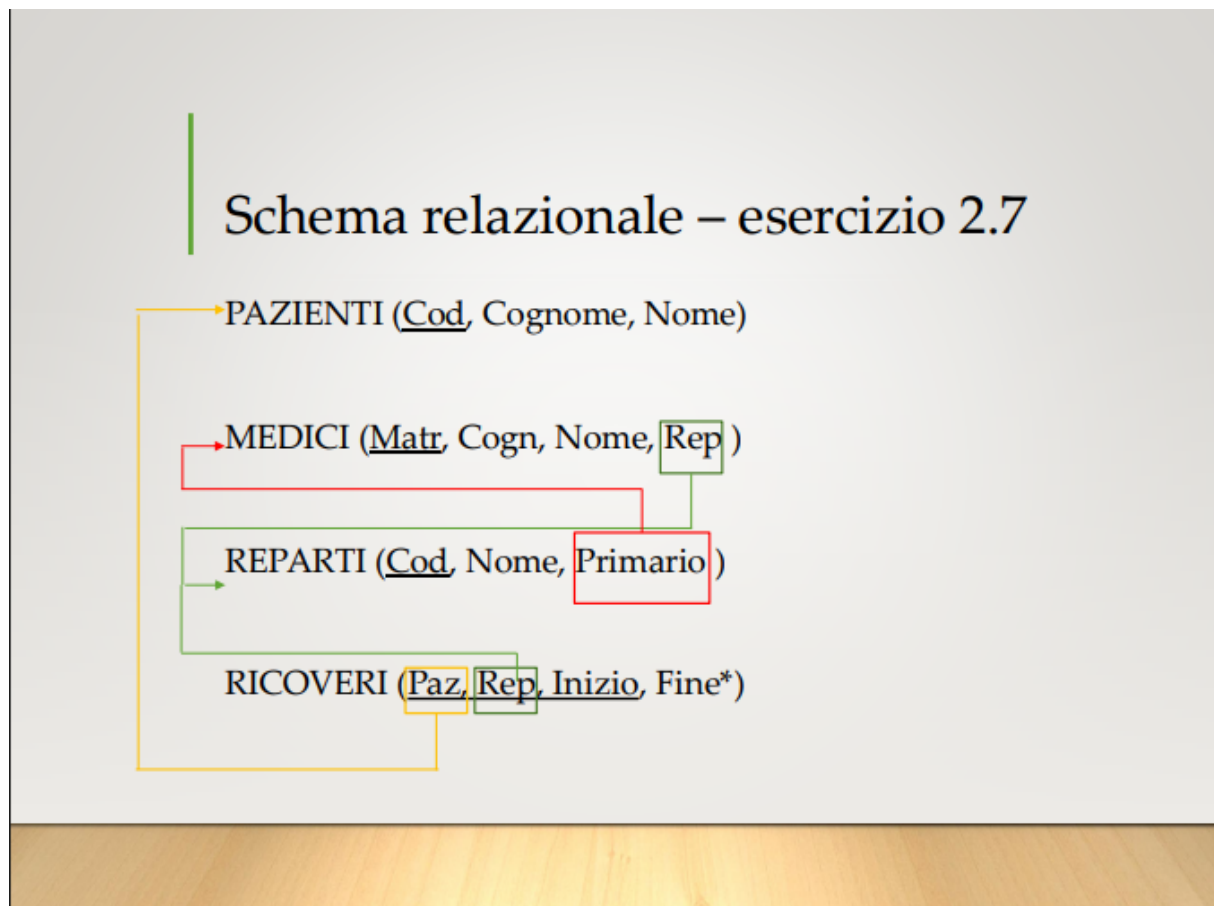


Figura 25: Soluzione

5.11 Progettazione Logica

L'obiettivo della progettazione logica è quello di produrre uno **schema logico** che rappresenti in modo corretto, completo ed efficace tutte le informazioni contenute nello **schema concettuale**.

Prima della traduzione può essere necessario **ristrutturare lo schema ER** per i seguenti motivi:

- **Semplificare** la fase di traduzione verso il modello logico.
- **Ottimizzare** le strutture in base al *carico di lavoro* previsto (operazioni e query più frequenti).

Le principali fasi della **ristrutturazione dello schema ER** sono:

- Analisi delle **ridondanze** dovute alla presenza di dati derivabili.
- **Eliminazione delle generalizzazioni** tramite le diverse strategie di trasformazione.
- **Accorpamento o partizionamento** di entità e relazioni, quando necessario.
- Scelta degli **identificatori principali** più adatti ed efficienti.

5.11.1 Analisi della derivabilità

L'analisi della derivabilità serve a individuare **dati che possono essere ricavati (derivati) da altri dati già presenti nello schema**. Questi dati non vanno memorizzati perché occupano spazio inutilmente e possono creare inconsistenze.

Esempi di dati derivabili:

- **Attributi calcolabili** a partire da altri attributi della stessa entità (es. “età” derivabile dalla “data di nascita”).
- **Attributi derivabili** da informazioni presenti in entità diverse (es. “numero totale degli ordini di un cliente” ricavabile dalla relazione *Ordini*).
- **Attributi ottenuti con un conteggio** (es. “numero di iscritti a un corso” ricavabile contando le partecipazioni).
- **Relazioni derivabili** dalla combinazione di altre relazioni (es. una relazione “LavoraIn” può essere derivata combinando “Impiegato” e “Dipartimento” tramite una relazione intermedia).

5.11.2 Analisi delle ridondanze

In questa fase vengono analizzati i **dati derivabili** per decidere se conviene:

- **Memorizzare il dato derivabile**, quando il costo del calcolo è *maggiore* del costo di aggiornamento.
- **Non memorizzarlo e calcolarlo quando serve**, quando il costo del calcolo è *minore* del costo di aggiornamento.

Il **costo** viene misurato in termini di numero di accessi per unità di tempo, valutando:

- la frequenza con cui il dato deve essere calcolato,
- la frequenza con cui il dato dovrebbe essere aggiornato.

5.11.3 Eliminazione delle Generalizzazioni

Nel modello relazionale le generalizzazioni non sono direttamente rappresentabili. Per questo motivo esistono tre possibili strategie per trasformarle:

- **Accorpamento delle entità figlie nel padre.**
- **Accorpamento dell'entità padre nelle entità figlie.**
- **Sostituzione della generalizzazione con relazioni.**

5.11.4 Accorpamento delle entità figlie nel padre

Questa soluzione prevede:

- Eliminazione delle entità figlie.
- Trasferimento nel padre di tutti gli attributi specifici delle entità figlie, trattandoli come **attributi opzionali**.
- Accorpamento nel padre di tutte le relazioni che coinvolgevano le entità figlie, impostando la **cardinalità minima a 0** dal lato del padre.
- Aggiunta di un attributo **TIPO** per distinguere le occorrenze corrispondenti alle ex entità figlie.

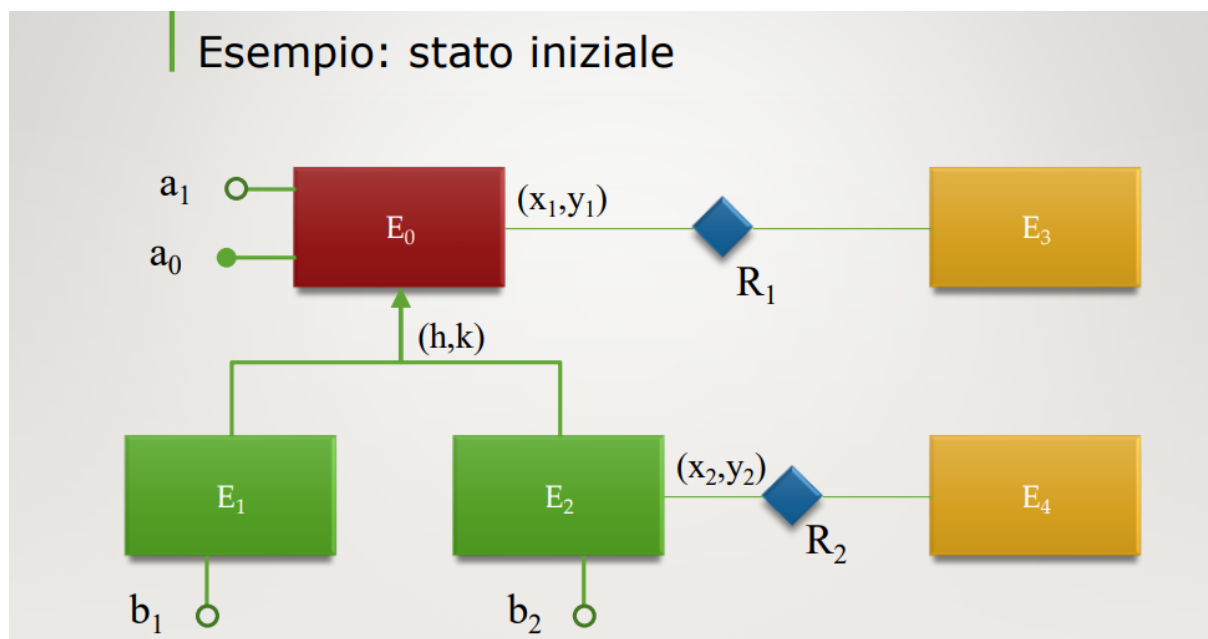


Figura 26: Stato iniziale

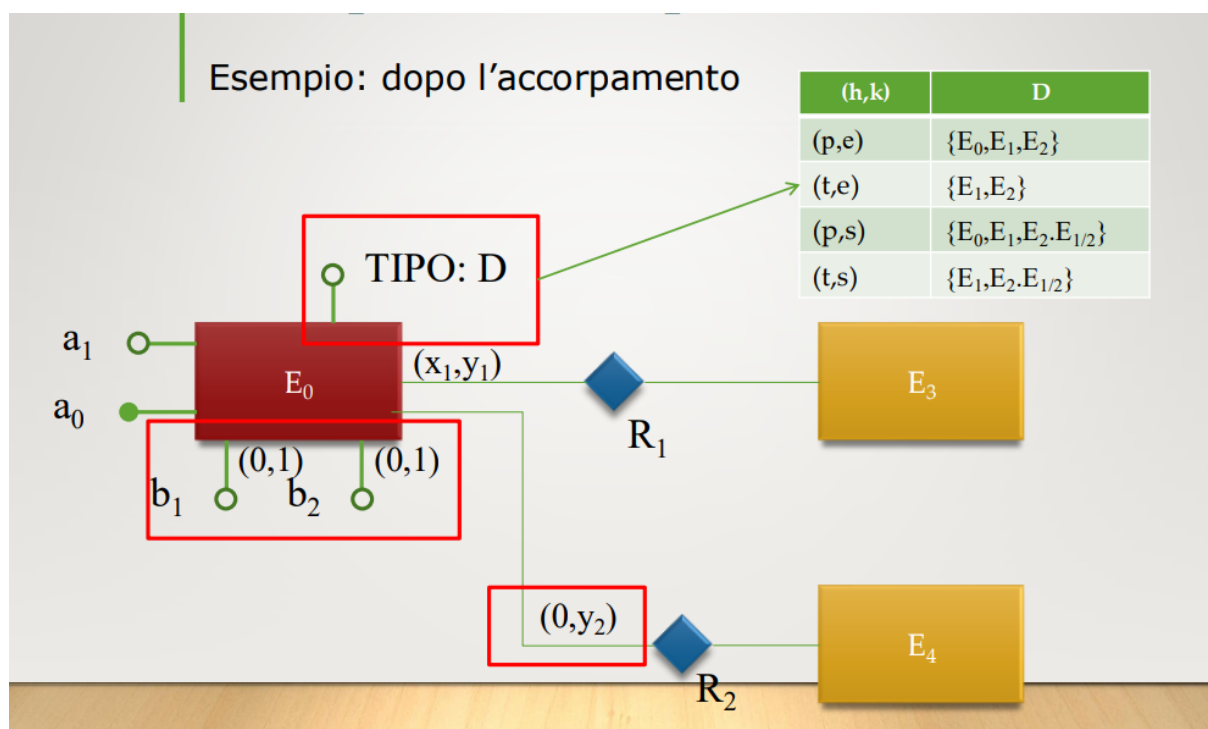


Figura 27: Accorpamento delle entità figlie nel padre

5.11.5 Accorpamento del padre nelle entità figlie

Questa trasformazione può essere applicata solo quando la generalizzazione è **totale**. La procedura consiste nei seguenti passi:

- Eliminazione dell'entità padre.
- Replicazione degli attributi dell'entità padre in ciascuna entità figlia.

- Partizionamento, tra le entità figlie, delle relazioni che coinvolgevano l'entità padre, assegnando **cardinalità minima pari a 0** dal lato delle entità esterne.

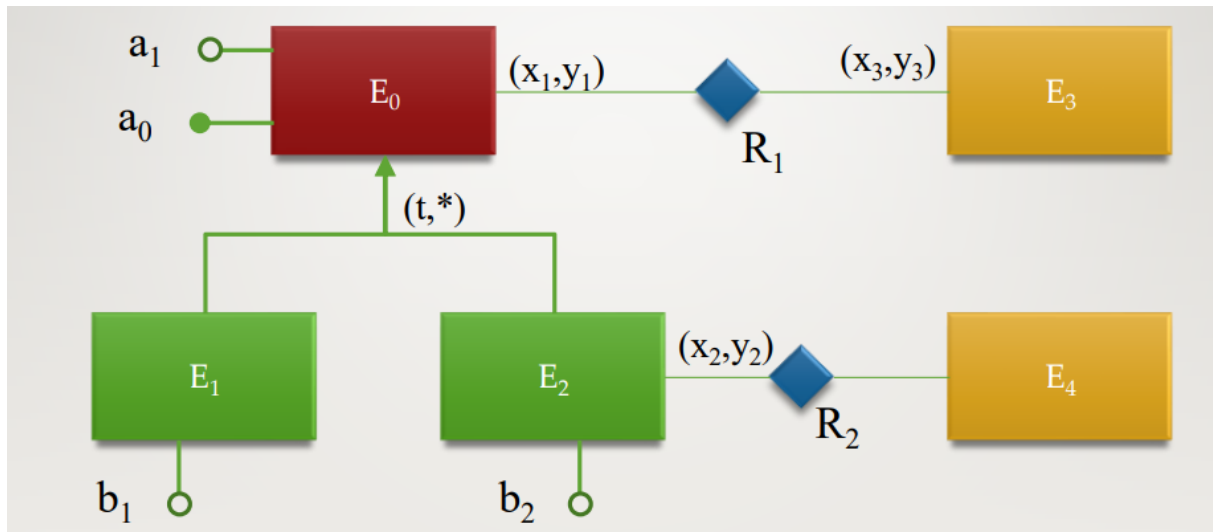


Figura 28: Schema iniziale

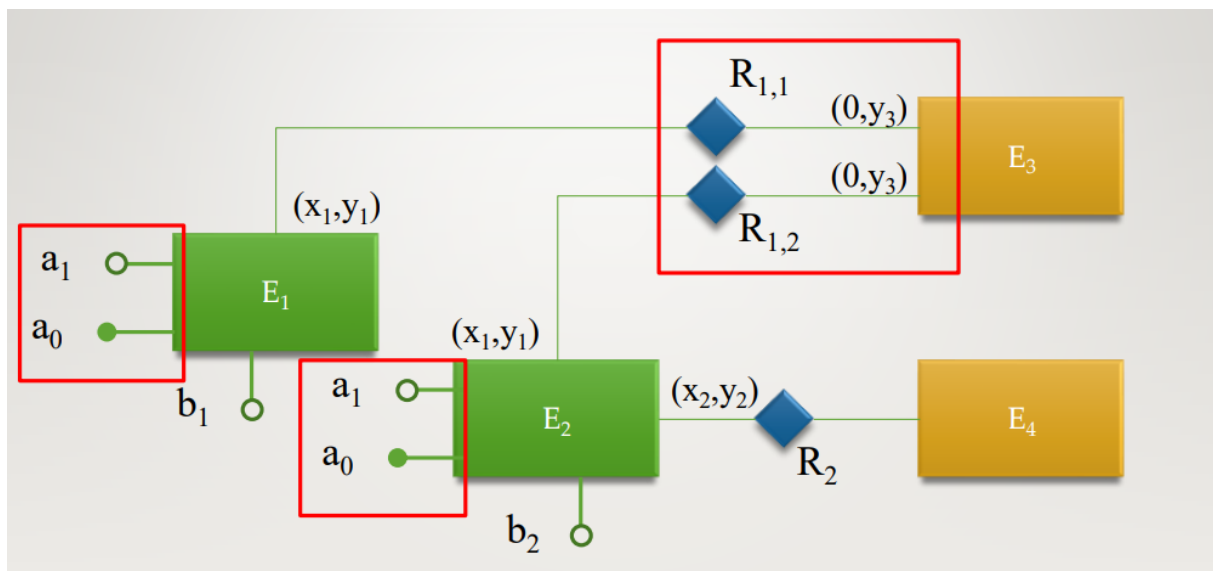


Figura 29: Accorpamento nelle entità figlie

5.11.6 Sostituzione con relazioni

Questa strategia sostituisce la generalizzazione con un insieme di relazioni esplicite. I passi da seguire sono:

- Eliminazione della generalizzazione.
- Inserimento di n relazioni tra l'entità padre e ciascuna delle n entità figlie.
- Mantenimento degli attributi nelle entità in cui si trovavano originalmente, senza spostamenti.

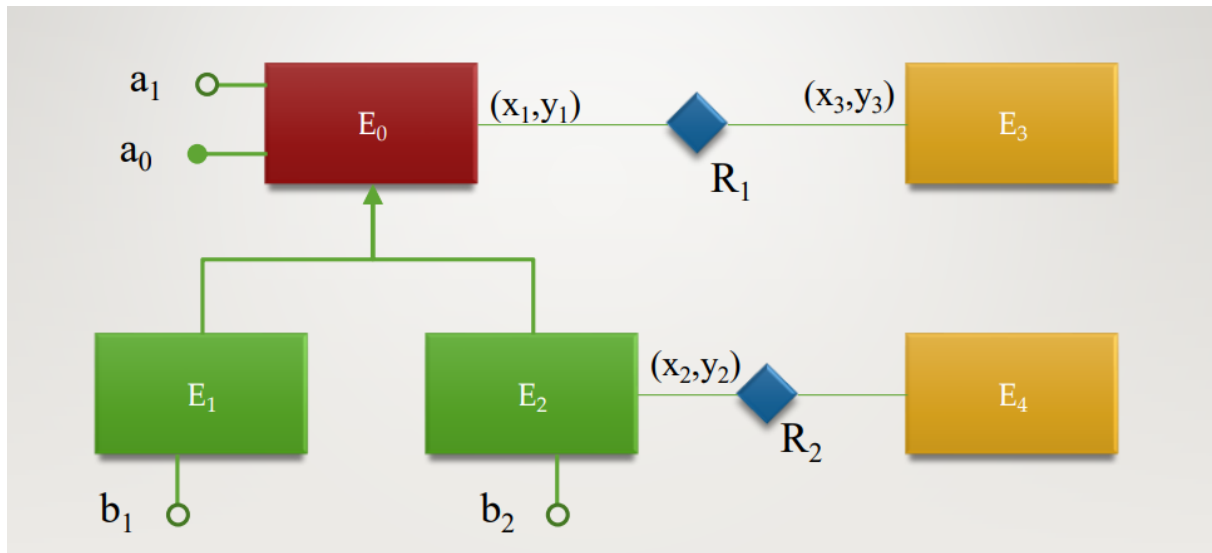


Figura 30: Schema iniziale

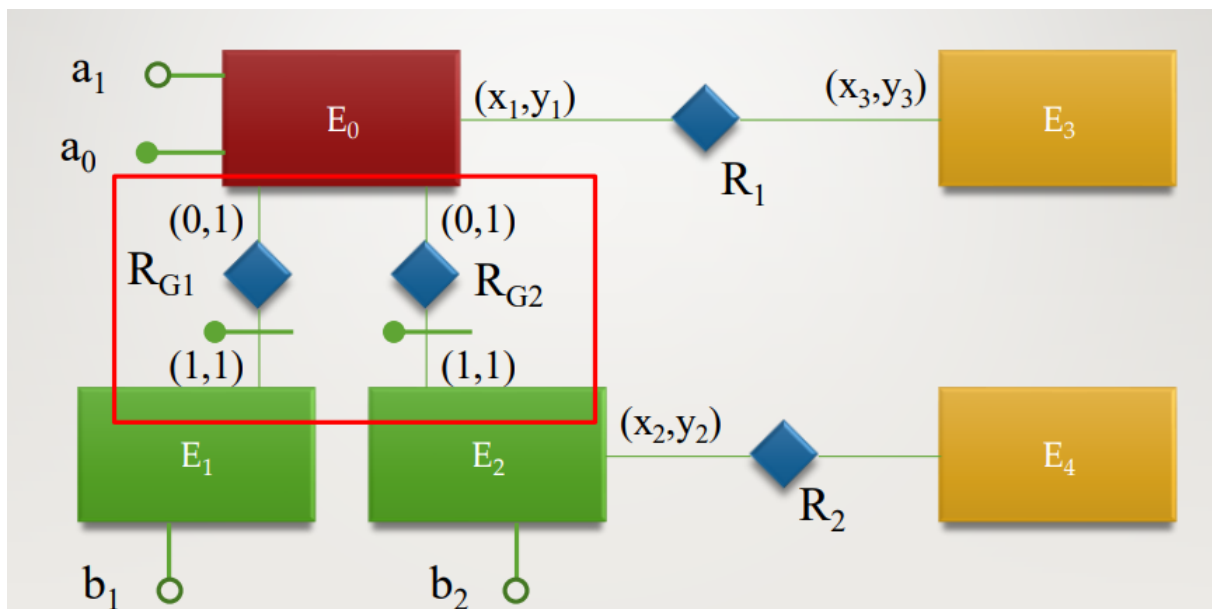


Figura 31: Sostituzione della generalizzazione con relazioni

5.12 Traduzione verso il modello relazionale

La traduzione verso il modello relazionale è un processo automatico che applica allo **schema concettuale ristrutturato** un insieme di regole di trasformazione. Ogni regola si applica a uno specifico costrutto del modello concettuale ER e produce una o più strutture del modello relazionale.

I principi fondamentali della traduzione sono:

- Un'istanza di entità viene rappresentata mediante una **tupla** della tabella che implementa l'entità.
- Un'istanza di relazione viene rappresentata tramite:
 - un **legame tra tuple** (mediante chiavi esterne), oppure

- una **tupla esplicita** in una tabella dedicata.
- L'**identificatore principale** di un'entità viene rappresentato come **chiave primaria** della tabella che implementa tale entità.

5.13 Classificazione delle relazioni binarie

Una relazione binaria del modello ER può assumere tre forme, in base alle **cardinalità massime** delle entità coinvolte:

- **Uno a Uno (1:1)**: entrambe le entità coinvolte nella relazione hanno cardinalità massima pari a 1.
- **Uno a Molti (1:N)**: una delle entità ha cardinalità massima pari a 1, mentre l'altra ha cardinalità massima maggiore di 1.
- **Molti a Molti (N:M)**: entrambe le entità hanno cardinalità massima maggiore di 1.

5.14 Regole di traduzione

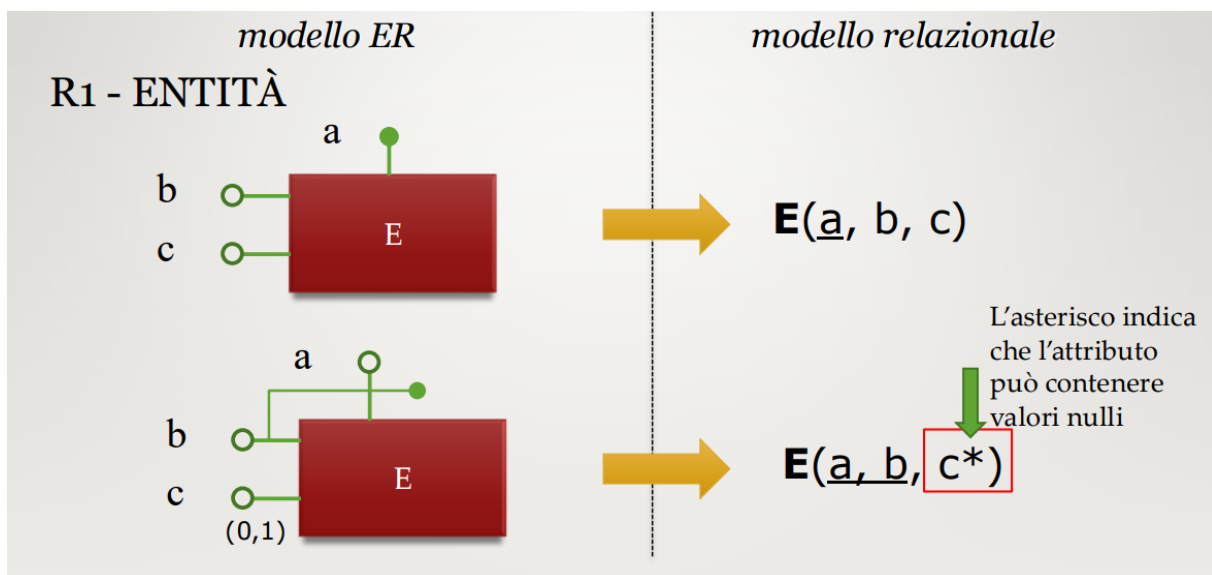


Figura 32: Entità

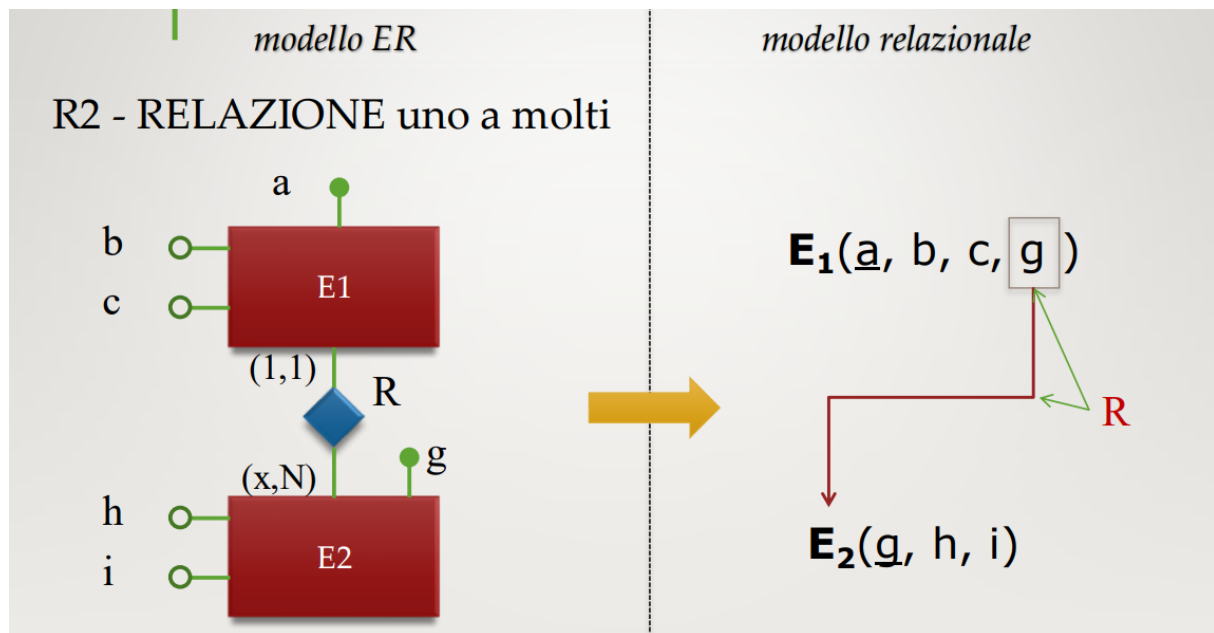


Figura 33: Relazione uno a molti

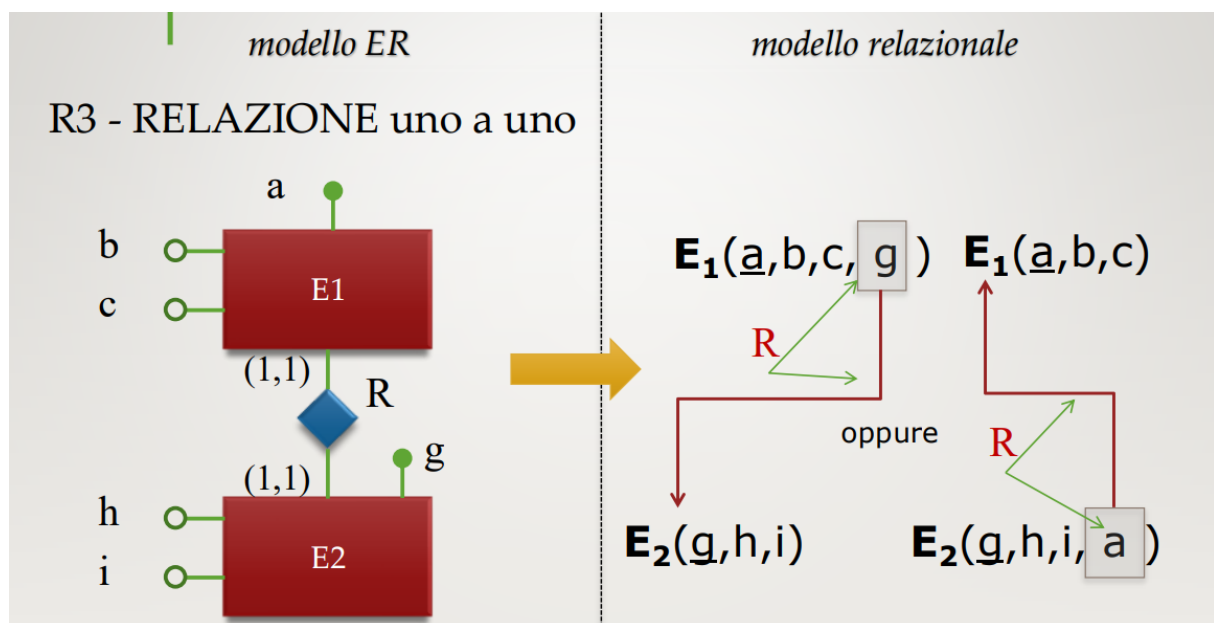


Figura 34: Relazione uno a uno

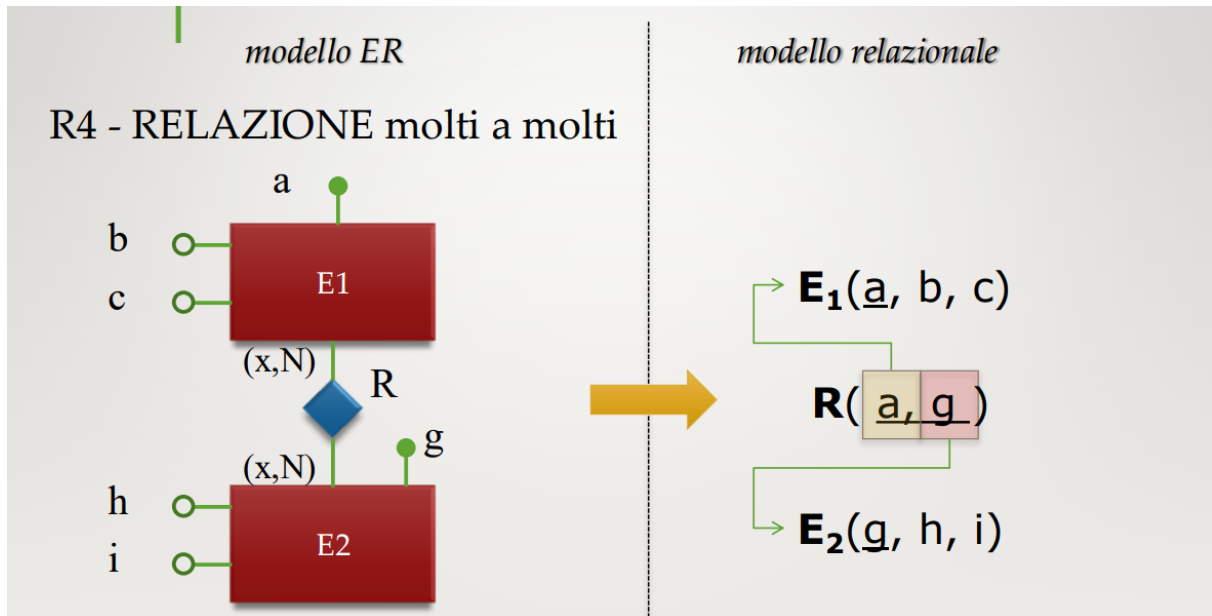


Figura 35: Relazione molti a molti

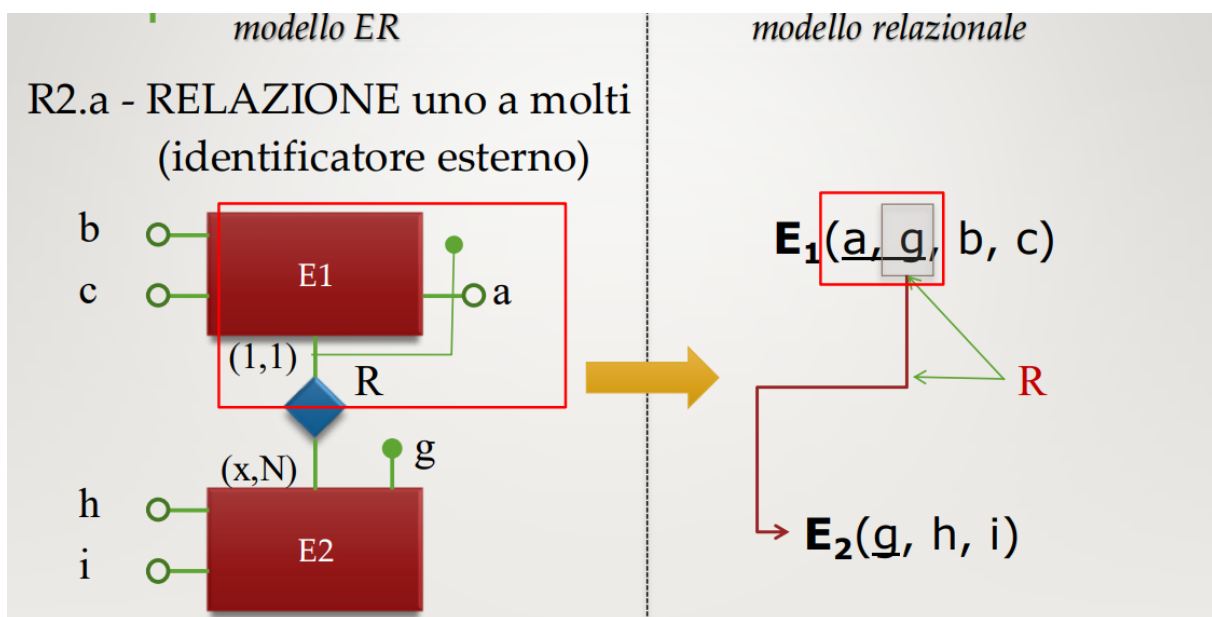


Figura 36: Relazione uno a molti identificatore esterno

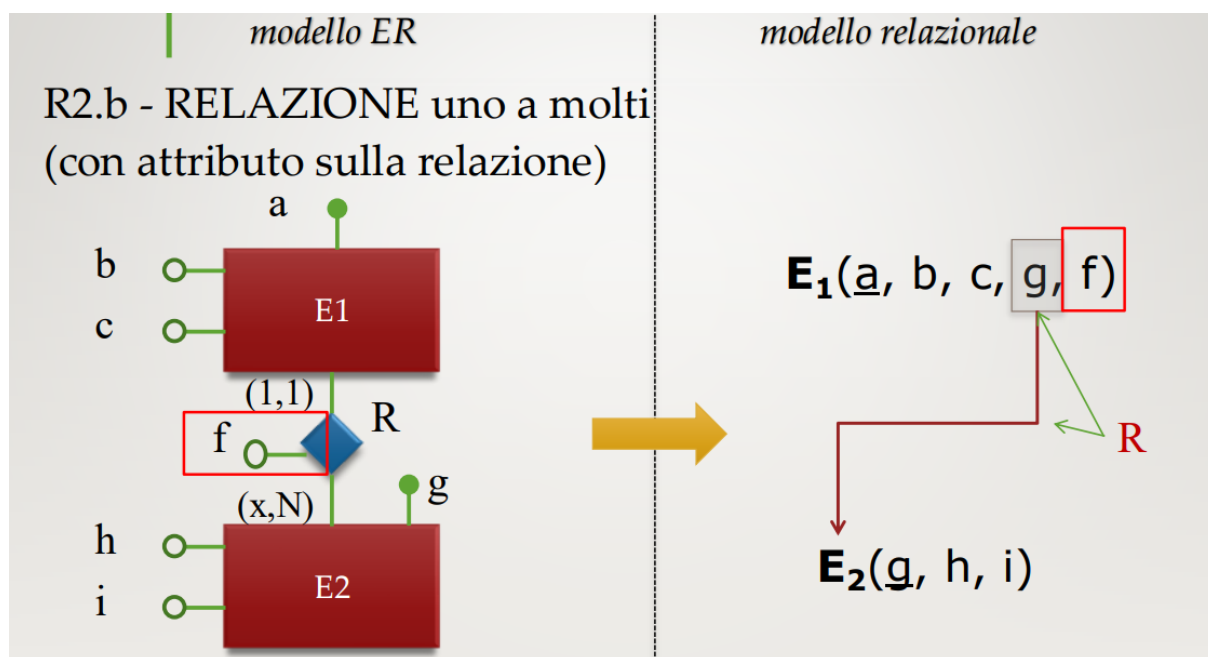


Figura 37: Relazione uno a molti con attributo sulla relazione

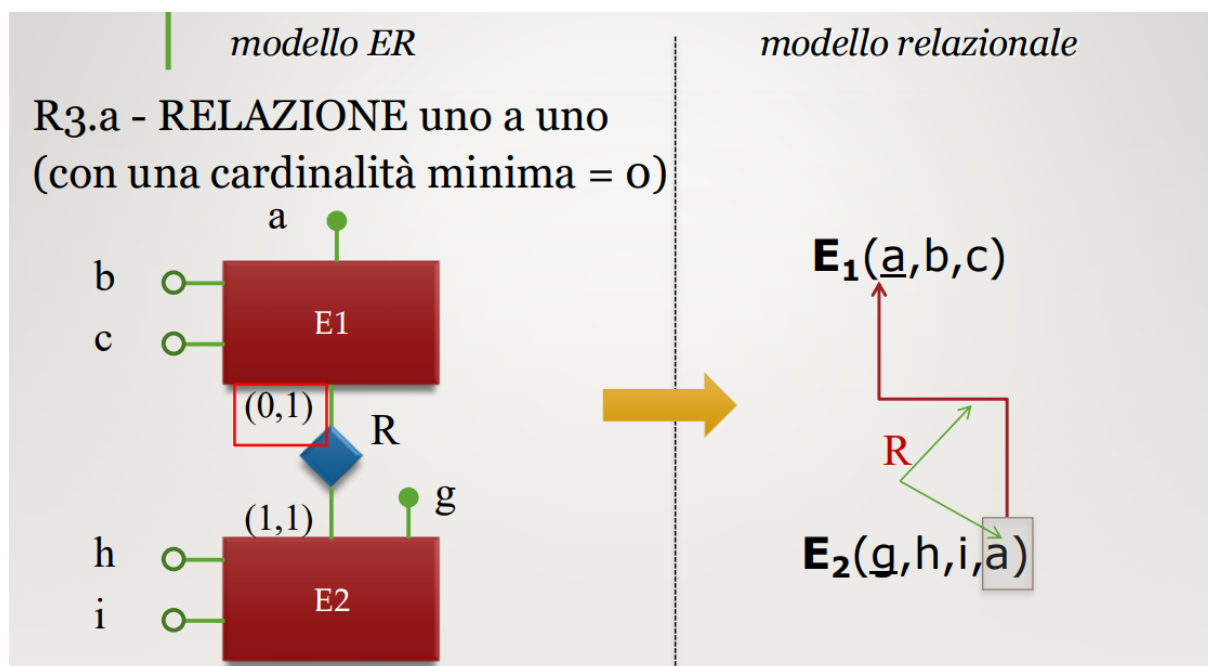


Figura 38: RELAZIONE uno a uno (con una cardinalità minima = 0)

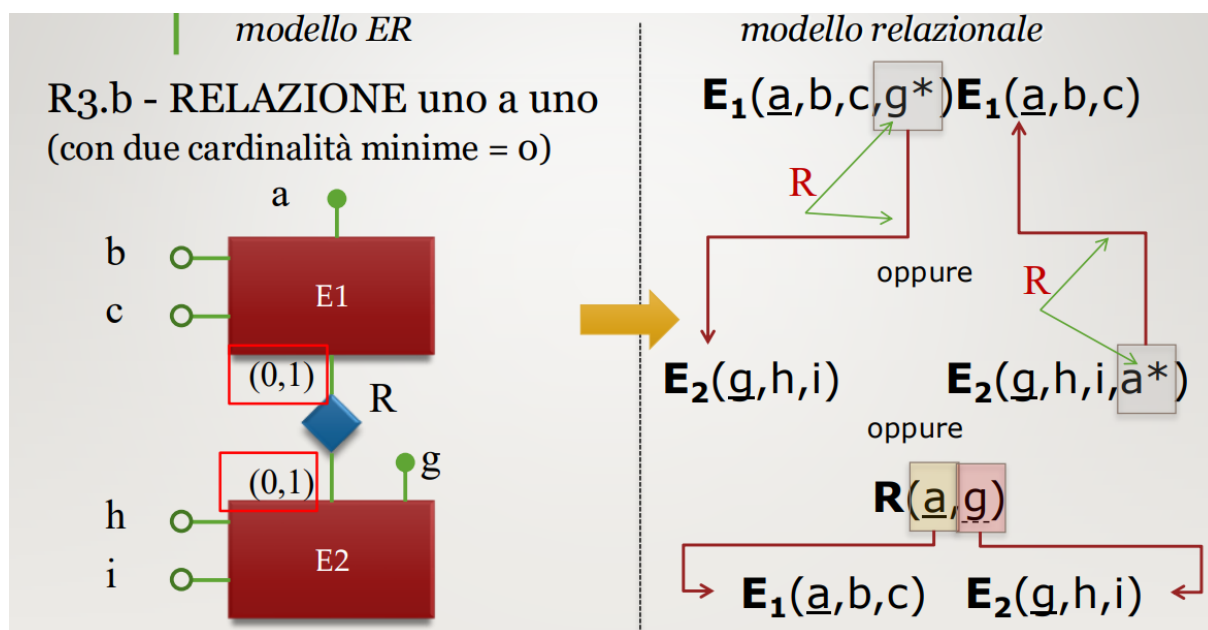


Figura 39: RELAZIONE uno a uno (con due cardinalità minime = 0)

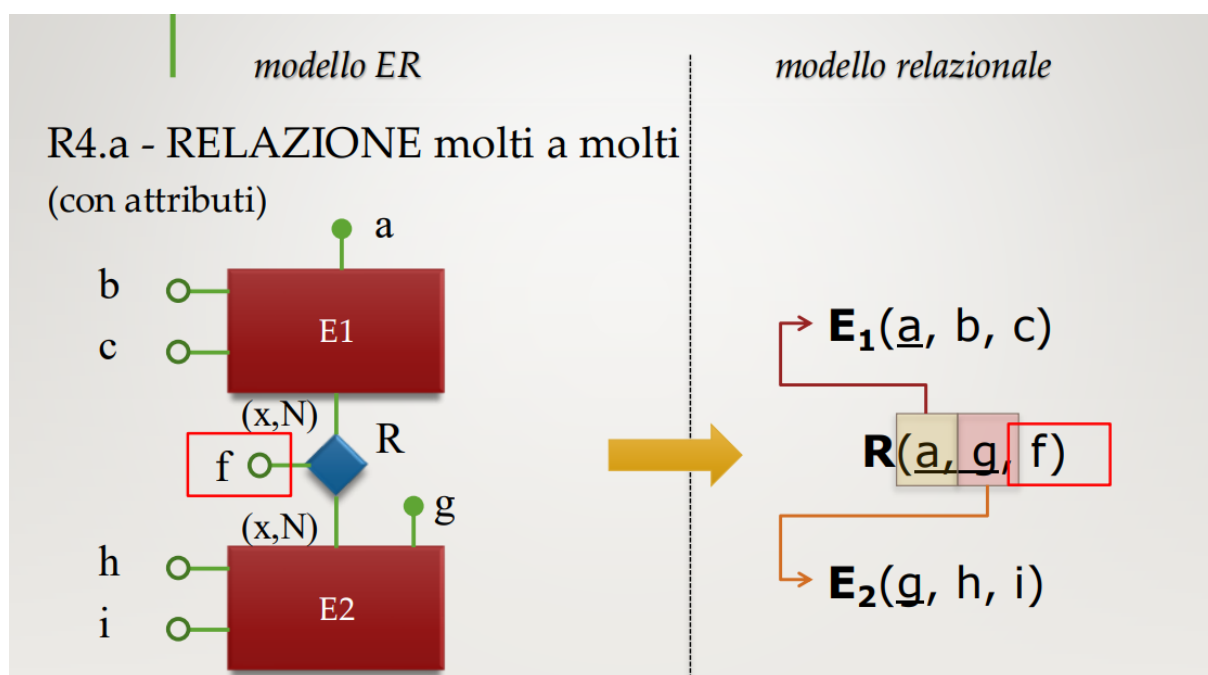


Figura 40: - RELAZIONE molti a molti (con attributi)

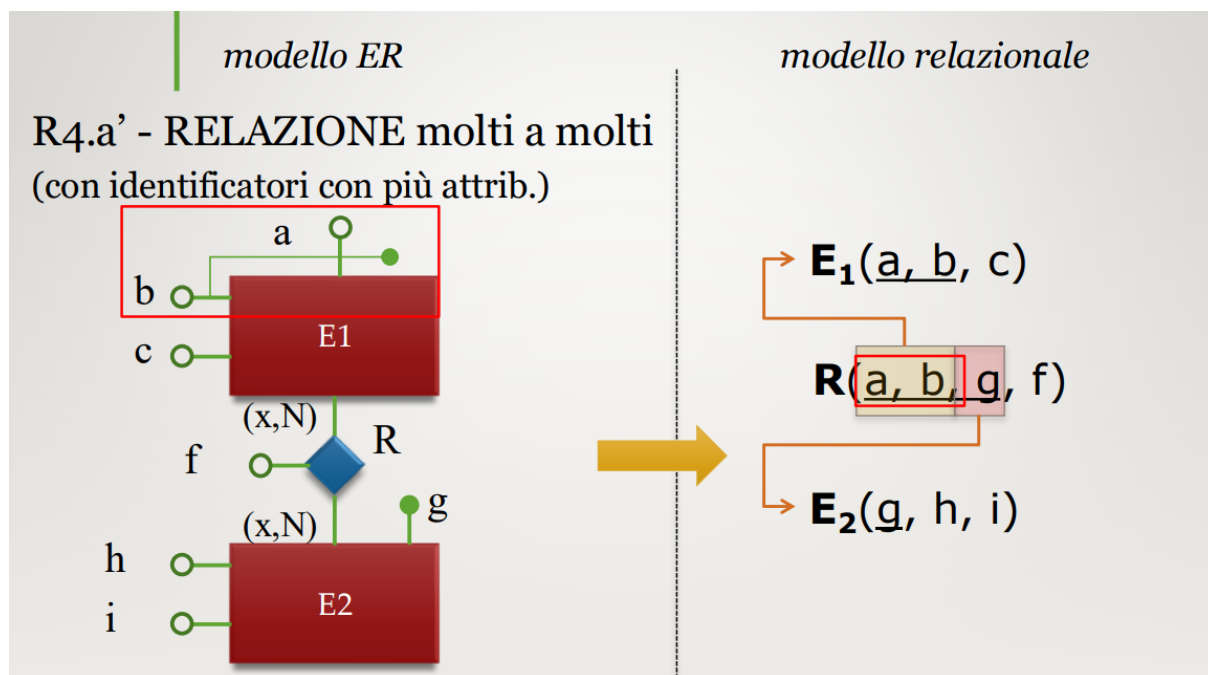


Figura 41: RELAZIONE molti a molti (con identificatori con più attrib.)

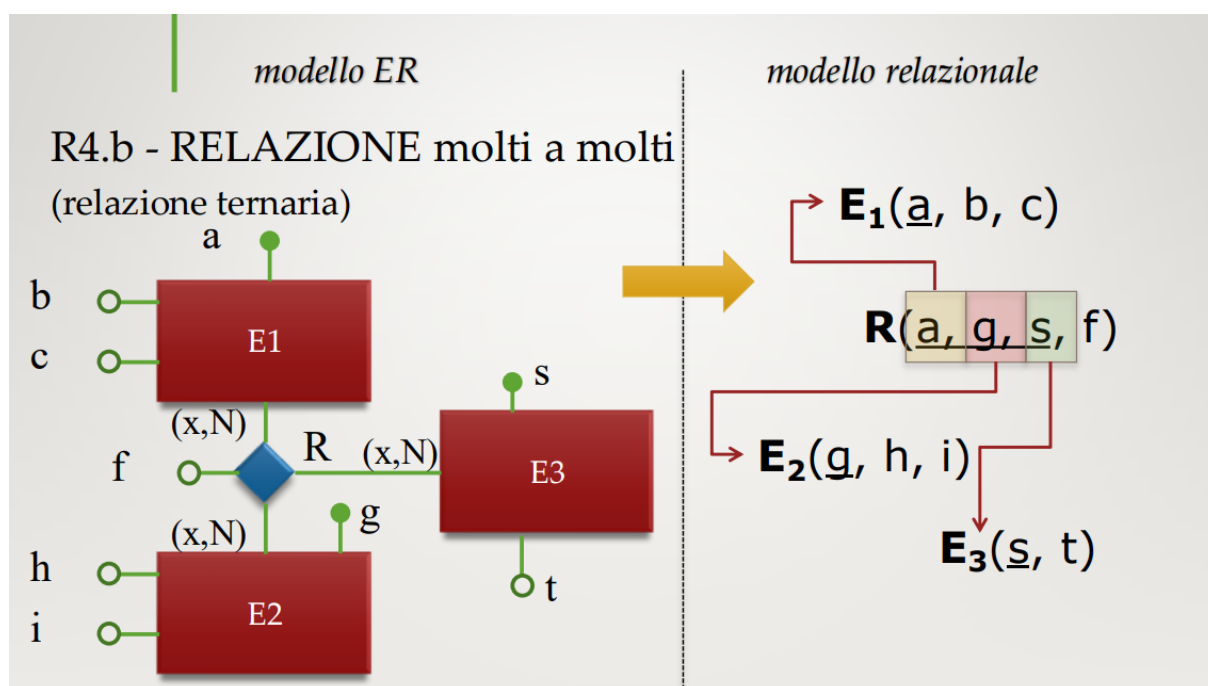


Figura 42: RELAZIONE molti a molti (relazione ternaria)

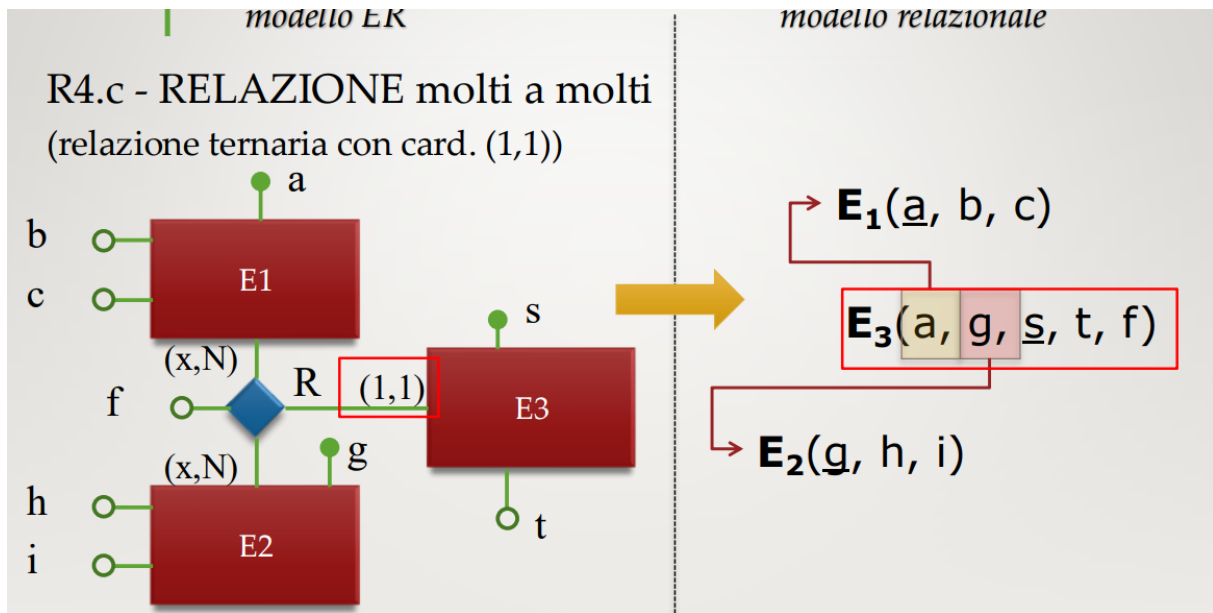


Figura 43: RELAZIONE molti a molti (relazione ternaria con card. (1,1))

6 Tecnologie NoSQL: Sistemi Document-Based

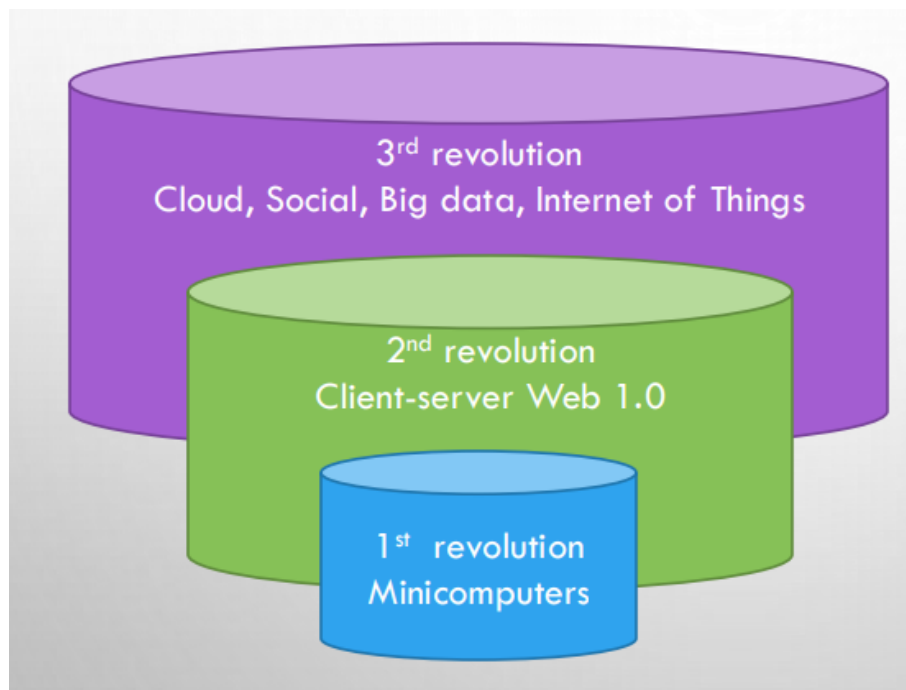


Figura 44: Evoluzione Database

La figura mostra l'evoluzione storica delle tecnologie di database attraverso tre grandi rivoluzioni, rappresentate come livelli sovrapposti. Ogni rivoluzione introduce nuovi modelli, nuovi paradigmi e nuove esigenze dovute ai cambiamenti tecnologici del periodo.

Prima rivoluzione (1950–1960)

Alla base si trova la prima rivoluzione dei database, nata nell'epoca dei **minicomputer**. In questo periodo emergono i primi sistemi per la gestione dei dati, caratterizzati da strutture rigide e fortemente dipendenti dall'organizzazione fisica dei file. Le principali tecnologie di questo periodo sono:

- database **gerarchici**;
- database **reticolari** (network databases);
- file organizzati tramite **ISAM** (Indexed Sequential Access Method).

Seconda rivoluzione (1970–1990)

Il secondo livello corrisponde alla diffusione del modello **relazionale**, che nasce negli anni '70 grazie ai lavori di E. F. Codd. In questa fase il paradigma relazionale si impone come standard grazie alla sua semplicità, astrazione e indipendenza dalla struttura fisica dei dati. È l'epoca dei sistemi **client-server** e del Web 1.0.

Terza rivoluzione (2005–oggi)

La terza rivoluzione si sviluppa nell'era del **cloud**, dei **big data** e dell'**Internet of Things**. In questo contesto il volume, la varietà e la velocità dei dati aumentano notevolmente, rendendo

necessarie nuove soluzioni oltre al modello relazionale classico. Accanto ai database tradizionali si diffondono:

- sistemi **NoSQL**;
- sistemi **NewSQL**;
- piattaforme di **Big Data**.

Questa rivoluzione rappresenta un ampliamento, non una sostituzione: i database relazionali restano fondamentali, ma vengono affiancati da tecnologie progettate per gestire grandi moli di dati distribuiti.

6.1 Approcci NoSQL

I modelli NoSQL nascono per rispondere ai limiti dei tradizionali database relazionali in scenari caratterizzati da grandi quantità di dati, alta eterogeneità e necessità di scalare orizzontalmente. Questi sistemi adottano strutture più flessibili e meno vincolate rispetto allo schema rigido delle tabelle relazionali, permettendo una gestione più efficiente di dati distribuiti su larga scala. Tra gli approcci più diffusi troviamo:

- **Sistemi Key-Value**: tutti i dati sono rappresentati tramite coppie (chiave, valore), dove la chiave identifica le istanze e il valore può avere una qualsiasi struttura, anche non omogenea. Un esempio di questo approccio è Hadoop.
- **Sistemi Document-Store**: i dati sono rappresentati come collezioni di documenti con struttura complessa e senza schema fisso. Esempi: **MongoDB**, **CouchBase**.
- **Column Store**: i dati sono rappresentati tramite record con struttura variabile. Esempi: **BigTable**, **HBase**, **Cassandra**.
- **Graph-Store**: i dati sono rappresentati come grafi, con nodi e archi. Esempi: **Neo4j**, **OrientDB**.

6.2 Sistemi Document-Store

Caratteristiche principali:

- i dati sono organizzati come collezioni di **documenti**;
- ogni documento può contenere dati complessi e nidificati;
- l'**incapsulamento** è fondamentale: tutte le informazioni relative a un'entità stanno nello stesso documento;
- non è richiesto uno schema fisso: i documenti di una collezione possono avere strutture diverse.

Progettazione dei dati come documenti

Decisioni fondamentali:

- scelta della chiave di accesso;
- struttura interna dei documenti e livello di incapsulamento;
- rappresentazione delle relazioni tra documenti.

Le relazioni possono essere modellate tramite:

- **riferimenti** (ID dell'altro documento);
- **documenti nidificati**.

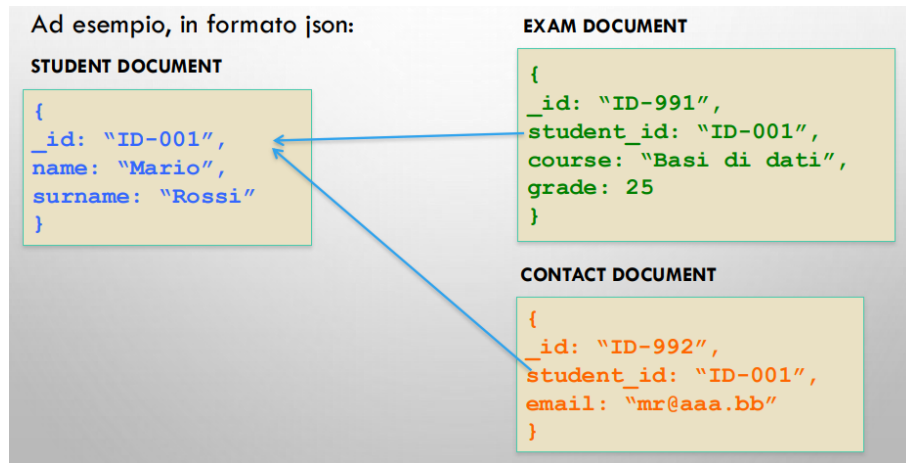


Figura 45: Formato JSON

6.3 Regole di traduzione di uno schema ER in collezioni di documenti

Assunzioni preliminari:

- si adotta un approccio **embedded**;
- lo schema ER è privo di generalizzazioni;
- relazioni ternarie o superiori eliminate;
- identificate collezioni $\{DOC_1, \dots, DOC_N\}$.

Regole fondamentali

1. Ogni collezione DOC_i corrisponde a un'entità principale.
2. Ogni istanza dell'entità principale produce un documento.
3. Le entità non principali vengono incapsulate nella principale.
4. Le relazioni sono rappresentate tramite incapsulamento o riferimenti.

6.3.1 Prima Fase (entità principali)

Ad esempio, se la collezione **DOC₃** ha come entità principale l'entità **E₂**, questa va etichettata come segue:

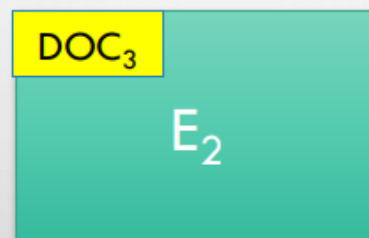


Figura 46: Individuazione entità principali

6.3.2 Seconda Fase (incapsulamento entità non principali)

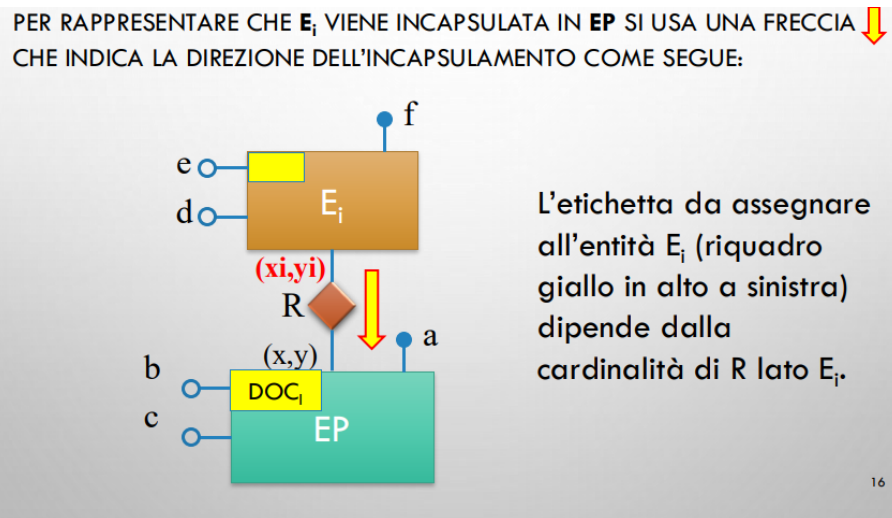


Figura 47: Incapsulamento

Diversi Casi

- (2.1 E 2.2) E_i È COLLEGATA ATTRAVERSO UNA RELAZIONE R AD UNA ENTITÀ PRINCIPALE EP (UNICA O SCELTA TRA QUELLE POSSIBILI). E_i VERRÀ QUINDI INCAPSULATA IN EP . E_i PARTECIPA ALLA RELAZIONE R CON VINCOLO DI CARDINALITÀ $(1,1)$.

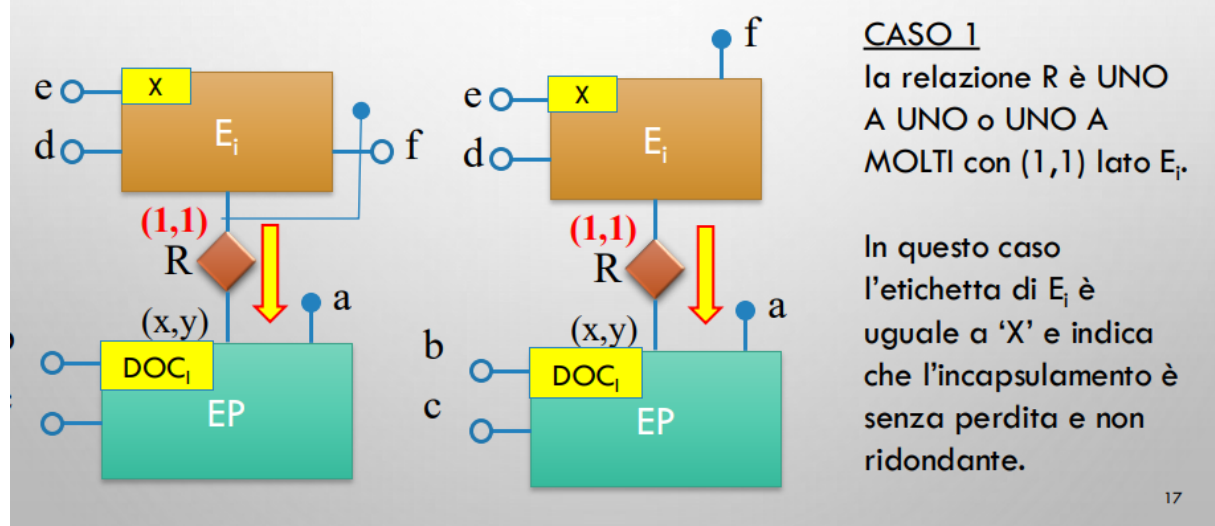
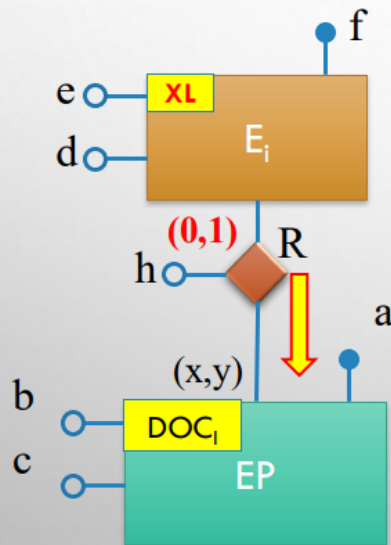


Figura 48: Uno a uno o uno a molti con $(1,1)$

- (2.1 E 2.2) E_i È COLLEGATA ATTRAVERSO UNA RELAZIONE R AD UNA ENTITÀ PRINCIPALE EP (UNICA O A SCELTA). E_i VERRÀ QUINDI INCAPSULATA IN EP . E_i PARTECIPA ALLA RELAZIONE R CON VINCOLO DI CARDINALITÀ $(0,1)$.



CASO 2

la relazione R è UNO A UNO o UNO A MOLTI con $(0,1)$ lato E_i .

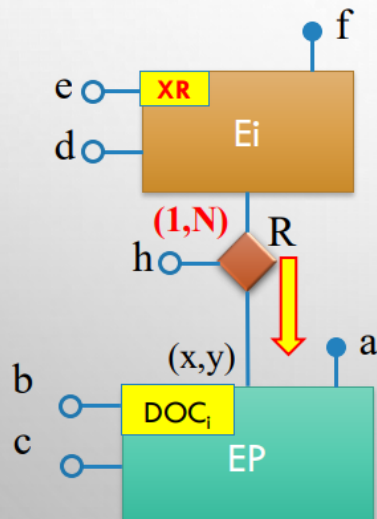
In questo caso l'etichetta di E_i è uguale a 'XL' e indica che si perdono le istanze di E_i che non sono collegate a nessuna istanza di EP . Ciò viene indicato dal suffisso L (loss) aggiunto all'etichetta di E_i .

18

Figura 49: Uno a uno o uno a molti con $(0,1)$

SECONDA FASE(INCAPSULAMENTO DELLE ENTITA' NON PRINCIPALI)

- (2.1 E 2.2) E_i È COLLEGATA ATTRAVERSO UNA RELAZIONE R AD UNA ENTITÀ PRINCIPALE EP (UNICA O A SCELTA). E_i VERRÀ QUINDI INCAPSULATA IN EP . E_i PARTECIPA ALLA RELAZIONE R CON VINCOLO DI CARDINALITÀ $(1,N)$.



CASO 3

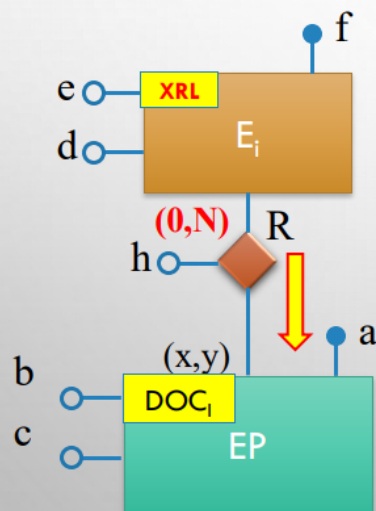
la relazione R è MOLTI A MOLTI o MOLTI A UNO con $(1,N)$ lato E_i .

In questo caso le istanze di E_i vengono rappresentate in modo ridondante in quanto sono collegate a più istanze di EP . Ciò viene indicato dal suffisso R (redundant) aggiunto all'etichetta di E_i .

19

Figura 50: Molti a molti o molti a uno con $(1,N)$

- (2.1 E 2.2) E_i È COLLEGATA ATTRAVERSO UNA RELAZIONE R AD UNA ENTITÀ PRINCIPALE EP (UNICA O A SCELTA). E_i VERRÀ QUINDI INCAPSULATA IN EP . E_i PARTECIPA ALLA RELAZIONE R CON VINCOLO DI CARDINALITÀ $(0,N)$.



CASO 4

la relazione R è MOLTI A MOLTI o MOLTI A UNO con $(0,N)$ lato E_i .

In questo caso le istanze di E_i vengono rappresentate in modo ridondante in quanto sono collegate a più istanze di EP e si perdono quelle non collegate. Ciò viene indicato dall'aggiunta del suffisso RL all'etichetta di E_i .

20

Figura 51: Molti a molti o molti a uno con $(0,N)$ lato E_i

6.3.3 Terza Fase (relazioni non strutturali)

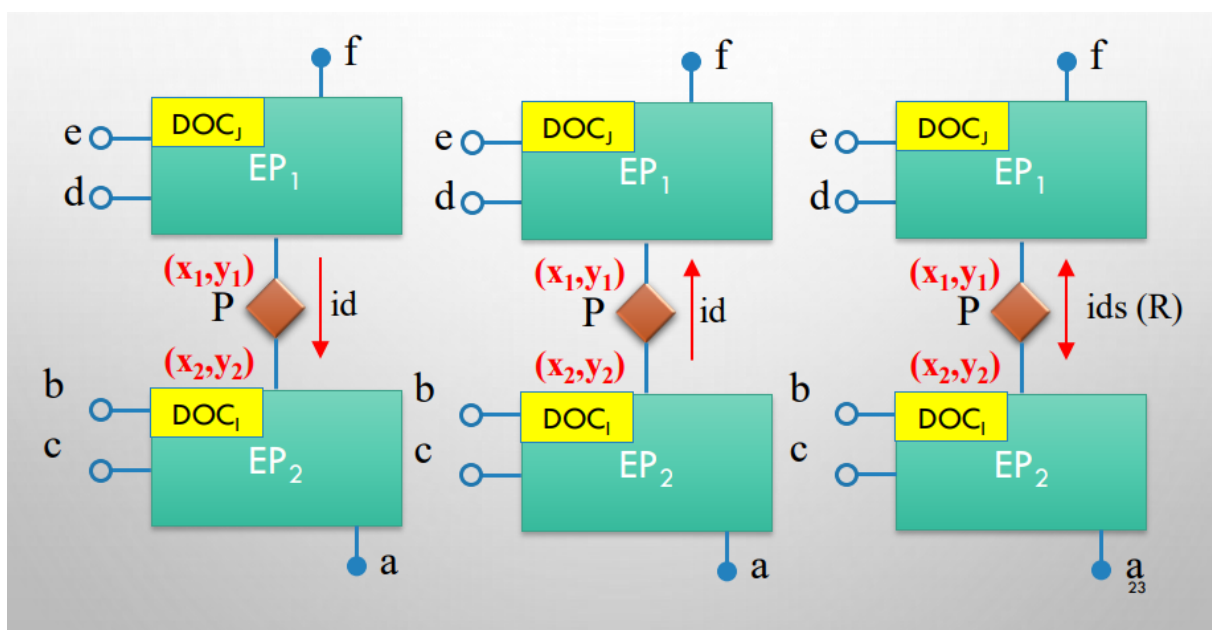


Figura 52: Etichettatura dello schema ER

Casi particolari

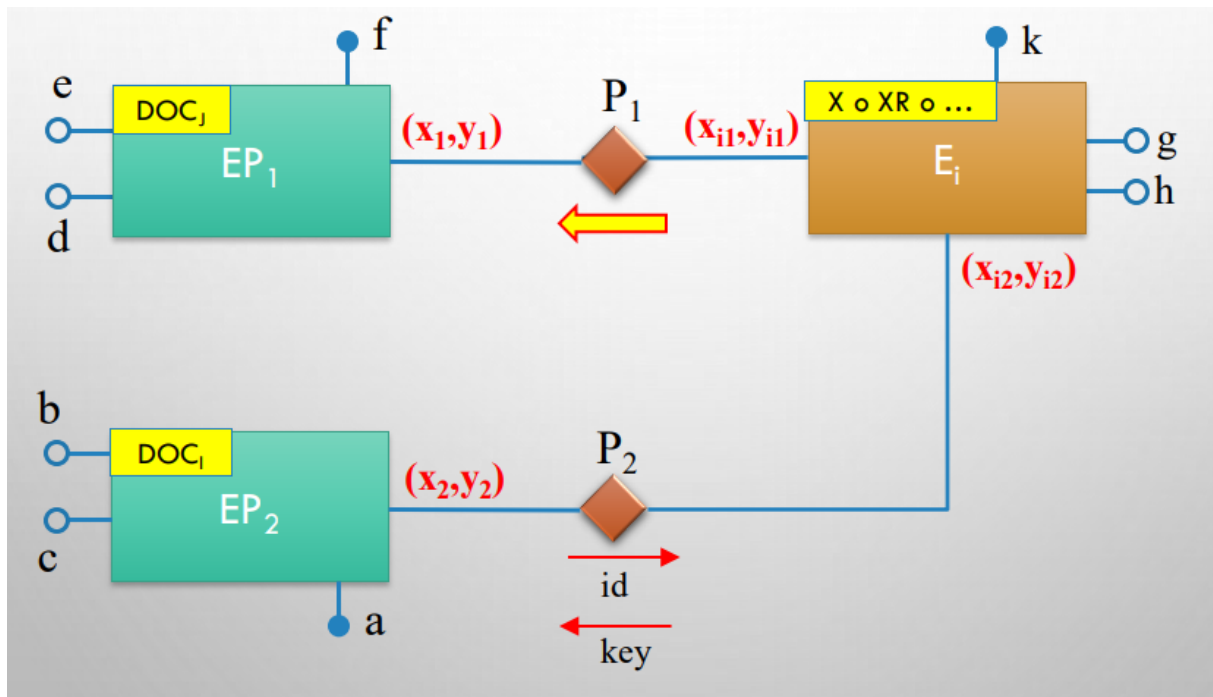


Figura 53: Caso Particolare A

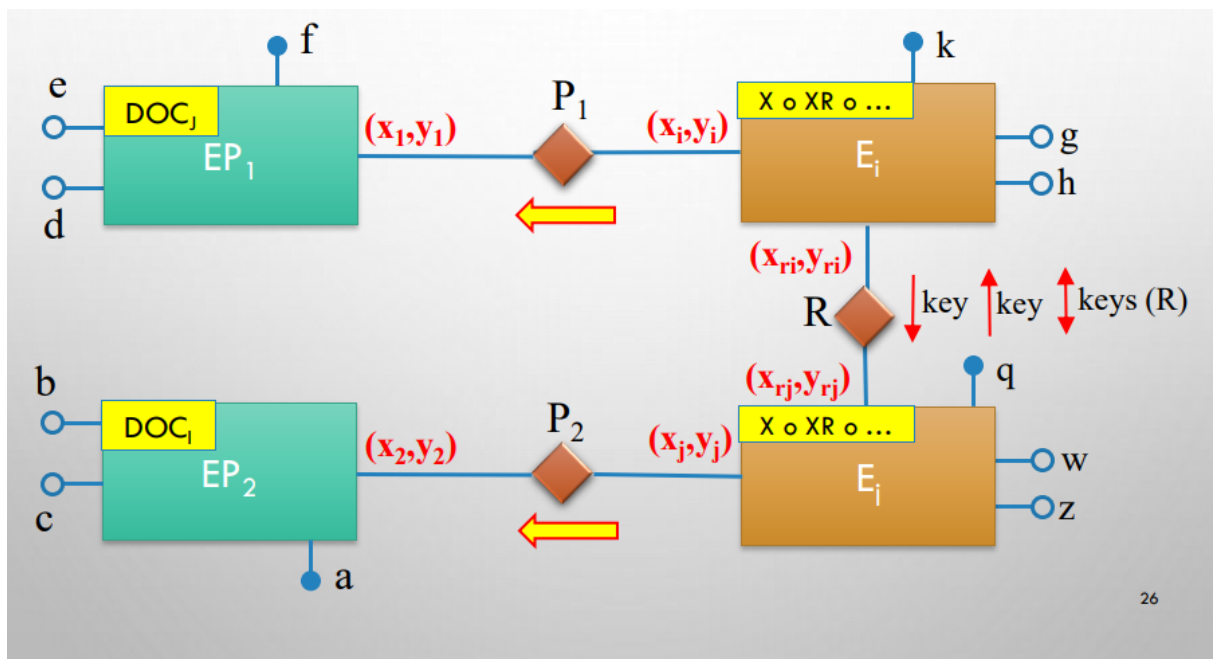


Figura 54: Caso Particolare B

6.4 Quarta fase

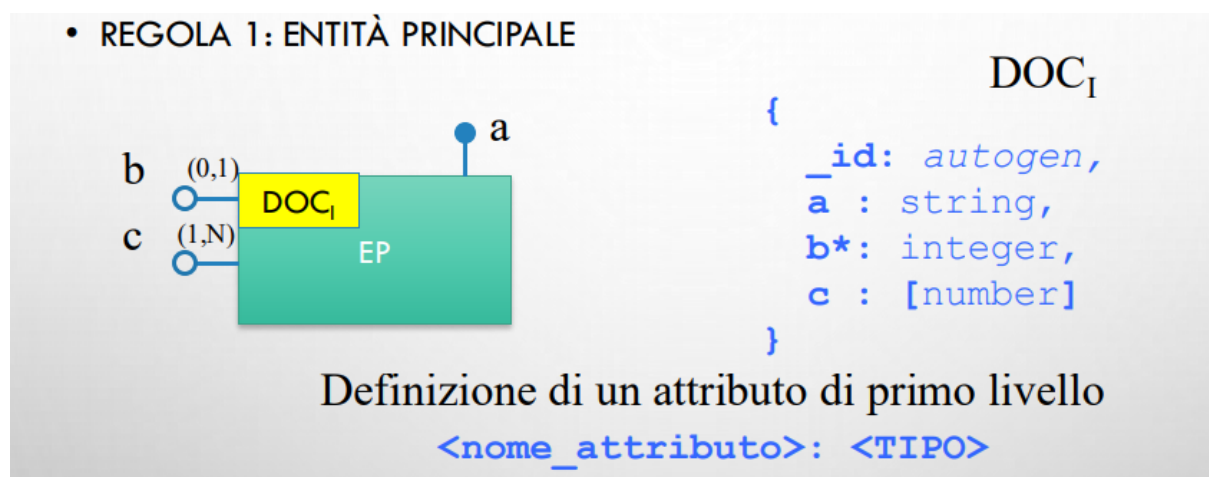
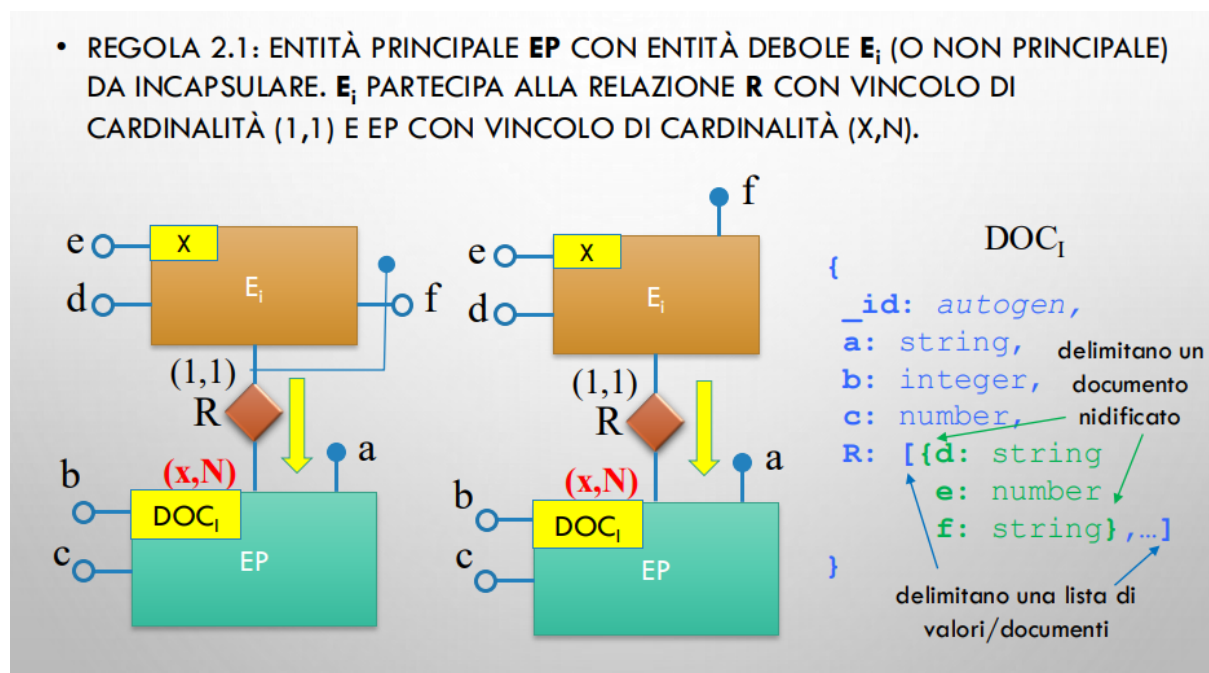


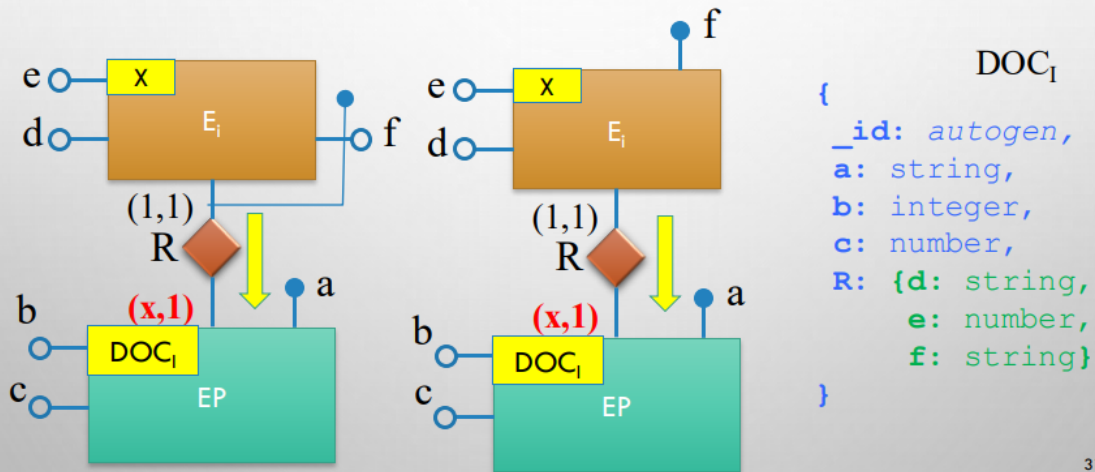
Figura 55: Entità Principale

<TIPO> può essere: string, integer, number, boolean, date, time, timestamp.

L'attributo `_id` è sempre autogenerato dal sistema.

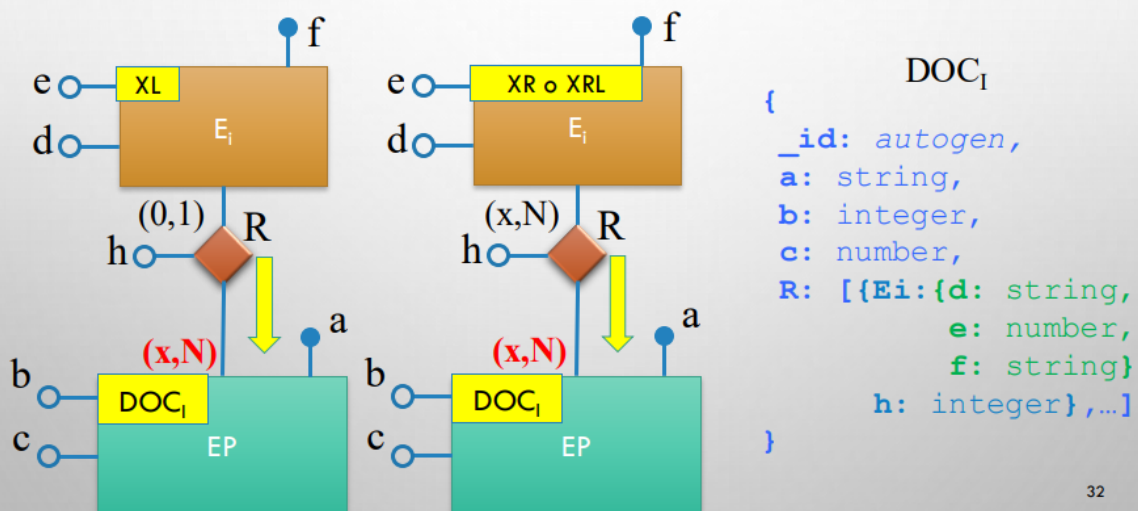


- REGOLA 2.2: ENTITÀ PRINCIPALE **EP** CON ENTITÀ DEBOLE **E_i** (O NON PRINCIPALE) DA INCAPSULARE. **E_i** PARTECIPA ALLA RELAZIONE **R** CON VINCOLO DI CARDINALITÀ (1,1) E **EP** CON VINCOLO DI CARDINALITÀ (X,1).



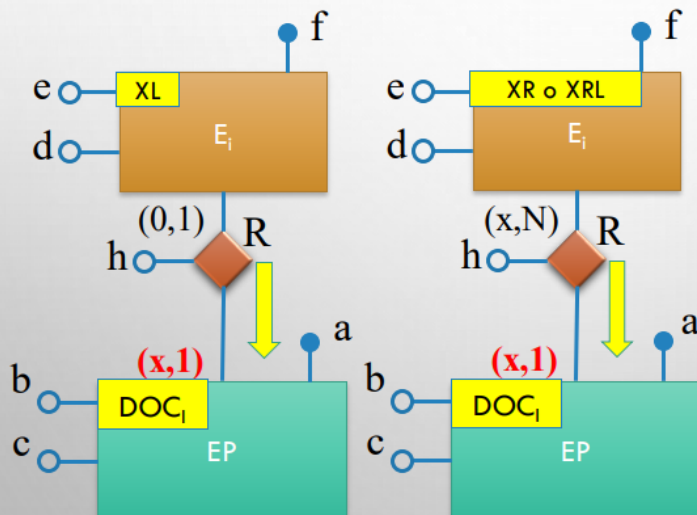
31

- REGOLA 3.1: ENTITÀ PRINCIPALE **EP** CON ENTITÀ DEBOLE **E_i** (O NON PRINCIPALE) DA INCAPSULARE. **E_i** PARTECIPA ALLA RELAZIONE **R** CON VINCOLO DI CARDINALITÀ (0,1) O (0,N) O (1,N) ED **EP** CON VINCOLO DI CARDINALITÀ (X,N).



32

- REGOLA 3.2: ENTITÀ PRINCIPALE **EP** CON ENTITÀ DEBOLE **E_i** (O NON PRINCIPALE) DA INCAPSULARE. **E_i** PARTECIPA ALLA RELAZIONE **R** CON VINCOLO DI CARDINALITÀ (0,1) O (0,N) O (1,N) E **EP** CON VINCOLO DI CARDINALITÀ (X,1).



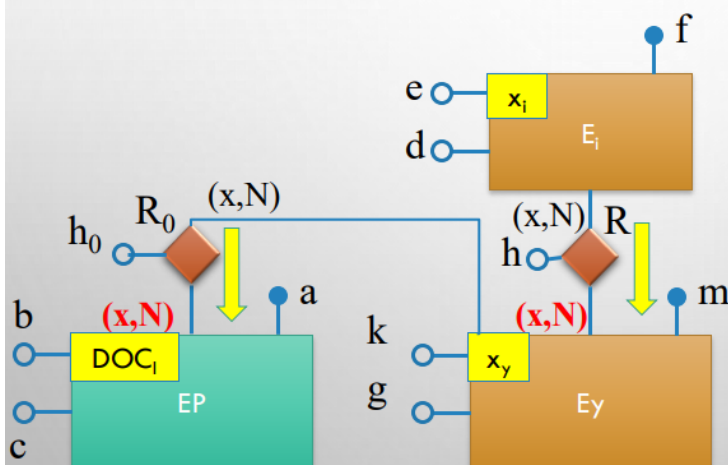
```

DOC1
{
  _id: autogen,
  a: string,
  b: integer,
  c: number,
  R: {Ei: {d: string,
            e: number,
            f: string},
      h: integer}
}

```

33

- REGOLA 4.1: **E_i** È COLLEGATA ATTRAVERSO UNA RELAZIONE **R** AD UNA ENTITÀ **E_y** (UNICA O A SCELTA) A SUA VOLTA COLLEGATA CON UN'ENTITÀ **EP**. **E_i** VIENE INCAPSULATA IN **E_y** E **E_y** VIENE INCAPSULATA IN **EP**. VINCOLI SU **E_y** E **EP** PARI A (0,N) O (1,N).



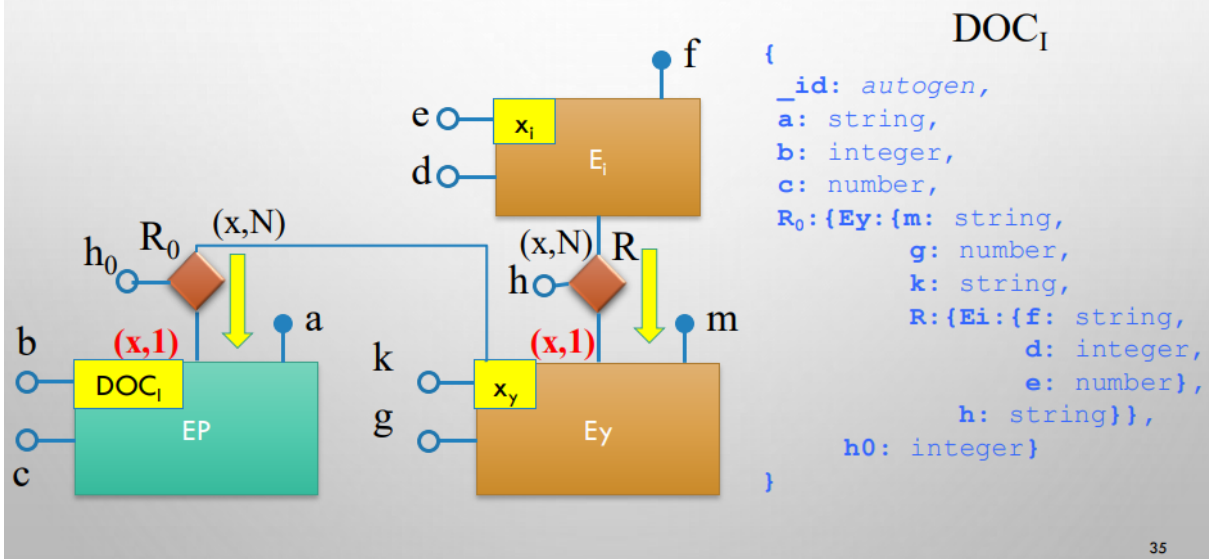
```

DOC1
{
  _id: autogen,
  a: string,
  b: integer,
  c: number,
  R0: [{Ey: {m: string,
              g: number,
              k: string,
              R: [{Ei: {f: string,
                      d: integer,
                      e: number},
                  h: string} ,...]} ,
        h0: integer} ,...]}
}

```

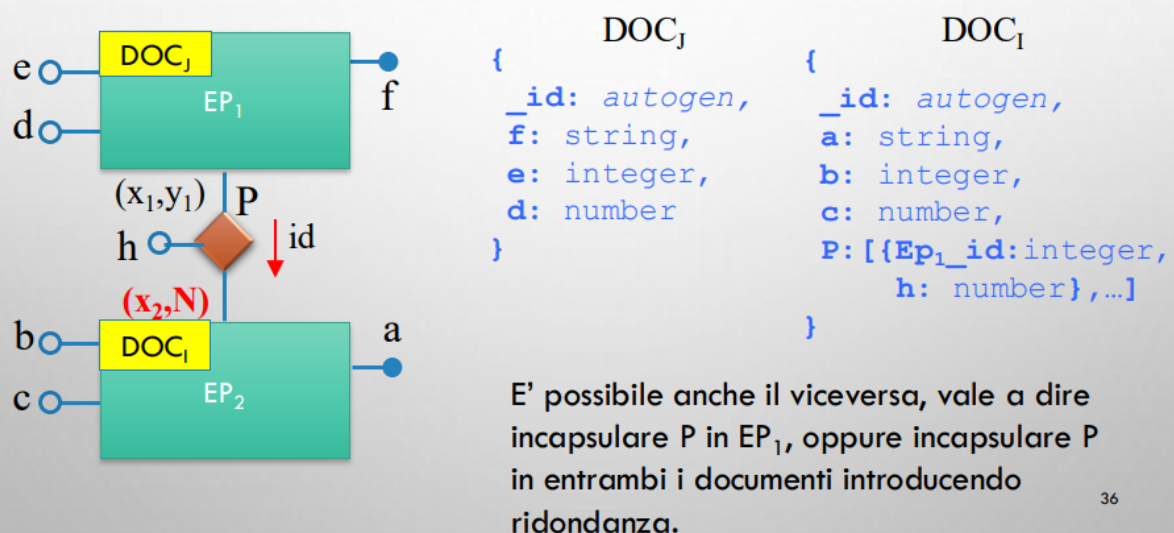
34

- REGOLA 4.2: E_i È COLLEGATA ATTRAVERSO UNA RELAZIONE R AD UNA ENTITÀ E_y (UNICA O A SCELTA) A SUA VOLTA COLLEGATA CON UN'ENTITÀ EP . E_{i1} VIENE INCAPSULATA IN E_y E E_y VIENE INCAPSULATA IN EP . VINCOLI SU E_y E EP PARI A (0,1) O (1,1).



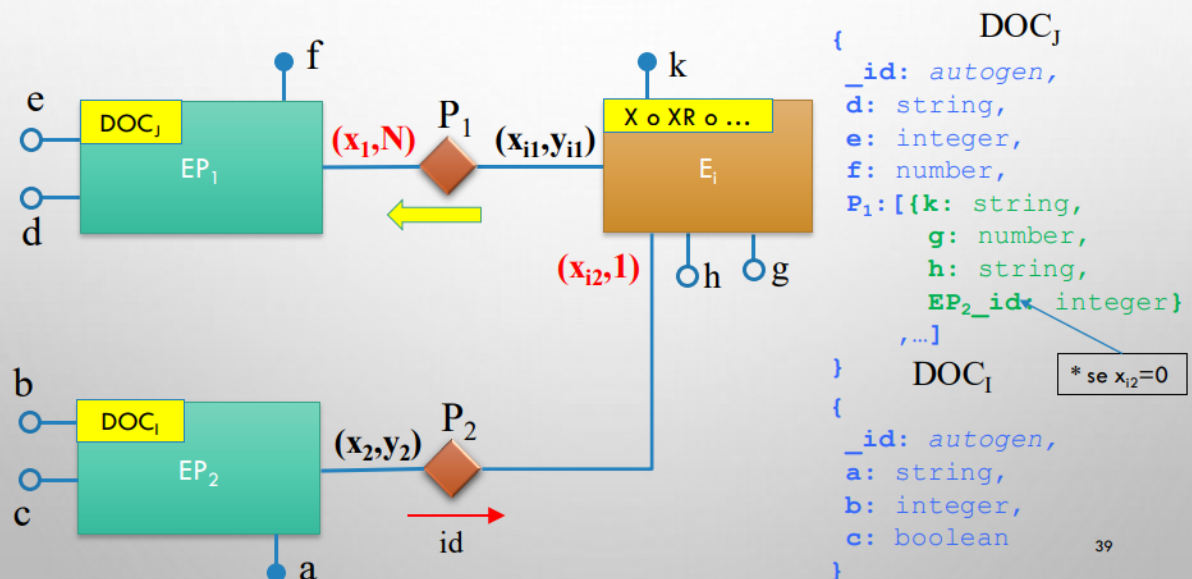
35

- REGOLA 5.1: RELAZIONE MOLTI A MOLTI TRA ENTITÀ PRINCIPALI EP_1 E EP_2 . EP_2 PARTECIPA ALLA RELAZIONE R CON VINCOLO DI CARDINALITÀ (0,N) O (1,N) E INCAPSULO R SOLO IN EP_2 .



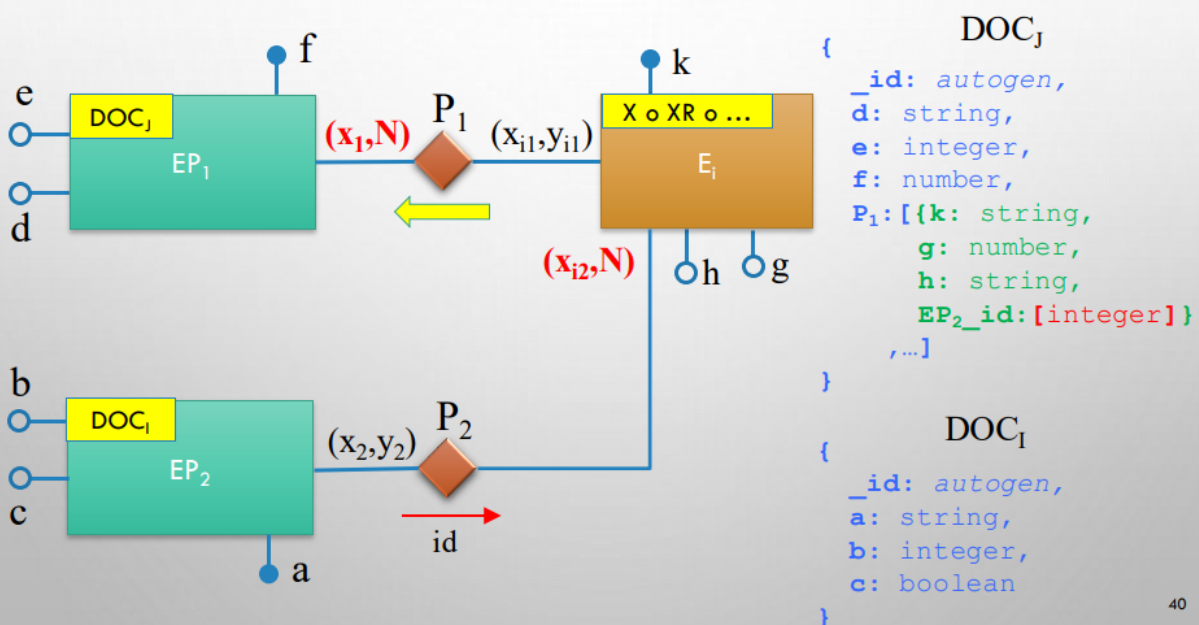
36

- REGOLA 6.1: RELAZIONE BINARIA **P2** TRA UN'ENTITÀ NON PRINCIPALE **E_i**, DA INCAPSULARE IN **EP₁**, E UN'ENTITÀ PRINCIPALE **EP₂** CON VINCOLO (X,1) LATO **E_i**.



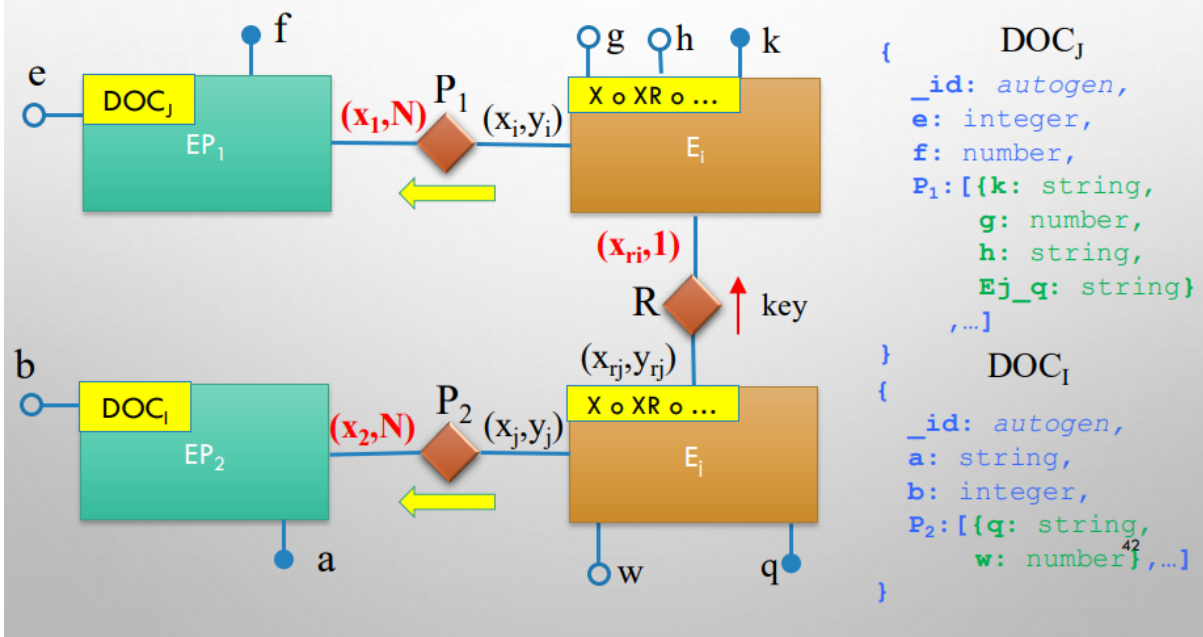
39

- REGOLA 6.2: RELAZIONE BINARIA **P2** TRA UN'ENTITÀ NON PRINCIPALE **E_i**, DA INCAPSULARE IN **EP₁**, E UN'ENTITÀ PRINCIPALE **EP₂** CON VINCOLO (X,N) LATO **E_i**.

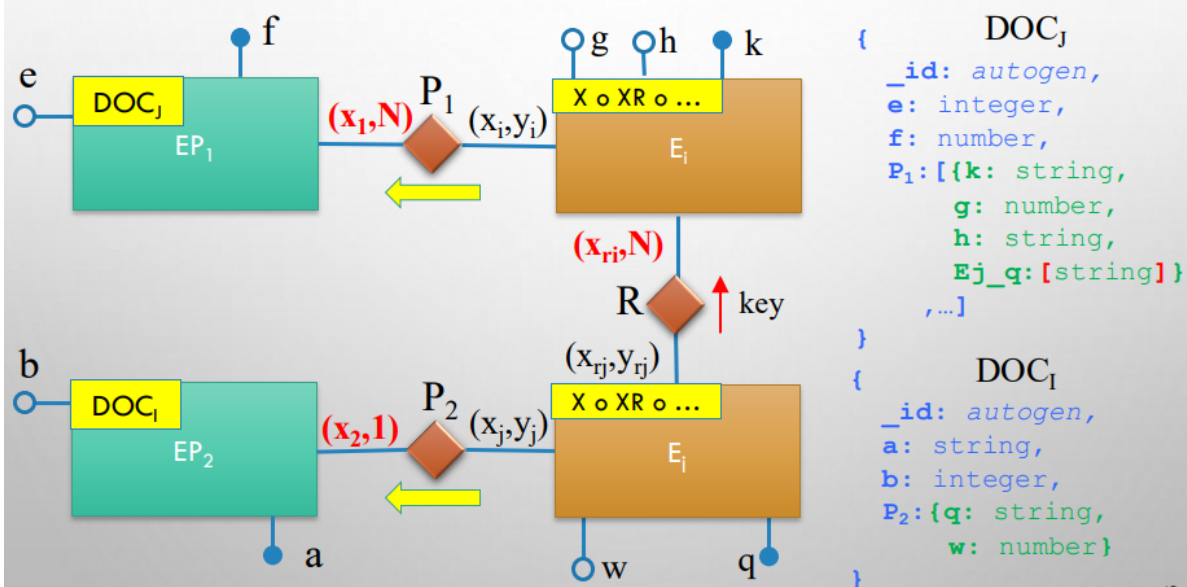


40

- REGOLA 7.1: RELAZIONE BINARIA **P2** TRA DUE ENTITÀ NON PRINCIPALI E_i E E_j , DA INCAPSULARE IN EP_1 E EP_2 RISPETTIVAMENTE, CON VINCOLO $(X,1)$ LATO E_i .



- REGOLA 7.2: RELAZIONE BINARIA **P2** TRA DUE ENTITÀ NON PRINCIPALI E_i E E_j DA INCAPSULARE IN EP_1 E EP_2 RISPETTIVAMENTE CON VINCOLO (X,N) LATO E_i .



7 Algebra relazionale

L'**algebra relazionale** è un linguaggio formale procedurale che definisce un insieme di **operatori** applicabili a relazioni (tabelle) e che producono come risultato **nuove relazioni**. Essa costituisce il fondamento teorico dei linguaggi di interrogazione dei database relazionali, come SQL.

Ogni operatore dell'algebra relazionale è **chiuso**, ovvero restituisce sempre una relazione come risultato, permettendo la composizione di più operatori in un'unica espressione.

La **segnatura** di un operatore descrive:

- il numero di relazioni di input;
- l'eventuale presenza di parametri;
- il tipo del risultato prodotto.

Un **operatore unario** agisce su una singola relazione di input e produce una relazione di output:

$$Op_{parametri}(r_1) \rightarrow r_2$$

dove:

- r_1 è la relazione di input;
- Op è l'operatore applicato;
- i *parametri* specificano il comportamento dell'operatore;
- r_2 è la relazione risultante.

Un **operatore binario** agisce su due relazioni di input e produce una nuova relazione:

$$r_1 Op_{parametri} r_2 \rightarrow r_3$$

dove:

- r_1 e r_2 sono le relazioni di input;
- Op è l'operatore binario;
- i *parametri* definiscono le condizioni dell'operazione;
- r_3 è la relazione risultante.

7.1 Classificazione degli operatori

Gli operatori dell'algebra relazionale si classificano secondo due criteri principali:

Classificazione funzionale

- Operatori insiemistici
- Operatori specifici
- Operatori di giunzione (join) tra relazioni

Classificazione in base alla derivabilità

- Operatori di base
- Operatori derivati

7.2 Operatori insiemistici

Poiché le relazioni sono insiemi di tuple, è naturale applicare ad esse le operazioni dell'algebra degli insiemi. È tuttavia necessario ricordare che una relazione è un insieme di tuple **omogenee**, cioè definite sullo stesso insieme di attributi (stesso schema).

Di conseguenza, gli operatori insiemistici possono essere applicati esclusivamente a relazioni che hanno **lo stesso schema**.

Dati due relazioni r_1 e r_2 definite sullo stesso schema X , si definiscono i seguenti operatori:

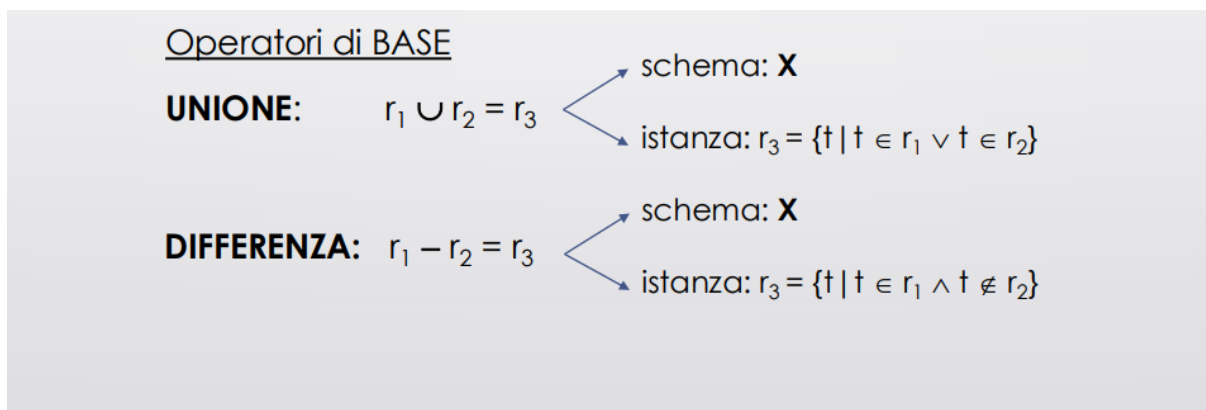


Figura 56: operazione di unione e differenza

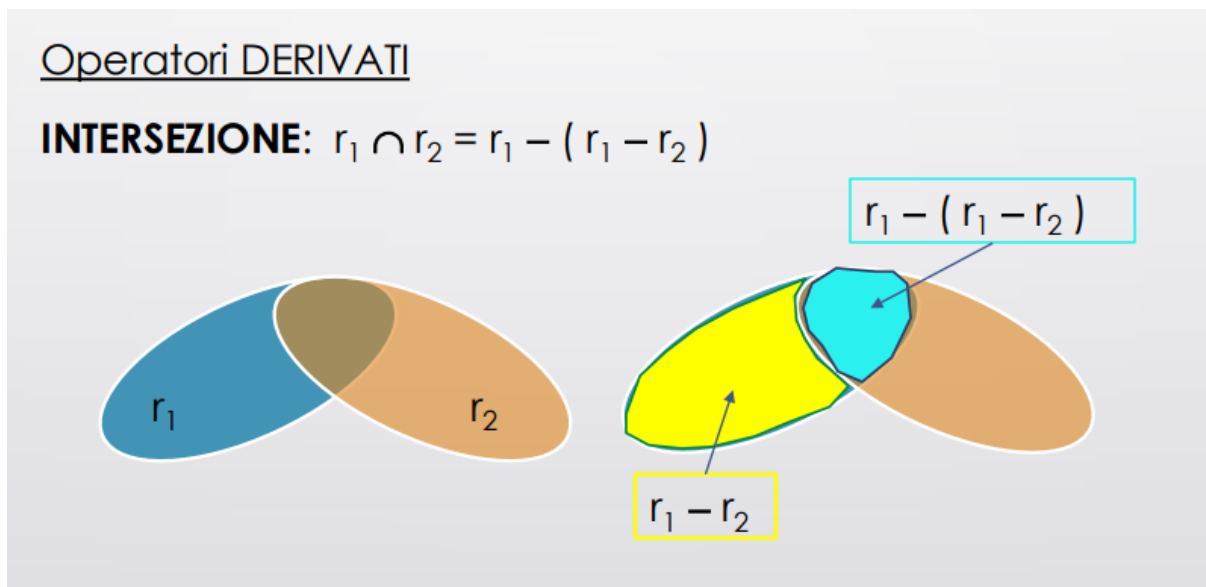


Figura 57: Intersezione

7.2.1 Cardinalità degli operatori insiemistici

La **cardinalità** di una relazione indica il numero di tuple contenute nella relazione risultante dall'applicazione di un operatore dell'algebra relazionale.

Unione

L'operatore di unione combina le tuple di due relazioni eliminando le eventuali tuple duplicate. La cardinalità della relazione risultante dipende dal numero di tuple comuni tra le due relazioni.

Il valore minimo si ottiene quando una relazione è interamente contenuta nell'altra; in questo caso, la cardinalità dell'unione coincide con quella della relazione più grande. Il valore massimo si ottiene quando le due relazioni non hanno tuple in comune; in questo caso, la cardinalità dell'unione è pari alla somma delle cardinalità delle due relazioni.

$$\max(|r_1|, |r_2|) \leq |r_1 \cup r_2| \leq |r_1| + |r_2|$$

Differenza

L'operatore di differenza restituisce le tuple appartenenti alla prima relazione che non sono presenti nella seconda relazione. La cardinalità della relazione risultante può variare da zero fino alla cardinalità della prima relazione.

Il valore minimo si ottiene quando tutte le tuple della prima relazione sono presenti anche nella seconda, producendo una relazione vuota. Il valore massimo si ottiene quando nessuna tupla della prima relazione è presente nella seconda, per cui tutte le tuple della prima relazione vengono mantenute.

$$0 \leq |r_1 - r_2| \leq |r_1|$$

Intersezione

L'operatore di intersezione restituisce le tuple comuni a entrambe le relazioni. La cardinalità della relazione risultante dipende quindi dal numero di tuple condivise.

Il valore minimo si ottiene quando le due relazioni non hanno tuple in comune, producendo una relazione vuota. Il valore massimo si ottiene quando la relazione più piccola è interamente contenuta nella relazione più grande.

$$0 \leq |r_1 \cap r_2| \leq \min(|r_1|, |r_2|)$$

7.3 Operatori specifici

7.3.1 Operatore di rinominaizione

Dato una relazione r definita sullo schema X , con

$$X = \{A_1, \dots, A_n\},$$

si definiscono i seguenti operatori. L'operatore di **rinominazione** consente di modificare il nome degli attributi di una relazione senza alterarne le tuple. Sia

$$Y = \{B_1, \dots, B_n\}$$

un insieme di attributi tale che $|Y| = |X|$.

La rinominazione è definita come:

$$\rho_{A_1 \rightarrow B_1, \dots, A_n \rightarrow B_n}(r) = r_2$$

dove:

- lo schema della relazione risultante è Y ;
- l'istanza r_2 contiene le stesse tuple di r ;
- per ogni tupla $t \in r$ e per ogni $i \in \{1, \dots, n\}$, vale $t[B_i] = t[A_i]$.

Esempio

Schema relazionale:

- CAPOLUOGO_PROV(Nome, Popolazione, CAP)
- COMUNI_MINORI(Nome, Abitanti, CAP)

Per produrre la lista di tutti i comuni è necessario rendere compatibili gli schemi delle due relazioni. Si applica quindi l'operatore di rinominazione per uniformare il nome degli attributi:

$$\rho_{\text{Popolazione} \rightarrow \text{Abitanti}}(\text{CAPOLUOGO_PROV}) \cup \text{COMUNI_MINORI}$$

Dato una relazione r definita sullo schema X , con

$$X = \{A_1, \dots, A_n\},$$

si definiscono i seguenti operatori.

7.3.2 Operatore di Selezione

L'operatore di **selezione** consente di estrarre da una relazione le tuple che soddisfano una determinata condizione.

La selezione è definita come:

$$\sigma_F(r_1) = r_2$$

dove:

- lo schema della relazione risultante è lo stesso di r_1 ;
- l'istanza risultante è

$$r_2 = \{t \mid t \in r_1 \wedge F(t)\};$$

- $F(t)$ indica che la tupla t rende vera la formula F .

La formula $F(t)$ è una formula proposizionale ottenuta combinando, mediante i connettivi logici $\wedge$, $\vee$ e $\neg$, condizioni atomiche del tipo:

$$A \theta B \quad \text{oppure} \quad A \theta c$$

dove:

- A e B sono attributi della relazione;
- $\theta \in \{=, \neq, >, \geq, <, \leq\}$;
- c è una costante appartenente al dominio di A o compatibile con esso.

Una formula del tipo $A \theta B$ è vera sulla tupla t se $t[A] \theta t[B]$, mentre una formula del tipo $A \theta c$ è vera se $t[A] \theta c$.

Le formule composte assumono valori booleani determinati dalle tavole di verità dei connettivi logici utilizzati.

Esempio

Schema relazionale:

- TRENO(Numero, PartOra, PartMin, Cat, Dest, ArrOra, ArrMin)
- FERMATA(NumTreno, Stazione, Ora, Min)

Si richiede di produrre la lista di tutti i treni non regionali che partono alle ore 12:00 o dopo le 12:00 e prima delle 16:00.

L'espressione in algebra relazionale è:

$$\sigma_{\text{PartOra} \geq 12 \wedge \text{PartOra} < 16 \wedge \text{Cat} \neq \text{'REGIONALE'}}(\text{TRENO})$$

7.3.3 Proiezione

L'operatore di proiezione consente di ridurre una relazione eliminando alcuni attributi, mantenendo solo quelli di interesse. La riduzione avviene in senso verticale.

Sia $Y = \{A_1, \dots, A_m\} \subseteq X$ un sottoinsieme degli attributi dello schema X della relazione r_1 .

La proiezione è definita come:

$$\pi_Y(r_1) = r_2$$

dove:

- lo schema della relazione risultante è Y ;
- l'istanza risultante è

$$r_2 = \{t \mid \exists t' \in r_1 : t'[Y] = t\};$$

- $t'[Y]$ indica la tupla ottenuta da t' considerando solo gli attributi appartenenti a Y ;
- eventuali tuple duplicate vengono eliminate.

Esempio

Schema relazionale:

- TRENO(Numero, PartOra, PartMin, Cat, Dest, ArrOra, ArrMin)
- FERMATA(NumTreno, Stazione, Ora, Min)

Si richiede di estrarre il **numero** e l'**ora di partenza** dei treni *Freccia Rossa* (categoria = 'FR') con destinazione *Milano Centrale*.

L'espressione in algebra relazionale è:

$$\pi_{\text{Numero}, \text{PartOra}}(\sigma_{\text{Dest} = \text{'Milano Centrale'} \wedge \text{Cat} = \text{'FR'}}(\text{TRENO}))$$

7.3.4 Cardinalità degli operatori specifici

La cardinalità indica il numero di tuple contenute nella relazione risultato di un'operazione di algebra relazionale.

Selezione: L'operatore di selezione σ filtra le tuple di una relazione in base a una condizione logica, senza modificare gli attributi.

La cardinalità della relazione risultante soddisfa la seguente relazione:

$$0 \leq |\sigma_F(r_1)| \leq |r_1|$$

Questo significa che:

- la selezione può restituire nessuna tupla;
- può restituire tutte le tuple della relazione originale;
- non può mai aumentare il numero di tuple.

Proiezione L'operatore di proiezione Π seleziona un sottoinsieme di attributi della relazione ed elimina eventuali tuple duplicate.

La cardinalità della relazione risultante è compresa tra:

$$\min(|r_1|, 1) \leq |\Pi_Y(r_1)| \leq |r_1|$$

In particolare:

- la proiezione può ridurre il numero di tuple a causa dell'eliminazione dei duplicati;
- nel caso migliore, la cardinalità rimane invariata.

Se l'insieme di attributi Y è una superchiave per la relazione r_1 , allora non si generano duplicati e vale:

$$|\Pi_Y(r_1)| = |r_1|$$

7.3.5 Operatori di giunzione (JOIN)

Dati due relazioni r_1 e r_2 con schema rispettivamente X_1 e X_2 , gli operatori di *join* generano tuple nella relazione risultato a partire da coppie di tuple $(t_1, t_2) \in r_1 \times r_2$ che soddisfano una determinata condizione, detta *predicato di join*.

7.3.6 JOIN naturale

Il join naturale è un operatore di base in cui il predicato di join è definito automaticamente sugli attributi comuni alle due relazioni. Tale operatore è quindi dipendente dallo schema.

Lo schema della relazione risultante è:

$$\text{schema}(r_3) = X_1 \cup X_2$$

dove gli attributi comuni compaiono una sola volta.

L'istanza della relazione risultato è definita come:

$$r_3 = \{ t \mid \exists t_1 \in r_1, \exists t_2 \in r_2 : t[X_1] = t_1 \wedge t[X_2] = t_2 \}$$

Il predicato di join è una condizione di uguaglianza sugli attributi comuni alle due relazioni.

Esempio

Si consideri il seguente schema relazionale:

$$\begin{aligned} &\text{DOCENTE}(\underline{CFDocente}, Nome, Cognome) \\ &\text{CORSO}(Nome, \underline{CFDocente}) \end{aligned}$$

Si richiede di produrre l'insieme dei corsi riportando il nome del corso e il cognome del docente che lo tiene.

Poiché le informazioni richieste sono distribuite su due relazioni diverse, è necessario applicare un operatore di join. Il collegamento tra le relazioni avviene tramite l'attributo comune **CFDocente**.

Dal momento che l'attributo **Nome** della relazione **CORSO** deve essere rinominato in **NomeCorso**, si applica preliminarmente l'operatore di rinomina.

L'espressione di algebra relazionale è la seguente:

$$\Pi_{NomeCorso, Cognome}(\rho_{Nome \rightarrow NomeCorso}(CORSO) \bowtie DOCENTE)$$

L'operatore di join naturale combina le tuple delle due relazioni sulla base dell'uguaglianza dell'attributo comune **CFDocente**. Successivamente, la proiezione consente di ottenere esclusivamente gli attributi richiesti nel risultato finale.

DOCENTE			CORSO	
CFDocente	Nome	Cognome	Nome	CFDocente
BLSLRT	Alberto	Belussi	Basi di dati TEORIA	BLSLRT
QNTLSA	Elisa	Quintarelli	Basi di dati LAB	MGLSRA
MGLSRA	Sara	Migliorini	Programmazione	QNTLSA
			Data integration	QNTLSA

$$\Pi_{NomeCorso, Cognome}(\rho_{Nome \rightarrow NomeCorso}(CORSO) \bowtie DOCENTE)$$

NomeCorso	Cognome
Basi di dati TEORIA	Belussi
Basi di dati LAB	Migliorini
Programmazione	Quintarelli
Data integration	Quintarelli

Figura 58: Esempio più visivo del JOIN naturale

Proprietà del JOIN naturale

Il join naturale tra due relazioni r_1 e r_2 si dice *completo* se tutte le tuple delle due relazioni contribuiscono a generare almeno una tupla nella relazione risultato.

Formalmente, il join naturale è completo se valgono entrambe le seguenti condizioni:

$$\forall t_1 \in r_1 : \exists t \in r_1 \bowtie r_2 \text{ tale che } t[X_1] = t_1$$

$$\forall t_2 \in r_2 : \exists t \in r_1 \bowtie r_2 \text{ tale che } t[X_2] = t_2$$

Queste condizioni garantiscono che ogni tupla di r_1 e di r_2 compaia almeno una volta nel risultato del join.

Se il join naturale non è completo, le tuple che non contribuiscono alla generazione di alcuna tupla nel risultato si dicono **dangling tuples**. Il join naturale gode di alcune proprietà algebriche analoghe a quelle degli operatori classici dell'algebra.

Commutatività: Il join naturale è commutativo:

$$r_1 \bowtie r_2 = r_2 \bowtie r_1$$

Associatività: Il join naturale è associativo:

$$r_1 \bowtie (r_2 \bowtie r_3) = (r_1 \bowtie r_2) \bowtie r_3$$

Relazioni con lo stesso schema: Se le relazioni r_1 e r_2 hanno lo stesso schema, allora il join naturale coincide con l'intersezione:

$$r_1 \bowtie r_2 = r_1 \cap r_2$$

Assenza di attributi comuni: Se le relazioni r_1 e r_2 non hanno attributi in comune, allora il join naturale coincide con il prodotto cartesiano:

$$r_1 \bowtie r_2 = r_1 \times r_2$$

7.3.7 Cardinalità del JOIN naturale

In generale, la cardinalità del join naturale tra due relazioni r_1 e r_2 è compresa tra zero e il prodotto delle cardinalità delle due relazioni:

$$0 \leq |r_1 \bowtie r_2| \leq |r_1| \cdot |r_2|$$

Se il join naturale è completo, allora ogni tupla di entrambe le relazioni contribuisce a generare almeno una tupla nel risultato, e vale:

$$\max(|r_1|, |r_2|) \leq |r_1 \bowtie r_2| \leq |r_1| \cdot |r_2|$$

Se l'insieme degli attributi comuni $X_1 \cap X_2$ è una superchiave per la relazione r_2 , allora a ogni tupla di r_1 può corrispondere al più una tupla di r_2 , e quindi:

$$0 \leq |r_1 \bowtie r_2| \leq |r_1|$$

Se $X_1 \cap X_2$ è una superchiave per r_2 ed esiste un vincolo di integrità referenziale tra $X_1 \cap X_2$ (o una sua parte) di r_1 e r_2 , allora ogni tupla di r_1 trova corrispondenza in r_2 e vale:

$$|r_1 \bowtie r_2| = |r_1|$$

7.3.8 THETA JOIN

Date due relazioni r_1 e r_2 di schema X_1 e X_2 rispettivamente, si introduce, oltre al *join naturale*, il seguente operatore. **Theta Join**: il predicato di join viene indicato esplicitamente ed è indipendente dallo schema delle relazioni.

Precondizione

Gli schemi delle relazioni devono essere disgiunti:

$$X_1 \cap X_2 = \emptyset$$

Definizione

Il theta join è definito come una selezione applicata al prodotto cartesiano:

$$r_1 \bowtie_{\theta} r_2 = \sigma_{\theta}(r_1 \times r_2)$$

dove:

- $\times$ è il prodotto cartesiano
- σ_{θ} è l'operatore di selezione
- θ è un predicato conforme alla sintassi della selezione

Il predicato θ può contenere confronti tra attributi delle due relazioni utilizzando operatori di confronto come $=, \neq, <, >, \leq, \geq$.

ESEMPIO

Schema relazionale:

→ DOCENTE(CF, Nome, Cognome)

CORSO(Nome, Docente)

Si richiede di produrre l'elenco di tutti i corsi riportando: nome del corso e cognome del docente.

$\Pi_{\text{NomeCorso, Cognome}}(\rho_{\text{Nome} \rightarrow \text{NomeCorso}}(\text{CORSO}) \bowtie_{\text{CF = Docente}} \text{DOCENTE})$

Figura 59: Esempio dell'uso del Theta Join

Proprietà del Theta Join

Il **theta join** si dice **equi-join** se la condizione θ è una congiunzione di uguaglianze:

$$r_1 \bowtie_{A_1=B_1 \wedge \dots \wedge A_n=B_n} r_2$$

N.B.: ATTENZIONE non esiste un operatore misto che applica la logica del *join naturale* per gli attributi comuni e aggiunge ulteriori condizioni specificate esplicitamente tramite il predicato θ .

Teorema

Date due relazioni r_1 e r_2 di schema X_1 e X_2 con

$$X_1 \cap X_2 = \{C_1, \dots, C_m\} \quad \text{con } m > 0,$$

vale la seguente equivalenza:

$$r_1 \bowtie r_2 = \Pi_{X_1 \cup X_2} \left(r_1 \bowtie_{C_1=C'_1 \wedge \dots \wedge C_m=C'_m} \rho_{C_1, \dots, C_m \rightarrow C'_1, \dots, C'_m}(r_2) \right)$$

Ridenominazione Data la relazione r_1 di schema

$$X_1 = \{A_1, \dots, A_n\},$$

per cambiare nome a tutti gli attributi si può scrivere:

$$\rho_{A_1, \dots, A_n \rightarrow A'_1, \dots, A'_n}(r_1)$$

oppure, in forma breve:

$$\rho_{X \rightarrow X'}(r_1)$$

N.B.: non sono ammesse notazioni simili alla *dot notation* per indicare l'attributo di una tabella.

La notazione $T.A$ (per indicare l'attributo A della tabella T) **NON è una notazione ammessa dall'algebra relazionale**.

7.4 Algebra con valori nulli

È opportuno estendere l'algebra relazionale affinché possa operare anche su relazioni che contengono valori nulli. In particolare, è necessario definire in modo esplicito il comportamento delle operazioni che coinvolgono il confronto tra attributi, poiché la presenza di valori nulli introduce ambiguità semantiche.

Le operazioni che richiedono una gestione specifica dei valori nulli sono:

- **Selezione**
- **Join naturale**

Le altre operazioni dell'algebra relazionale (proiezione, unione, differenza e prodotto cartesiano) si limitano a propagare i valori nulli eventualmente presenti nelle tuple di input, senza introdurre nuove problematiche semantiche.

Motivazione: La selezione e il join naturale si basano su confronti tra valori di attributi. In presenza di valori nulli, tali confronti non possono essere valutati secondo la logica booleana classica, ma richiedono una logica a tre valori (*vero*, *falso*, *sconosciuto*). Per questo motivo, è necessario estendere formalmente l'algebra relazionale per garantire un comportamento coerente e ben definito di queste operazioni.

7.4.1 Selezione

Le condizioni di selezione in presenza di valori nulli assumono i seguenti valori di verità:

- Sia $A \theta B$ una condizione atomica applicata a una tupla t . Se $t[A] = \text{NULL}$ oppure $t[B] = \text{NULL}$, allora l'espressione $t[A] \theta t[B]$ è valutata come **FALSO**.
- Sia $A \theta c$ una condizione atomica applicata a una tupla t , dove c è una costante. Se $t[A] = \text{NULL}$, allora l'espressione $t[A] \theta c$ è valutata come **FALSO**.
- Sono introdotte le seguenti condizioni atomiche aggiuntive:

$$A \text{ IS NULL} \quad \text{e} \quad A \text{ IS NOT NULL.}$$

L'espressione $A \text{ IS NULL}$, valutata sulla tupla t , è **VERO** se $t[A] = \text{NULL}$, altrimenti è **FALSO**. Viceversa, l'espressione $A \text{ IS NOT NULL}$ è **VERO** se $t[A] \neq \text{NULL}$, altrimenti è **FALSO**.

7.4.2 Join naturale

Nel join naturale, la condizione di uguaglianza sugli attributi comuni alle due relazioni è valutata come **FALSA** sulle tuple t_1 e t_2 se almeno uno degli attributi comuni di t_1 oppure di t_2 assume valore **NULL**.

Si osservi, in particolare, che il confronto

$$\text{NULL} = \text{NULL}$$

è sempre valutato come **FALSO**.

7.5 Ottimizzazione di espressioni DML

Ogni espressione DML (generalmente specificata tramite un linguaggio dichiarativo) ricevuta dal DBMS è sottoposta a un processo di elaborazione e ottimizzazione prima della sua esecuzione.

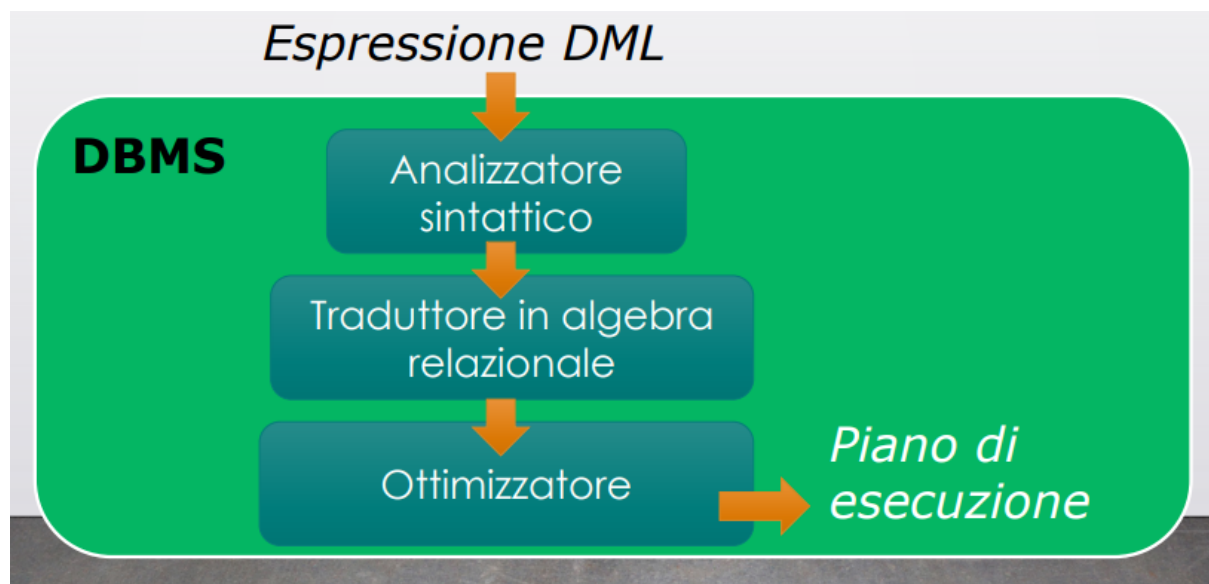


Figura 60: Ottimizzazione di espressioni DML

L'ottimizzatore genera un'espressione equivalente a quella di input, ma caratterizzata da un costo inferiore. Il costo viene stimato principalmente in termini di dimensione dei risultati intermedi, numero di accessi ai dati e operazioni eseguite.

Per ridurre il costo complessivo dell'esecuzione, l'ottimizzatore applica trasformazioni di equivalenza sull'espressione di input, con l'obiettivo di minimizzare la dimensione dei risultati intermedi e migliorare l'efficienza dell'elaborazione.

7.6 Equivalenza tra espressioni algebriche

7.6.1 Equivalenza dipendente dallo schema

Dato uno schema R , due espressioni algebriche E_1 ed E_2 si dicono *equivalenti rispetto allo schema R* , e si scrive:

$$E_1 \equiv_R E_2$$

se vale che

$$E_1(r) = E_2(r) \quad \text{per ogni istanza } r \text{ dello schema } R.$$

Esempio:

$$\Pi_{AB}(R_1) \bowtie \Pi_{AC}(R_2) \equiv_R \Pi_{ABC}(R_1 \bowtie R_2)$$

7.6.2 Equivalenza assoluta

L'equivalenza assoluta è indipendente dallo schema. Due espressioni E_1 ed E_2 si dicono *assolutamente equivalenti*, e si scrive:

$$E_1 \equiv E_2$$

se vale che

$$E_1 \equiv_R E_2 \quad \text{per ogni schema } R \text{ compatibile con } E_1 \text{ ed } E_2.$$

Esempio:

$$\Pi_{AB}(\sigma_{A>0}(R)) \equiv \sigma_{A>0}(\Pi_{AB}(R))$$

7.7 Trasformazioni di equivalenza

Sia E un'espressione di schema X . Si definiscono le seguenti trasformazioni di equivalenza.

Atomizzazione delle selezioni

$$\sigma_{F_1 \wedge F_2}(E) \equiv \sigma_{F_1}(\sigma_{F_2}(E))$$

Questa trasformazione è propedeutica ad altre trasformazioni. Da sola non produce un'ottimizzazione se non è seguita da ulteriori riscritture.

7.7.1 Idempotenza delle proiezioni

$$\Pi_Y(E) \equiv \Pi_Y(\Pi_Z(E)) \quad \text{con } Z \subseteq X$$

Anche questa trasformazione è propedeutica e non comporta un'ottimizzazione se applicata isolatamente.

7.8 Trasformazioni di equivalenza con il join

Siano E_1 ed E_2 espressioni di schema rispettivamente X_1 e X_2 .

Anticipazione delle selezioni rispetto al join

$$\sigma_F(E_1 \bowtie E_2) \equiv E_1 \bowtie \sigma_F(E_2)$$

La trasformazione è applicabile solo se la condizione F fa riferimento esclusivamente ad attributi di E_2 .

Anticipazione della proiezione rispetto al join

$$\Pi_{X_1 \cup Y}(E_1 \bowtie E_2) \equiv_R E_1 \bowtie \Pi_Y(E_2)$$

La trasformazione è applicabile solo se:

$$Y \subseteq X_2 \quad \text{e} \quad (X_2 - Y) \cap X_1 = \emptyset$$

7.9 Combinazione di anticipazione e idempotenza

Combinando l'anticipazione della proiezione con l'idempotenza delle proiezioni si ottengono le seguenti equivalenze:

$$\Pi_Y(E_1 \bowtie_F E_2) \equiv \Pi_Y(\Pi_{Y_1}(E_1) \bowtie_F \Pi_{Y_2}(E_2))$$

$$\Pi_Y(E_1 \bowtie E_2) \equiv \Pi_Y(\Pi_{Y_1}(E_1) \bowtie \Pi_{Y_2}(E_2))$$

dove:

- $Y_1 = (X_1 \cap Y) \cup J_1$
- $Y_2 = (X_2 \cap Y) \cup J_2$
- J_1 e J_2 sono gli attributi di E_1 ed E_2 coinvolti nel join. Nel caso di theta-join, tali attributi sono quelli presenti nella condizione F ; nel caso di join naturale vale:

$$J_1 = J_2 = X_1 \cap X_2$$

TRENO(NumTreno, OrarioPart, Cat, Dest, OrarioArr)
FERMATA(NumTreno, Stazione, Orario)

Q: Trovare il numero e l'ora di partenza dei treni freccia rossa che fermano a Vicenza.

$\Pi_{\text{NumTreno, OrarioPart}}(\sigma_{\text{Cat}='FR' \wedge \text{Stazione}='Vicenza'}(\text{TRENO} \bowtie \text{FERMATA}))$

Figura 61: Esempio di applicazione delle trasformazioni

Esempio di applicazione delle trasformazioni

TRENO(NumTreno, OrarioPart, Cat, Dest, OrarioArr)
 FERMATA(NumTreno, Stazione, Orario)

$\Pi_{\text{NumTreno}, \text{OrarioPart}}(\sigma_{\text{Cat}='FR' \wedge \text{Stazione}='Vicenza'}(\text{TRENO} \bowtie \text{FERMATA}))$

Atomizzazione delle selezioni

$\Pi_{\text{NumTreno}, \text{OrarioPart}}(\sigma_{\text{Cat}='FR'}(\sigma_{\text{Stazione}='Vicenza'}(\text{TRENO} \bowtie \text{FERMATA})))$

Anticipo delle selezioni

$\Pi_{\text{NumTreno}, \text{OrarioPart}}(\sigma_{\text{Cat}='FR'}(\text{TRENO}) \bowtie \sigma_{\text{Stazione}='Vicenza'}(\text{FERMATA}))$

Figura 62: Esempio di applicazione delle trasformazioni

TRENO(NumTreno, OrarioPart, Cat, Dest, OrarioArr)
 FERMATA(NumTreno, Stazione, Orario)

Anticipo delle selezioni (riportato dal lucido precedente)

$\Pi_{\text{NumTreno}, \text{OrarioPart}}(\sigma_{\text{Cat}='FR'}(\text{TRENO}) \bowtie \sigma_{\text{Stazione}='Vicenza'}(\text{FERMATA}))$

Anticipo delle proiezioni

$\Pi_{\text{NumTreno}, \text{OrarioPart}}(\Pi_{\text{NumTreno}, \text{OrarioPart}}(\sigma_{\text{Cat}='FR'}(\text{TRENO})) \bowtie \Pi_{\text{NumTreno}}(\sigma_{\text{Stazione}='Vicenza'}(\text{FERMATA})))$

Figura 63: Esempio di applicazione delle trasformazioni

Ulteriori trasformazioni di equivalenza

Siano E_1 ed E_2 espressioni di schema rispettivamente X_1 e X_2 . Si definisce la seguente trasformazione di equivalenza.

Inglobamento di una selezione in un prodotto cartesiano

Questa trasformazione consente di inglobare una selezione applicata a un prodotto cartesiano in un join con condizione:

$$\sigma_F(E_1 \times E_2) \equiv E_1 \bowtie_F E_2$$

La trasformazione è applicabile solo se:

$$X_1 \cap X_2 = \emptyset$$

Si osservi che questa regola va applicata solo dopo aver verificato che non sia possibile anticipare le selezioni rispetto al join.

- Applicazione delle proprietà commutativa e associativa di: unione, prodotto cartesiano, intersezione.
- Applicazione della proprietà distributiva:

$$\sigma_F(E_1 \cup E_2) \equiv \sigma_F(E_1) \cup \sigma_F(E_2)$$

$$\sigma_F(E_1 - E_2) \equiv \sigma_F(E_1) - \sigma_F(E_2)$$

$$\Pi_Y(E_1 \cup E_2) \equiv \Pi_Y(E_1) \cup \Pi_Y(E_2)$$

$$E_1 \bowtie (E_2 \cup E_3) \equiv (E_1 \bowtie E_2) \cup (E_1 \bowtie E_3)$$

Figura 64: Applicazione della proprietà distributiva

Applicazione di altre trasformazioni:

$$\sigma_{F_1 \vee F_2}(E) \equiv \sigma_{F_1}(E) \cup \sigma_{F_2}(E)$$

$$\sigma_{F_1 \wedge F_2}(E) \equiv \sigma_{F_1}(E) \cap \sigma_{F_2}(E) \equiv \sigma_{F_1}(E) \bowtie \sigma_{F_2}(E)$$

$$\sigma_{F_1 \wedge \neg F_2}(E) \equiv \sigma_{F_1}(E) - \sigma_{F_2}(E)$$

Figura 65: applicazione di altre trasformazioni

TRENO(NumTreno, Cat, OraPart, MinPart, OraArr, MinArr, Dest)

FERMATA(NumTreno, Stazione, Ora, Min)

Q: Trovare la destinazione dei treni non regionali che fermano a "Peschiera".

Dimensioni dei dati:

TRENO:

#attributi = 7

#tuple = 150

#tuple treni regionali = 40

FERMATA:

#attributi = 4

#tuple = 1000

#tuple staz peschiera = 50

#tuple staz peschiera e treno
non regionale = 45

Figura 66: Esempio

8 Calcolo Relazionale

Il calcolo relazionale è un linguaggio di interrogazione dichiarativo: esso specifica le proprietà che devono essere soddisfatte dal risultato dell'interrogazione, senza descrivere le modalità operative per ottenerlo.

Esistono due versioni del calcolo relazionale:

- il calcolo relazionale sui domini;
- il calcolo relazionale sulle tuple.

Caratteristiche del linguaggio Nel calcolo relazionale sulle tuple, un'interrogazione è specificata mediante una formula logica con variabili libere. La formula viene interpretata sul contenuto della base di dati e le variabili assumono come valori le tuple delle relazioni presenti nella base di dati.

Il calcolo relazionale sulle tuple costituisce il fondamento teorico del linguaggio SQL nella sua forma *semplice*, ossia senza operatori aggregati e senza join esterni, includendo le interrogazioni nidificate.

FORMULE LOGICHE CON VARIABILI LIBERE

Esempi

$$F_1(x) \equiv x > 3$$

$$F_2(x, y) \equiv x > 3 \wedge x < 6 \wedge y = 10$$

$$F_3(x) \equiv x > 0 \wedge \exists y (y < 0)$$

$$F_4(x) \equiv x > 3 \wedge \exists y (y < x)$$

$$F_5(x) \equiv x < 10 \wedge \forall y (y < x)$$

$$F_6(x) \equiv x < 10 \vee \forall y (y > x)$$

Figura 67: Esempi di formule logiche con variabili libere

Interpretazione L'interpretazione consiste nella scelta degli insiemi da cui prelevare i valori da assegnare alle variabili. Ad esempio, una formula può essere interpretata sull'insieme degli interi $\mathbb{Z}$ oppure sull'insieme dei numeri naturali $\mathbb{N}$.

$$\begin{aligned}
F_1(x) &\equiv x > 3 \\
F_2(x, y) &\equiv x > 3 \wedge x < 6 \wedge y = 10 \\
F_3(x) &\equiv x > 0 \wedge \exists y (y < 0) \\
F_4(x) &\equiv x > 3 \wedge \exists y (y < x) \\
F_5(x) &\equiv x < 10 \wedge \forall y (y < x) \\
F_6(x) &\equiv x < 10 \vee \forall y (y > x)
\end{aligned}$$

Figura 68: Formule logiche con variabili libere

Sintassi Le espressioni del calcolo relazionale sulle tuple hanno la seguente forma:

$$\{ T \mid L \mid F \}$$

dove:

- **Target list** (T): definisce lo schema della relazione risultato e specifica come le tuple del risultato sono ottenute a partire dalle variabili libere.
- **Range list** (L): definisce le variabili libere e le associa alle relazioni della base di dati coinvolte nell'interrogazione.
- **Formula** (F): specifica la condizione logica che deve essere soddisfatta dalle variabili libere, ossia dalle tuple coinvolte nell'interrogazione.

8.1 Sintassi

Sintassi Target List La **target list** T è una lista di elementi separati da virgole:

$$e_1, \dots, e_n$$

dove ogni elemento e_i può essere:

- $Y : x.(Z)$, dove Y e Z sono sequenze di attributi con la stessa cardinalità e x è una variabile libera.
- $x.(Z)$, dove Z è una sequenza di attributi e x è una variabile libera (equivalente a $Z : x.(Z)$).
- $x.*$, dove x è una variabile libera; in questo caso gli attributi sono tutti quelli della relazione associata alla variabile x nella *range list* L .

Sintassi range list La **range list** L è una lista di elementi separati da virgole:

$$r_1, \dots, r_m$$

dove ogni elemento r_i ha la seguente forma:

$$x_j(R)$$

dove x_j è una variabile libera e R è il nome di una relazione dello schema della base di dati.

Per ogni variabile libera della formula F esiste uno e un solo elemento r_i nella *range list* L .

Sintassi della formula La formula F può essere:

Formula atomica

- $x.A \theta c$ oppure
- $x_1.A_1 \theta x_2.A_2$

dove x, x_1, x_2 sono variabili, A, A_1, A_2 sono attributi, c è una costante e θ è un operatore di confronto appartenente all'insieme:

$$\{=, \neq, >, <, \geq, \leq\}.$$

Formula non atomica

- se f_1 e f_2 sono formule, allora anche

$$f_1 \wedge f_2, \quad f_1 \vee f_2, \quad \neg f_1, \quad \neg f_2$$

sono formule;

- se f è una formula, allora anche

$$\forall x(R)(f) \quad \text{e} \quad \exists x(R)(f)$$

sono formule.

Notazione Variabili libere e legate nelle formule:

- Nelle formule atomiche tutte le variabili sono libere.
- Nelle formule $\forall x(f)$ e $\exists x(f)$, la variabile x è una variabile legata, mentre tutte le altre variabili mantengono lo stato (libere o legate) che avevano in f .
- Le congiunzioni, le disgiunzioni e la negazione non modificano l'insieme delle variabili libere o legate di una formula.

8.2 Semantica

Semantica Le formule vengono interpretate sull'istanza corrente della base di dati

$$db = \{r_1(x_1, \dots, x_n), \dots, r_m(x_1, \dots, x_n)\}.$$

Ogni formula contiene un certo numero di variabili libere

$$f(x_1, \dots, x_k).$$

Data una ennupla di tuple $(t_1, \dots, t_k)$, tale ennupla rende vera la formula quando, sostituendo le tuple $t_1, \dots, t_k$ alle variabili libere $x_1, \dots, x_k$, la formula risulta soddisfatta.

Nel caso delle formule atomiche, una formula del tipo

$$x_{i,A} \theta x_{j,B}$$

è vera sulle tuple $(t_1, \dots, t_k)$ se il confronto $t_{i,A} \theta t_{j,B}$ è soddisfatto; analogamente,

$$x_{i,A} \theta c$$

è vera se il confronto $t_{i,A} \theta c$ è soddisfatto, dove θ è un operatore di confronto.

Le formule non atomiche ottenute mediante i connettivi logici $\wedge, \vee, \neg, \Rightarrow$ sono vere secondo le usuali definizioni di verità dei connettivi logici.

Per quanto riguarda i quantificatori, la formula

$$\exists x_k (f)$$

è vera sulle tuple $(t_1, \dots, t_{k-1})$ se esiste almeno una tupla t_k nell'istanza di R_k tale che, sostituendo $(t_1, \dots, t_k)$ alle variabili $(x_1, \dots, x_k)$, la formula f risulta vera. Analogamente, la formula

$$\forall x_k (f)$$

è vera sulle tuple $(t_1, \dots, t_{k-1})$ se per ogni tupla t_k nell'istanza di R_k la formula f risulta vera quando si sostituiscono le tuple $(t_1, \dots, t_k)$ alle variabili $(x_1, \dots, x_k)$.

Interpretazione di un'espressione del calcolo come interrogazione Un'espressione del calcolo relazionale sulle tuple può essere interpretata come un'interrogazione della forma

$$Q = \{ Y_1 : x_1(Z_1), \dots, Y_k : x_k(Z_k) \mid x_1(R_1), \dots, x_n(R_n) \mid f(x_1, \dots, x_n) \}.$$

La valutazione dell'interrogazione Q produce una relazione risultato R tale che:

- lo schema di R è

$$Y = Y_1 \cup \dots \cup Y_k;$$

- R contiene tutte le tuple t su Y per le quali esiste una ennupla di tuple

$$(t_1, \dots, t_n) \in R_1 \times \dots \times R_n$$

che genera t e che, sostituita alle variabili libere $(x_1, \dots, x_n)$, rende vera la formula f .

8.3 Calcolo relazionale sulle tuple

Il calcolo relazionale sulle tuple non è equivalente né all'algebra relazionale né al calcolo relazionale sui domini, in quanto l'operatore di unione tra due relazioni non è esprimibile nel calcolo relazionale sulle tuple. Tuttavia, nel calcolo relazionale sulle tuple è possibile rappresentare sia l'intersezione sia la differenza tra due relazioni.

Differenza tra due relazioni R_1 e R_2 di schema $X = (A_1, \dots, A_n)$

$$R_1 - R_2 \equiv \{x.* \mid x(R_1) \mid \neg \exists y(R_2) (x.A_1 = y.A_1 \wedge \dots \wedge x.A_n = y.A_n)\}$$

Intersezione tra due relazioni R_1 e R_2 di schema $X = (A_1, \dots, A_n)$

$$R_1 \cap R_2 \equiv \{x.* \mid x(R_1) \mid \exists y(R_2) (x.A_1 = y.A_1 \wedge \dots \wedge x.A_n = y.A_n)\}$$

oppure

$$R_1 \cap R_2 \equiv \{x.* \mid x(R_1), y(R_2) \mid x.A_1 = y.A_1 \wedge \dots \wedge x.A_n = y.A_n\}$$

Figura 69: differenza e intersezione

Unione tra due relazioni R_1 e R_2 di schema $X = (A_1, \dots, A_n)$

$$R_1 \cup R_2 \equiv \{???? \mid x(R_1), y(R_2) \mid \text{true}\}$$

L'unione non è esprimibile in quanto non è possibile associare una variabile a più relazioni della base di dati.

Figura 70: unione

Esempio

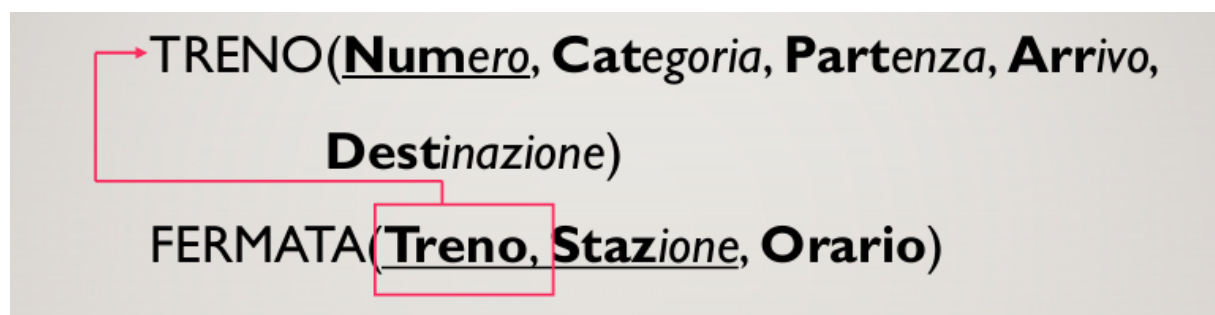


Figura 71: Progettazione Logica

TRENO(Num, Cat, Part, Arr, Dest)

FERMATA(Treno, Staz, Orario)

Q_0 : si richiede di restituire il numero e l'ora di partenza dei treni Freccia Rossa (Cat = 'FR') con destinazione 'Milano Centrale'

```
Q0 = {Numero, Orario_partenza: x.(Num, Part) |
      x (TRENO) |
      x.Cat='FR' ^ x.Dest='Milano Centrale' }
```

Figura 72: Q_0

TRENO(Num, Cat, Part, Arr, Dest)

FERMATA(Treno, Staz, Orario)

Q_1 : Trovare l'elenco di tutte le fermate dei treni regionali (Cat='REG'), riportando nel risultato il numero del treno e la stazione di fermata.

```
Q1 = {Numero, Fermata: y.(Treno, Staz) |
      x (TRENO) , y (FERMATA) |
      x.Cat='REG' ^ x.Num=y.Treno }
```

Figura 73: Q_1

TRENO(Num, Cat, Part, Arr, Dest)

FERMATA(Treno, Staz, Orario)

Q₂: Trovare l'elenco di tutti i treni Freccia Rossa che fermano a Venezia Mestre riportando: il numero del treno, l'orario di partenza e l'orario di arrivo a Venezia Mestre.

$$Q_2 \equiv \{ \text{Numero, Partenza: } x.(\text{Num, Part}), \text{ Arrivo: } y.(\text{Orario}) \mid \\ x(\text{TRENO}), y(\text{FERMATA}) \mid x.\text{Cat} = \text{'FR'} \wedge x.\text{Num} = y.\text{Treno} \wedge \\ y.\text{Staz} = \text{'Venezia Mestre'} \}$$

Figura 74: Q₂

TRENO(Num, Cat, Part, Arr, Dest)

FERMATA(Treno, Staz, Orario)

Q₃: Trovare la destinazione e l'orario di partenza dei treni che fermano a Padova.

$$Q_3 \equiv \{ \text{Destinazione, Partenza: } x.(\text{Dest, Part}) \mid \\ x(\text{TRENO}) \mid \exists y(\text{FERMATA}) (x.\text{Num} = y.\text{Treno} \wedge \\ y.\text{Staz} = \text{'Padova'}) \} \quad \text{oppure}$$

$$Q_3 \equiv \{ \text{Destinazione, Partenza: } x.(\text{Dest, Part}) \mid \\ x(\text{TRENO}), y(\text{FERMATA}) \mid x.\text{Num} = y.\text{Treno} \wedge \\ y.\text{Staz} = \text{'Padova'}) \}$$

Figura 75: Q₃

TRENO(Num, Cat, Part, Arr, Dest)

FERMATA(Treno, Staz, Orario)

Q₄: Trovare la destinazione e l'orario di partenza dei treni che non fermano a Padova.

$$Q_3 \equiv \{ \text{Destinazione, Partenza} : x.(\text{Dest}, \text{Part}) \mid \\ x(\text{TRENO}) \mid \neg \exists y (\text{FERMATA}) (x.\text{Num} = y.\text{Treno} \wedge \\ y.\text{Staz} = \text{'Padova'}) \}$$

Figura 76: Q_4

Il calcolo relazionale sulle tuple è equivalente al linguaggio di interrogazione adottato dallo standard SQL2, se si considera la forma base delle interrogazioni SQL, ovvero senza l'uso di operatori aggregati e di raggruppamenti. In tale contesto, ogni espressione del calcolo relazionale sulle tuple è direttamente traducibile in una interrogazione SQL di base.

In particolare, un'espressione del calcolo della forma

$$\{ T \mid L \mid F \}$$

è in diretta corrispondenza con la forma base delle interrogazioni SQL:

SELECT T **FROM** L **WHERE** F .

La *target list* T individua gli attributi da restituire nel risultato dell'interrogazione ed è mappata sulla clausola **SELECT**; la *range list* L specifica le relazioni coinvolte ed è mappata sulla clausola **FROM**; infine, la formula F esprime le condizioni di selezione ed è mappata sulla clausola **WHERE**.

9 Linguaggi di interrogazione nei sistemi NoSql

Nei sistemi NoSql spesso non sono disponibili veri e propri linguaggi di interrogazione come nei sistemi relazionali. Solo i sistemi document-based forniscono strumenti simili all'algebra e al calcolo relazionale per l'accesso ai dati.

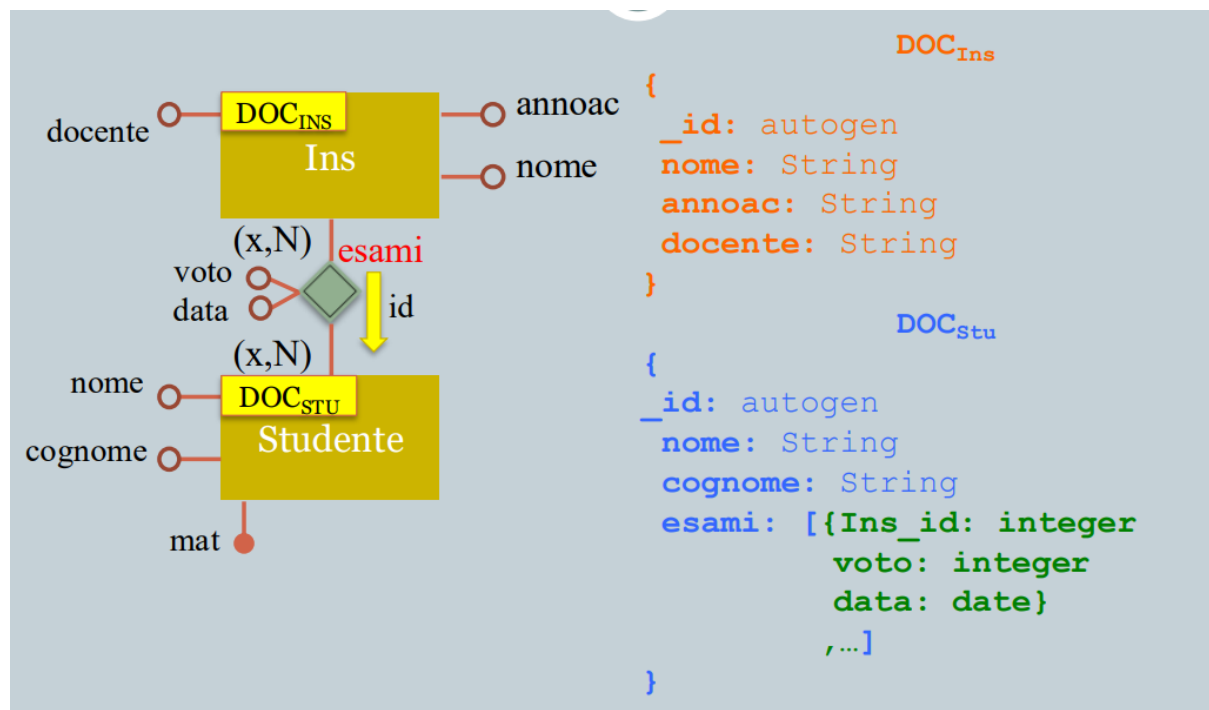


Figura 77: esempio sistemi document-store

La maggior parte del linguaggio di interrogazione di MongoDB è realizzato attraverso il metodo `find`.

```

db.studenti.insertMany([
  {_id: "VR00010", nome: "Mario", cognome: "Rossi"
   esami: [{ins: "Basi di dati", voto: 22, data: "1/7/2016" },
           {ins: "Algebra", voto: 26, data: "5/7/2015" }]}
  {_id: "VR00011", nome: "Maria", cognome: "Bianchi"
   esami: [{ins: "Algebra", voto: 30, data: "1/7/2016" }]}
]);

```

Figura 78: esempio con mongo db del document based di prima

- Trovare tutti gli studenti:
`db.studenti.find( {} )`
- Trovare tutti gli studenti di cognome “Rossi”:
`db.studenti.find( {cognome: "Rossi"} )`
- Trovare tutti gli studenti di cognome “Rossi” e nome “Mario”:
`db.studenti.find( {cognome: "Rossi", nome: "Mario"} )`
- Trovare tutti gli studenti di cognome “Rossi” o “Bianchi”:
`db.studenti.find( {cognome: {$in: ["Rossi", "Bianchi"]}} )`

Figura 79: Interrogazioni Semplici

Name	Description	
<code>\$and</code>	Joins query clauses with a logical AND returns all documents that match the conditions of both clauses.	<code>{ \$or: [{Cognome: "Rossi"}, {Cognome: "Bianchi"}] }</code>
<code>\$not</code>	Inverts the effect of a query expression and returns documents that do <i>not</i> match the query expression.	<code>{ \$and: [{Cognome: "Rossi"}, {Nome: "Paolo"}] }</code>
<code>\$nor</code>	Joins query clauses with a logical NOR returns all documents that fail to match both clauses.	
<code>\$or</code>	Joins query clauses with a logical OR returns all documents that match the conditions of either clause.	

Name	Description
<code>\$eq</code>	Matches values that are equal to a specified value.
<code>\$gt</code>	Matches values that are greater than a specified value.
<code>\$gte</code>	Matches values that are greater than or equal to a specified value.
<code>\$in</code>	Matches any of the values specified in an array.
<code>\$lt</code>	Matches values that are less than a specified value.
<code>\$lte</code>	Matches values that are less than or equal to a specified value.
<code>\$ne</code>	Matches all values that are not equal to a specified value.
<code>\$nin</code>	Matches none of the values specified in an array.


```
{ $and: [{Esami.voto: { $gte: 21 }}, {Esami.voto: { $lte: 26 }}]}
```

Figura 80: Operatori di Confronto

Interrogazioni su dati incapsulati

- Trovare tutti gli studenti che hanno registrato almeno un esame con voto 30:
`db.studenti.find( {esami.voto: 30} )`
- Trovare tutti gli studenti che hanno registrato almeno un esame con voto maggiore di 22:
`db.studenti.find( {esami.voto: { $gt : 22 } } )`

Figura 81: Esempio di interrogazioni con gli operatori

Interrogazioni con proiezione

- Trovare il cognome degli studenti di nome “Mario”:
`db.studenti.find( {nome: "Mario"}, {cognome: 1} )`
- Trovare il cognome degli studenti di nome “Mario” escludendo il campo `_id`:
`db.studenti.find({nome: "Mario"},
 {cognome: 1, _id: 0})`
- Trovare la matricola e i voti di tutti gli studenti:
`db.studenti.find( { }, {_id: 1, esami.voto: 1} )`

Figura 82: Interrogazioni con proiezione

- Trovare tutti gli studenti che hanno ottenuto un voto maggiore di 28 nell'appello del 30/10/2025:

```
db.studenti.find(  
  { esami: { $elemMatch: { data: '30/10/2025',  
                           voto: { $gt : 22 } } } }
```

Figura 83: condizioni multiple

10 Tecnologie - Transazioni

10.1 Tecnologia dei DBMS

Un **DBMS** (Database Management System), o *sistema di gestione di basi di dati*, è un sistema software progettato per gestire collezioni di dati che siano:

- **Grandi** (di dimensioni tali da richiedere gestione su memoria secondaria);
- **Condivise** (accessibili da più utenti o applicazioni contemporaneamente);
- **Persistenti** (i dati sopravvivono all'esecuzione dei programmi che li utilizzano).

Un DBMS garantisce:

- **Affidabilità** (integrità e recupero dei dati in caso di guasti);
- **Privatezza e sicurezza** (controllo degli accessi e gestione dei permessi).

Essendo un sistema software, un DBMS deve essere:

- **Efficiente**, ottimizzando l'uso delle risorse (tempo di risposta e spazio);
- **Efficace**, fornendo strumenti adeguati per la gestione e l'organizzazione dei dati.

Le prestazioni e la qualità complessiva del sistema dipendono in modo significativo dalla **corretta progettazione della base di dati**.

Un DBMS:

- Estende le funzionalità del *file system*;
- Gestisce collezioni di dati memorizzate in **memoria secondaria**;
- Fornisce meccanismi di accesso, manipolazione e controllo dei dati.

Base di dati: una collezione organizzata di dati, gestita e controllata da un DBMS.

10.2 Architettura su 3 livelli dei DBMS

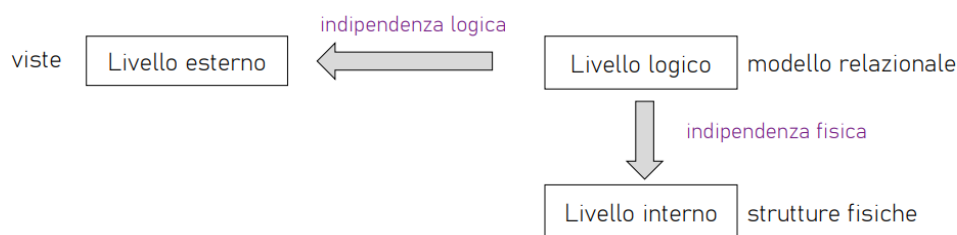


Figura 84: Architettura su 3 livelli dei DBMS

Un DBMS è organizzato secondo un'architettura a tre livelli: **esterno**, **logico** e **interno**. Il **livello esterno** descrive le viste degli utenti: ad esempio, uno studente può visualizzare solo i propri voti, mentre la segreteria può accedere ai dati di tutti gli studenti. Il **livello logico** definisce la struttura concettuale della base di dati nel modello relazionale, ad esempio la tabella **Studenti** (**Matricola**, **Nome**, **Cognome**, **Corso**), indipendentemente dalla memorizzazione fisica. Il **livello interno**,

infine, descrive come i dati sono effettivamente memorizzati su disco, ad esempio in file organizzati in pagine con eventuali indici per ottimizzare le ricerche.

Questa separazione garantisce l'**indipendenza logica**, per cui modifiche allo schema logico non richiedono cambiamenti nelle viste esterne, e l'**indipendenza fisica**, per cui variazioni nell'organizzazione fisica dei dati non alterano lo schema logico.

10.3 Transazione

Una **transazione** è un'unità logica di lavoro eseguita da un programma applicativo che interagisce con una base di dati. È costituita da una sequenza di operazioni che devono essere considerate come un unico blocco indivisibile.

Una transazione deve garantire proprietà di **correttezza**, **robustezza** e **isolamento**, in modo da mantenere la consistenza della base di dati anche in presenza di errori o accessi concorrenti.

La caratteristica fondamentale di una transazione è il principio *all-or-nothing*: essa o termina con successo (**commit**) producendo effetti permanenti sulla base di dati, oppure viene annullata (**abort/rollback**) senza lasciare alcuna modifica.

```
<transazione> → begin transaction
                <programma>
                end transaction

<programma> → {<istruzione> | commit work | rollback work}
               {<istruzione> | commit work | rollback work}
```

Figura 85: Sintassi di una transazione

Una **transazione** si dice *ben formata* se rispetta le seguenti condizioni:

- Inizia esplicitamente con un'istruzione **BEGIN TRANSACTION**.
- Termina con un'istruzione **END TRANSACTION**.
- Durante la sua esecuzione viene raggiunto un punto di terminazione esplicito, ovvero un'istruzione **COMMIT WORK** oppure **ROLLBACK WORK**.
- Dopo l'esecuzione di **COMMIT** o **ROLLBACK** non vengono effettuati ulteriori accessi o modifiche alla base di dati all'interno della stessa transazione.

```
begin transaction;  
    update CONTO set saldo = saldo - 1200  
        where filiale = '005' and numero = 15;  
    update CONTO set saldo = saldo + 1200  
        where filiale = '005' and numero = 205;  
    commit work;  
end transaction;
```

Figura 86: Esempio transazione ben formata

10.3.1 Proprietà delle transazioni

Una transazione in un DBMS deve rispettare quattro proprietà fondamentali, note come proprietà **ACID**:

- **Atomicità (Atomicity)**: una transazione è un'unità di esecuzione indivisibile. Essa viene eseguita completamente oppure non viene eseguita affatto.

Implicazioni:

- Se una transazione viene interrotta prima del **commit**, il lavoro eseguito fino a quel momento deve essere annullato, ripristinando lo stato della base di dati precedente all'inizio della transazione.
- Se una transazione viene interrotta dopo l'esecuzione del **commit** (commit completato con successo), il sistema deve garantire che gli effetti della transazione siano permanentemente registrati nella base di dati.

- **Consistenza (Consistency)**: l'esecuzione di una transazione non deve violare i vincoli di integrità della base di dati.

Implicazioni:

- **Verifica immediata**: il controllo dei vincoli viene effettuato durante l'esecuzione della transazione. Se un vincolo viene violato, viene annullata solo l'ultima operazione eseguita e il sistema restituisce all'applicazione una segnalazione di errore. L'applicazione può quindi reagire alla violazione.
- **Verifica differita**: il controllo dei vincoli viene effettuato al momento del **commit**. Se un vincolo di integrità risulta violato, l'intera transazione viene abortita senza possibilità per l'applicazione di reagire alla violazione.

- **Isolamento (Isolation)**: l'esecuzione di una transazione deve essere indipendente dalla contemporanea esecuzione di altre transazioni.

Implicazioni:

- Il rollback di una transazione non deve causare rollback a catena di altre transazioni che si trovano in esecuzione contemporaneamente.

- Il sistema deve regolare l'esecuzione concorrente tramite meccanismi di controllo dell'accesso alle risorse.
- **Persistenza (Durability)**: l'effetto di una transazione che ha eseguito il **commit** non deve andare perso.

Implicazioni:

- Il sistema deve essere in grado, in caso di guasto, di garantire gli effetti delle transazioni che al momento del guasto avevano già eseguito un **commit**.

10.4 Architettura di un DBMS

Un DBMS è composto da diversi moduli che gestiscono le varie funzionalità necessarie all'esecuzione delle transazioni.

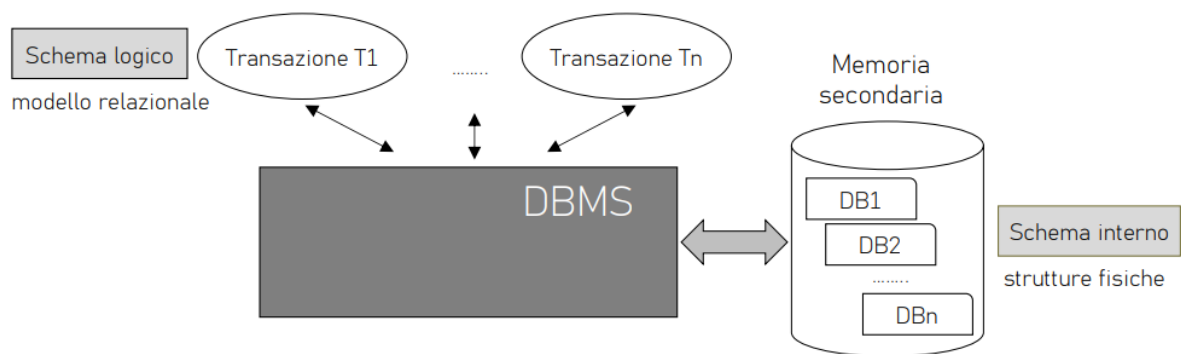


Figura 87: Architettura di riferimento di un DBMS

La figura rappresenta l'interazione tra le transazioni degli utenti, il DBMS e la memoria secondaria. Le transazioni ($T_1, \dots, T_n$) operano sullo **schema logico** del database secondo il modello relazionale e inviano le loro operazioni al DBMS. Il **DBMS** funge da intermediario, gestendo l'esecuzione delle transazioni e garantendo le proprietà di correttezza del sistema. I dati sono memorizzati nella **memoria secondaria** ($DB_1, DB_2, \dots, DB_n$) secondo lo **schema interno**, che descrive le strutture fisiche utilizzate per organizzare e memorizzare i dati.

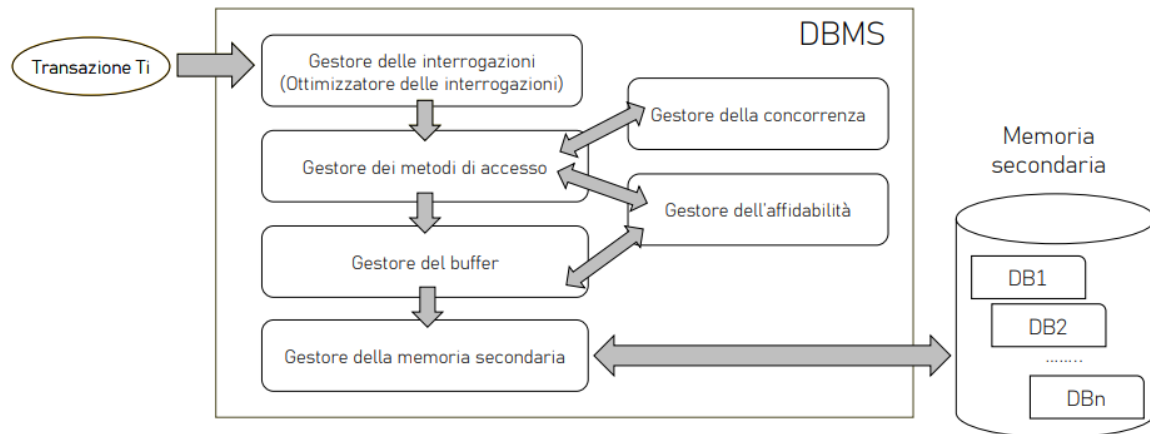


Figura 88: Moduli principali di un DBMS

La figura mostra i principali moduli che compongono l'architettura interna di un DBMS e il modo in cui collaborano durante l'esecuzione di una transazione.

Una transazione T_i invia le proprie interrogazioni al **gestore delle interrogazioni** (query processor), che analizza e ottimizza le query per determinare il piano di esecuzione più efficiente. Il **gestore dei metodi di accesso** stabilisce come accedere ai dati, utilizzando strutture come indici o scansioni delle tabelle.

Il **gestore del buffer** si occupa della gestione delle pagine di dati in memoria principale, mentre il **gestore della memoria secondaria** gestisce l'accesso fisico ai dati memorizzati su disco.

Il **gestore della concorrenza** controlla l'esecuzione simultanea di più transazioni per evitare interferenze, mentre il **gestore dell'affidabilità** garantisce la correttezza delle operazioni anche in caso di guasti, assicurando il mantenimento delle proprietà ACID.

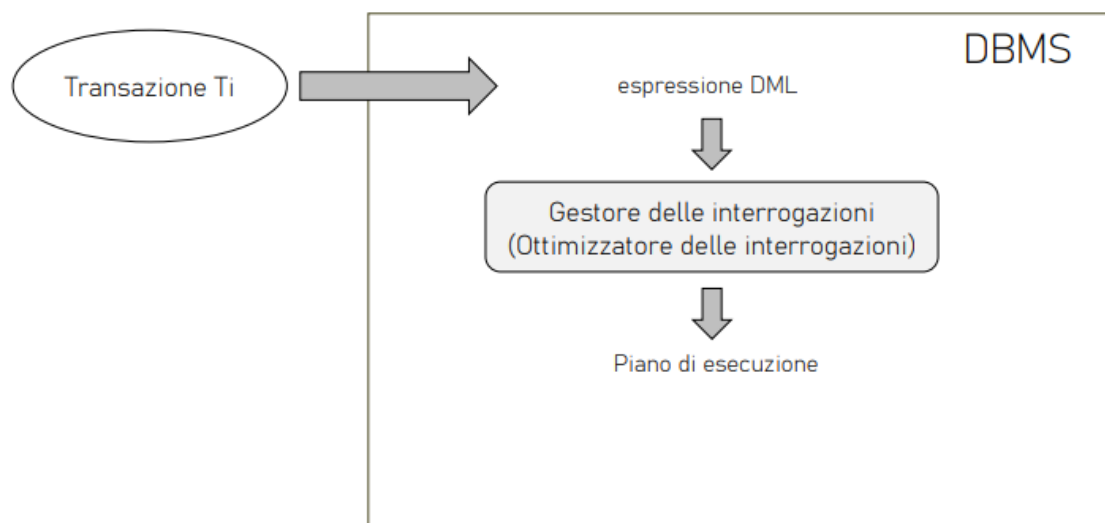


Figura 89: fase di elaborazione di una query

Una transazione T_i invia al DBMS un'espressione del linguaggio DML. La query viene elaborata dal **gestore delle interrogazioni** (query processor), che analizza e ottimizza l'interrogazione. Il risultato di questo processo è un **piano di esecuzione**, ovvero una sequenza di operazioni che il DBMS esegue per ottenere il risultato richiesto in modo efficiente.

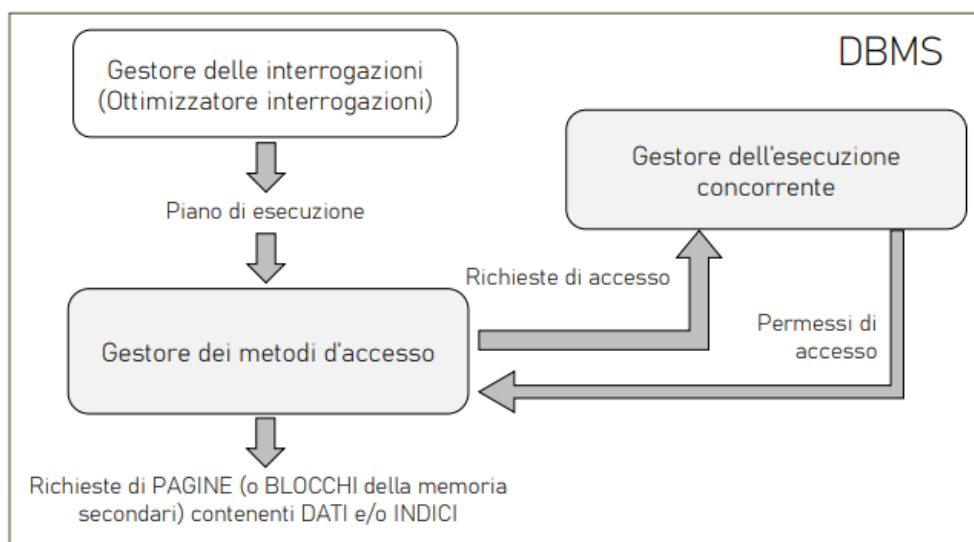


Figura 90: gestione delle interrogazioni

Il piano di esecuzione prodotto dal gestore delle interrogazioni viene eseguito dal gestore dei metodi di accesso, che richiede le pagine di dati o indici dalla memoria secondaria. Le richieste

di accesso ai dati vengono controllate dal gestore dell'esecuzione concorrente, che verifica la compatibilità con le altre transazioni in esecuzione e concede i permessi di accesso. In questo modo il DBMS garantisce l'esecuzione corretta delle transazioni concorrenti.

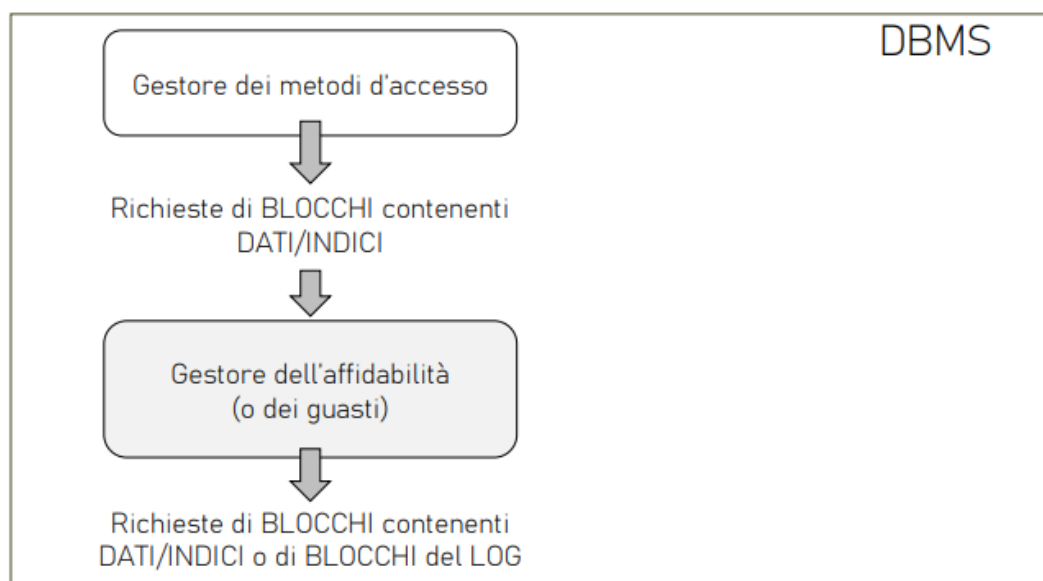


Figura 91: fase di recovery

Le richieste di accesso ai blocchi contenenti dati o indici, generate dal gestore dei metodi di accesso, vengono gestite dal **gestore dell'affidabilità** (recovery manager). Questo modulo ha il compito di garantire la correttezza del database anche in caso di guasti, utilizzando il **log delle transazioni**. Il log registra le operazioni eseguite dalle transazioni e permette al sistema di ripristinare lo stato corretto della base di dati attraverso operazioni di *redo* e *undo*.

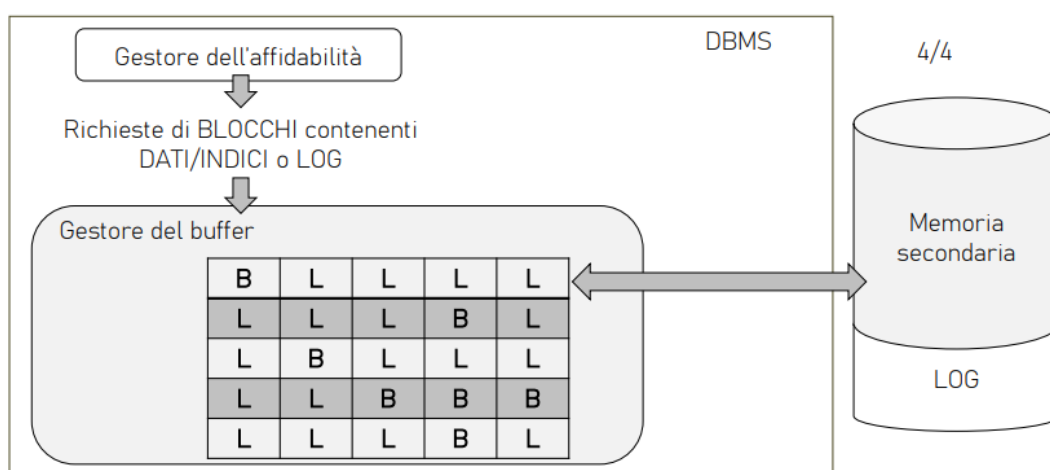


Figura 92: fase di gestione della memoria principale tramite il buffer manager quando accede ai dati sul disco

Le richieste di accesso ai blocchi contenenti dati, indici o record del log, generate dal gestore

dell'affidabilità, vengono gestite dal **gestore del buffer**. Questo modulo mantiene in memoria principale una copia delle pagine utilizzate più frequentemente, riducendo gli accessi alla memoria secondaria. Quando necessario, il buffer manager carica nuove pagine dal disco o scrive sul disco le pagine modificate.

- Quali moduli contribuiscono a garantire le proprietà delle transazioni?

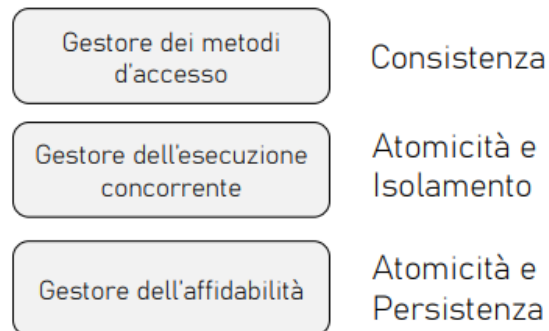


Figura 93:

11 Linguaggio di Interrogazione SQL

SQL (Structured Query Language) è il linguaggio di riferimento per la gestione delle **basi di dati relazionali**. In origine l'acronimo indicava *Structured Query Language*, mentre oggi viene considerato semplicemente un **nome proprio**.

SQL è un linguaggio molto completo che permette di svolgere diverse operazioni su una base di dati.

SQL consente di:

- Definire la struttura di una base di dati
- Modificare lo schema della base di dati
- Inserire, modificare e cancellare i dati
- Eseguire interrogazioni semplici
- Eseguire interrogazioni complesse (interrogazioni annidate, raggruppamenti, ecc.)

Per questo motivo SQL è diviso in diversi sottolinguaggi.

- **DDL (Data Definition Language)**: permette di definire la struttura della base di dati e i vincoli di integrità.
- **DML (Data Manipulation Language)**: permette di inserire, modificare o cancellare i dati.
- **DQL (Data Query Language)**: permette di interrogare la base di dati tramite query.

SQL è un **linguaggio dichiarativo**. Questo significa che l'utente specifica **cosa vuole ottenere**, ma non **come** il sistema deve eseguire l'operazione.

Ad esempio, quando si scrive una query SQL, si indica semplicemente il risultato desiderato.

```
SELECT nome FROM Studenti WHERE voto > 25
```

Il modo in cui la query viene eseguita è deciso automaticamente dal **DBMS**.

Una componente del DBMS chiamata **query optimizer** traduce la query SQL in operazioni interne più efficienti. Questo processo è nascosto all'utente e permette di esprimere interrogazioni in modo **astratto e ad alto livello**.

Nel tempo sono state definite diverse versioni dello standard SQL.

- **SQL-86**: prima standardizzazione del linguaggio
- **SQL-89**: introduzione dell'integrità referenziale
- **SQL-92 (SQL-2)**: estensione significativa del linguaggio
- **SQL:1999 (SQL-3)**: introduzione di concetti orientati agli oggetti, trigger e funzioni
- Versioni successive (2003, 2006, 2008, 2011, 2016): introduzione di nuove funzionalità come supporto XML, JSON e dati temporali

Le varie versioni dello standard hanno progressivamente ampliato le capacità del linguaggio, mantenendo comunque compatibilità con le versioni precedenti.

11.1 Clausola SELECT

Sintassi SELECT semplificata

```
SELECT [ DISTINCT ]
[ * | expression [[ AS] output_name ] [, ...] ]
[ FROM from_item [, ...] ]
[ WHERE condition ]
[ GROUP BY grouping_element [, ...] ]
[ HAVING condition [, ...] ]
[ { UNION | INTERSECT | EXCEPT } [ DISTINCT ] other_select ]
[ ORDER BY expression [ ASC | DESC | USING operator ]]
```

Figura 94: Sintassi SELECT

- **expression** è un'espressione che determina un attributo.
- **from_item** è un'espressione che determina una sorgente per gli attributi.
- **condition** è un'espressione booleana utilizzata per selezionare i valori degli attributi.
- **grouping_element** è un'espressione che permette di eseguire operazioni su più valori di un attributo e considerare il risultato complessivo.

```
SELECT ListaAttributi
FROM ListaTabelle
[WHERE Condizione]
```

Figura 95: Forma semplificata SELECT

- **SELECT**, **FROM** e **WHERE** sono chiamate **clausole**.
- La query estrae dal **prodotto cartesiano** delle tabelle elencate nella clausola **FROM** le righe che soddisfano la condizione specificata nella clausola **WHERE**.
- Il risultato dell'interrogazione è una **tabella** con una riga per ogni riga prodotta dalla clausola **FROM** ed eventualmente filtrata dalla clausola **WHERE** (se presente).
- Le colonne della tabella risultante dipendono dalla lista degli attributi specificata nella clausola **SELECT**.

Sintassi SELECT semplificata

```
SELECT [ DISTINCT ]
[ { * | expression [[ AS] output_name ] } [, ...] ]
```

Figura 96: DISTINCT

- ***** è un'abbreviazione utilizzata per indicare tutti gli attributi delle tabelle.

- **expression** è un'espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo).
- **output_name** è il nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione **expression** nella relazione risultato.
- **DISTINCT**: se presente richiede l'eliminazione delle tuple duplicate.
- **Q1: Estrarre lo stipendio degli impiegati di cognome 'Rossi'**

• **SELECT** Stipendio
FROM Impiegato
WHERE Cognome = 'Rossi';



Stipendio
45
80

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Figura 97: Esempio : Estrarre lo stipendio degli impiegati di cognome "Rossi"

SELECT Stipendio **as** Salario ← **alias**
FROM Impiegato
WHERE Cognome='Rossi'

Salario
45
80

- Restituisce lo stesso risultato della query precedente, ma **ridenominata** (temporaneamente) la colonna output Stipendio

Figura 98: Uso di AS (ridenominazione)

```
SELECT *
FROM Impiegato
WHERE Cognome='Rossi'
```

← Seleziona tutti gli attributi
della tabella che compaiono
nella clausola FROM

- L'uso dell'asterisco mi permette di non elencare il nome di tutte le colonne (è come se prioettassi su tutti gli attributi)

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Rossi	Direzione	14	80	Milano

Figura 99: Uso dell'asterisco

11.2 Clausola FROM

Sintassi
FROM from_item [, ...]

Figura 100: Sintassi FROM

dove **from_item** può essere una delle seguenti clausole (versione semplificata):

- **table_name** [[AS] alias [(column_alias [, ...])]]

Se sono presenti due o più tabelle, si esegue il **prodotto cartesiano** tra tutte le tabelle. Se ci sono attributi con lo stesso nome su tabelle diverse, nella clausola **SELECT** questi attributi sono identificati antepoendo il nome della tabella e un punto:

nomeTabella.nomeAttributo

- **(other_select)** [AS] nomeRisultato [(column_alias [, ...])]

È una **SELECT innestata**. Il risultato della SELECT interna viene trattato come una tabella temporanea con nome **nomeRisultato**, sulla quale viene eseguita la SELECT esterna.

- **from_item** [NATURAL] join_type from_item [ON condition]

È la clausola **JOIN**.

Rappresenta un metodo più sofisticato rispetto al caso 1), che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. Il parametro **join_type** specifica il tipo di join da utilizzare. I principali tipi di join sono descritti nelle slide successive.

- Q2: Estrarre i nomi ed il cognome degli impiegati e le città in cui lavorano

```
SELECT Impiegato.Nome, Impiegato.Cognome,
       Dipartimento.Città
FROM Impiegato, Dipartimento
WHERE Impiegato.Dipart = Dipartimento.Nome
```

Dipartimento	Nome	Indirizzo	Città
	Amministrazione	Via Tito Livio, 27	Milano
	Produzione	P.le Lavater, 3	Torino
	Distribuzione	Via Segre, 9	Roma
	Direzione	Via Tito Livio, 27	Milano
	Ricerca	Via Venosa, 6	Milano

Impiegato.Nome	Impiegato.Cognome	Dipartimento.Città
Mario	Rossi	Milano
Carlo	Bianchi	Torino
Giovanni	Verdi	Milano
Franco	Neri	Roma
Carlo	Rossi	Milano
Lorenzo	Gialli	Milano
Paola	Rosati	Milano
Marco	Franco	Torino

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Figura 101: Estrarre i nomi ed il cognome degli impiegati e le città in cui lavorano

- Q2: Estrarre i nomi ed il cognome degli impiegati e le città in cui lavorano

```
SELECT i.Nome, i.Cognome,
       d.Città
FROM Impiegato AS i, Dipartimento AS d
WHERE i.Dipart = d.Nome
```

uso dell'alias per abbreviare il nome delle tabelle

Dipartimento	Nome	Indirizzo	Città
	Amministrazione	Via Tito Livio, 27	Milano
	Produzione	P.le Lavater, 3	Torino
	Distribuzione	Via Segre, 9	Roma
	Direzione	Via Tito Livio, 27	Milano
	Ricerca	Via Venosa, 6	Milano

Impiegato.Nome	Impiegato.Cognome	Dipartimento.Città
Mario	Rossi	Milano
Carlo	Bianchi	Torino
Giovanni	Verdi	Milano
Franco	Neri	Roma
Carlo	Rossi	Milano
Lorenzo	Gialli	Milano
Paola	Rosati	Milano
Marco	Franco	Torino

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Figura 102: Stessa cosa di prima ma con AS

11.3 Clausola WHERE

Sintassi

WHERE condition

Figura 103: Sintassi WHERE

- La clausola **WHERE** ammette come argomento un'espressione booleana costruita combinando predicati semplici con gli operatori **AND**, **OR** e **NOT**.
- **AND**: seleziona le righe in cui tutti i predicati sono veri.
- **OR**: seleziona le righe in cui almeno un predicato è vero.
- **NOT**: seleziona le righe in cui il predicato è falso.

- Ciascun predicato semplice utilizza gli operatori =, <>, <, >, ≤ e ≥ per confrontare un'espressione costruita a partire dal valore di un attributo con un valore costante oppure con un'altra espressione.
- Per esprimere la precedenza tra gli operatori **AND** e **OR** si utilizzano le **parentesi**.
- Q3: Estrarre i nomi ed il cognome degli impiegati che lavorano nell'ufficio 20 del dipartimento Amministrazione.

• **SELECT** Nome, Cognome
FROM Impiegato
WHERE Ufficio = 20 **AND**
 Dipart = 'Amministrazione'



Nome	Cognome
Giovanni	Verdi

Dipartimento	Nome	Indirizzo	Città
Amministrazione	Produzione	Via Tito Livio, 27	Milano
	Distribuzione	P.le Lavater, 3	Torino
	Direzione	Via Segre, 9	Roma
	Ricerca	Via Tito Livio, 27	Milano
		Via Venosa, 6	Milano

Impiegato	Nome	Cognome	Dipart	Ufficio	Stipendio	Città
	Mario	Rossi	Amministrazione	10	45	Milano
	Carlo	Bianchi	Produzione	20	36	Torino
	Giovanni	Verdi	Amministrazione	20	40	Roma
	Franco	Neri	Distribuzione	16	45	Napoli
	Carlo	Rossi	Direzione	14	80	Milano
	Lorenzo	Gialli	Direzione	7	73	Genova
	Paola	Rosati	Amministrazione	75	40	Venezia
	Marco	Franco	Produzione	20	46	Roma

Figura 104: Estrarre i nomi ed il cognome degli impiegati che lavorano nell'ufficio 20 del dipartimento Amministrazione

- Q4: Estrarre i nomi ed il cognome degli impiegati che lavorano dipartimento Amministrazione o nel dipartimento Produzione.
- **SELECT** Nome, Cognome
FROM Impiegato
WHERE Dipart = 'Amministrazione'
OR Dipart = 'Produzione'



Nome	Cognome
Mario	Rossi
Carlo	Bianchi
Giovanni	Verdi
Paola	Rosati
Marco	Franco

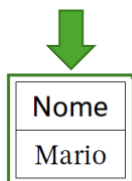
Dipartimento	Nome	Indirizzo	Città
Amministrazione	Produzione	Via Tito Livio, 27	Milano
	Distribuzione	P.le Lavater, 3	Torino
	Direzione	Via Segre, 9	Roma
	Ricerca	Via Tito Livio, 27	Milano
		Via Venosa, 6	Milano

Impiegato	Nome	Cognome	Dipart	Ufficio	Stipendio	Città
	Mario	Rossi	Amministrazione	10	45	Milano
	Carlo	Bianchi	Produzione	20	36	Torino
	Giovanni	Verdi	Amministrazione	20	40	Roma
	Franco	Neri	Distribuzione	16	45	Napoli
	Carlo	Rossi	Direzione	14	80	Milano
	Lorenzo	Gialli	Direzione	7	73	Genova
	Paola	Rosati	Amministrazione	75	40	Venezia
	Marco	Franco	Produzione	20	46	Roma

Figura 105: Estrarre i nomi ed il cognome degli impiegati che lavorano dipartimento Amministrazione o nel dipartimento Produzione.

- Q5: Estrarre i nomi degli impiegati di cognome Rossi che lavorano dipartimento Amministrazione o nel dipartimento Produzione.

```
SELECT Nome
FROM Impiegato
WHERE Cognome = 'Rossi' AND
(Dipart = 'Amministrazione' OR
Dipart = 'Produzione')
```



Dipartimento	Nome	Indirizzo	Città
Amministrazione		Via Tito Livio, 27	Milano
Produzione		P.le Lavater, 3	Torino
Distribuzione		Via Segre, 9	Roma
Direzione		Via Tito Livio, 27	Milano
Ricerca		Via Venosa, 6	Milano

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Figura 106: Estrarre i nomi degli impiegati di cognome Rossi che lavorano dipartimento Amministrazione o nel dipartimento Produzione.

11.3.1 Operatore LIKE

- Per il confronto tra stringhe, SQL mette a disposizione anche l'operatore **LIKE**, che consente di verificare se una stringa rispetta un certo **pattern** (oppure **ILIKE** per confronti *case insensitive*).
- Due caratteri speciali utilizzati nei pattern sono `_` e `%`:
 - `_`: rappresenta un singolo carattere arbitrario.
 - `%`: rappresenta una sequenza di lunghezza arbitraria (anche nulla) di caratteri arbitrari.
- Ad esempio:

`_o%i`

rappresenta una stringa che ha una **o** in seconda posizione, seguita da una sequenza qualsiasi di caratteri (anche vuota) e che termina con una **i**.

- Q6: Estrarre gli impiegati che hanno un cognome che ha una "o" in seconda posizione e finisce con una "i".

```
SELECT *
FROM Impiegato
WHERE Cognome LIKE '_o%i';
```

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Rossi	Direzione	14	80	Milano
Paola	Rosati	Amministrazione	75	40	Venezia

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Figura 107: Estrarre gli impiegati che hanno un cognome che ha una "o" in seconda posizione e finisce con una "i".

11.3.2 Operatore SIMILAR TO

L'operatore **SIMILAR TO** è una versione più espressiva dell'operatore **LIKE**. Permette di confrontare stringhe utilizzando un **pattern** basato su un sottoinsieme delle **espressioni regolari POSIX** nella versione prevista dallo standard SQL.

Questo operatore consente quindi di effettuare confronti più avanzati rispetto a **LIKE**, utilizzando operatori tipici delle espressioni regolari.

- **_** : rappresenta **un singolo carattere qualsiasi**.
- **%** : rappresenta **una sequenza di zero o più caratteri qualsiasi**.
- ***** : indica la **ripetizione del match precedente zero o più volte**.
- **+** : indica la **ripetizione del match precedente una o più volte**.
- **{n,m}** : indica la **ripetizione del match precedente almeno n volte e al massimo m volte**.
- **[...]** : rappresenta un **insieme di caratteri ammessi**; il match è valido se il carattere appartiene all'insieme specificato.

Esempio

Studenti con cognome che inizia con 'A' o 'B', o 'D', o 'N' e finisce con 'a':

```
SELECT cognome, nome, città FROM Studente
WHERE cognome SIMILAR TO '[ABDN]{1}%a';
```

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

Figura 108: Esempio SIMILAR TO

11.3.3 Operatore BETWEEN

L'operatore **BETWEEN** permette di verificare se il valore di un attributo è compreso tra due valori estremi.

Esempio					
Tutti gli studenti che hanno matricola tra 'IN0002' e 'IN0004'.					
<pre>SELECT cognome, nome, matricola FROM Studente WHERE matricola BETWEEN 'IN0002' AND 'IN0004';</pre>					
matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

Figura 109: Esempio Between

Esempio

Tutti gli studenti che hanno matricola tra 'IN0002' e 'IN0004'.

SELECT cognome, nome, matricola FROM Studente
WHERE matricola BETWEEN 'IN0002' AND 'IN0004';

matricola	cognome	nome	indirizzo	città	media	
IN0001	Ros	cognome	nome	matricola	na	27.50
IN0002	Paol	Rossi	Marco	IN0001	va	28.6666
IN0003	Bian	Paoli	Paolo	IN0002	NA	18.345
IN0004	Ros	Bianchi	Dante	IN0003	na	24.567
IN0005	Ver	Rossi	Matteo	IN0004		28.6666
IN0006	Nocasa	Caio				

Figura 110: Esempio 2 di Between

11.3.4 Operatore IN

Nella clausola **WHERE** può essere utilizzato l'operatore **[NOT] IN** per verificare se il valore di un attributo appartiene ad un insieme di valori.

Sintassi
<code>WHERE attributo [NOT] IN (valore [, ...]);</code>

Figura 111: Sintassi IN

Esempio					
Tutti gli studenti che hanno matricola nell'elenco 'IN0001', 'IN0003' e 'IN0005'.					
<pre>SELECT cognome, nome, matricola FROM Studente WHERE matricola IN('IN0001 ', 'IN0003 ', 'IN0005 ');</pre>					
matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

Figura 112: Esempio Operatore IN

11.3.5 Operatore IS NULL

Per verificare se un attributo ha un valore nullo oppure no si utilizza il predicato **IS NULL**.

- **attributo IS NULL**: restituisce **true** se l'attributo ha valore nullo.
- **attributo IS NOT NULL**: è la negazione del predicato precedente e restituisce **true** se l'attributo ha un valore diverso da NULL.

Sintassi
<code>WHERE attributo IS [NOT] NULL;</code>

Figura 113: Sintassi IS NULL

Esempio					
Tutti gli studenti che NON hanno una città.					
<pre>SELECT cognome, nome, città FROM Studente WHERE città IS NULL;</pre>					
matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

Figura 114: Esempio IS NULL

11.4 SQL e Algebra Relazionale

Vediamo ora alcuni esempi di interrogazioni in **SQL** e analizziamo la loro corrispondenza con le equivalenti interrogazioni procedurali espresse tramite l'**algebra relazionale**.

L'obiettivo è comprendere come le interrogazioni dichiarative scritte in SQL possano essere interpretate come una sequenza di operazioni dell'algebra relazionale, che descrivono in modo procedurale i passi necessari per ottenere il risultato desiderato.

- **SELECT** $T_1.\text{attributo}_{1l}, \dots, T_h.\text{attributo}_{hm}$
- **FROM** $\text{Tabella}_1 \text{ as } T_1, \dots, \text{Tabella}_h \text{ as } T_h$
- **WHERE** Condizione

$$\Pi_{T_1.\text{attributo}_{1l}, \dots, T_h.\text{attributo}_{hm}} (\sigma_{\text{Condizione}} (T_1 \bowtie \dots \bowtie T_h))$$

Figura 115: Sintassi algebra relazione e sql

Nel caso di condizioni più complesse, la formula di conversione tra una interrogazione **SQL** e la corrispondente espressione in **algebra relazionale** non è sempre direttamente rappresentabile.

Una condizione fondamentale per poter effettuare questa conversione è che l'interrogazione SQL utilizzi solo operatori che esistono anche nell'algebra relazionale.

Se l'interrogazione SQL utilizza operatori non presenti nell'algebra relazionale (ad esempio **operatori aggregati** come COUNT, SUM, AVG, ecc.), la conversione diretta non è possibile.

Maternità		Paternità		Persone		
Madre	Figlio	Padre	Figlio	Nome	Età	Reddito
Luisa	Maria	Sergio	Franco	Andrea	27	21
Luisa	Luigi	Luigi	Olga	Aldo	25	15
Anna	Olga	Luigi	Filippo	Maria	55	42
Anna	Filippo	Franco	Andrea	Anna	50	35
Maria	Andrea	Franco	Aldo	Filippo	26	30
Maria	Aldo			Luigi	50	40
				Franco	60	20
				Olga	30	41
				Sergio	85	35
				Luisa	75	87

Figura 116: Tabella usata per gli esempi

11.5 SELECT

- Nome e reddito delle persone con meno di trenta anni
 - $\Pi_{\text{Nome, Reddito}}(\sigma_{\text{Età} < 30}(\text{Persone}))$
 - `SELECT Nome, Reddito`
`FROM Persone`
`WHERE età < '30'`

Figura 117:

11.5.1 SELECT : alias di tabella o di attributo

ALIAS di Tabella

- `SELECT nome, reddito`
`FROM persone`
`WHRE Età < '30'`
- `SELECT p.nome, p.reddito`
`FROM persone as p`
`WHERE p.Età < '30'`

ALIAS di attributo

- `SELECT nome, reddito`
`FROM persone`
`WHRE Età < '30'`
- `SELECT p.nome as nomePersona,`
`p.reddito as redditoPersona`
`FROM persone as p`
`WHERE p.Età < '30'`

Figura 118: SELECT : alias di tabella o di attributo

11.5.2 SELECT: senza proiezione

- Nome, reddito ed età delle persone con meno di trenta anni
 - $\sigma_{\text{Età} < 30}(\text{Persone})$
 - `SELECT *`
`FROM Persone`
`WHERE età < '30'`

Figura 119: SELECT senza proiezione

11.5.3 Proiezione: gestione dei duplicati in SQL

Una differenza significativa tra **SQL** e l'**algebra relazionale** riguarda la gestione dei duplicati.

Nell'algebra relazionale i risultati delle operazioni sono considerati **insiemi** e quindi non possono contenere duplicati. In SQL, invece, i risultati sono trattati come **multinsiemi** (*bag*), per cui le righe duplicate possono essere presenti.

L'eliminazione delle righe duplicate è un'operazione computazionalmente costosa; per questo motivo, **SQL mantiene i duplicati per default**.

Il costrutto **DISTINCT**, utilizzato dopo la parola chiave **SELECT**, permette di eliminare le righe duplicate dal risultato.

Il costrutto **ALL** permette invece di mantenere esplicitamente i duplicati, anche se questa è già la scelta predefinita del linguaggio SQL.

Impiegati

Cognome	Nome	Filiale
Neri	Mario	Napoli
Neri	Giorgio	Napoli
Rossi	Claudio	Roma

- Trovare cognome e filiale di tutti gli impiegati

{(Neri, Napoli), (Rossi, Roma)}

- $\Pi_{\text{Cognome, Filiale}}(\text{Impiegati})$

Figura 120: Proiezione: gestione dei duplicati in SQL

Impiegati

Cognome	Filiale
Neri	Napoli
Neri	Napoli
Rossi	Roma

- **SELECT** Cognome, Filiale
FROM Impiegati

Impiegati

Cognome	Filiale
Neri	Napoli
Rossi	Roma

- **SELECT DISTINCT** Cognome, Filiale
FROM Impiegati

Figura 121:

- Q7: Estrarre le città delle persone il cui cognome è Rossi

• `SELECT Città`
`FROM Persona`
`WHERE Cognome = 'Rossi'`



Città
Verona
Verona
Milano

• `SELECT DISTINCT Città`
`FROM Persona`
`WHERE Cognome = 'Rossi'`



Città
Verona
Milano

Persona	CodFiscale	Nome	Cognome	Città
	RSSMRA55B21T234J	Mario	Rossi	Verona
	BNCCLR69T30H745Z	Carlo	Bianchi	Roma
	RSSGNN41A31B344C	Giovanni	Rossi	Verona
	RSSPRT75C12F205V	Pietro	Rossi	Milano

Figura 122: Esempio : gestione dei duplicati in SQL

11.5.4 SELECT: Selezione,Proiezione,Join

L'istruzione **SELECT** permette di esprimere diverse operazioni dell'algebra relazionale, a seconda della struttura dell'interrogazione.

- Con **una sola relazione** nella clausola **FROM** è possibile realizzare:
 - **selezioni** (σ)
 - **proiezioni** (π)
 - **ridenominazioni** (ρ), utilizzando il costrutto **AS**
- Con **più relazioni** nella clausola **FROM** si possono realizzare:
 - **join** ($\bowtie$)
 - **prodotti cartesiani**

Le principali corrispondenze tra le clausole SQL e le operazioni dell'algebra relazionale sono:

- **SELECT** $\rightarrow \pi$ **Proiezione** + ρ **Ridenominazione**
- **FROM** $\rightarrow \bowtie$ **Join** (prodotto cartesiano)
- **WHERE** $\rightarrow \sigma$ **Selezione** + $\bowtie$ **Join** (condizione)

11.6 Interrogazioni con JOIN

In **SQL** il **join** può essere definito in due modi diversi.

- Il primo metodo non richiede l'uso di una nuova sintassi: il legame tra le tabelle coinvolte viene specificato nella clausola **WHERE**.
- Il secondo metodo utilizza invece una sintassi dedicata all'interno della clausola **FROM**. In questo caso il join viene espresso esplicitamente tramite gli operatori di join.

Nel seguito verrà utilizzato il secondo approccio. La clausola **FROM** ammette come argomento:

`table_name [[AS] name] [, ...]`

Se nella clausola **FROM** sono presenti due o più nomi di tabelle, viene eseguito il **prodotto cartesiano** tra tutte le tabelle. Lo schema del risultato può quindi contenere tutti gli attributi derivanti dal prodotto cartesiano delle relazioni coinvolte.

Il prodotto cartesiano tra due o più tabelle è chiamato **CROSS JOIN**.

A partire dallo standard **SQL-2**, sono stati introdotti altri tipi di join (*join_type*), tra cui:

- **INNER JOIN**
- **LEFT [OUTER] JOIN**
- **RIGHT [OUTER] JOIN**
- **FULL [OUTER] JOIN**

Sintassi generale

```
table_name [ NATURAL ] join_type table_name [ON join_condition [, ...]].
```

Figura 123: Sintassi JOIN

dove *join_condition* è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato.

Le tuple selezionate possono essere successivamente filtrate mediante la condizione specificata nella clausola **WHERE**.